iZ-Library

0.0.1

Generated by Doxygen 1.16.1

1 iZprime 1
1.1 Table of Contents . . . 1
1.2 1. What the Library Provides . . . 2
1.2.1 Classical sieve algorithms . . . 2
1.2.2 SiZ family (iZ-space) . . . 3
1.2.3 Range operations . . . 3
1.2.4 Prime generation . . . 3
1.3 2. Core Design . . . 3
1.3.1 iZ index space . . . 3
1.3.2 iZm segmentation . . . 3
1.3.3 Deterministic + probabilistic stages . . . 4
1.4 3. Project Layout . . . 4
1.5 4. Dependencies . . . 4
1.5.1 Required . . . 4
1.5.2 Optional but recommended . . . 4
1.5.3 Install (macOS, Homebrew) . . . 4
1.5.4 Install (Ubuntu/Debian) . . . 4
1.6 5. Build and Run . . . 5
1.7 6. Public API Overview . . . 5
1.7.1 Input limits . . . 5
1.7.2 Return ownership rules . . . 5
1.7.3 Range API structure . . . 5
1.8 7. Data Structures . . . 6
1.8.1 BITMAP (include/bitmap.h) . . . 6
1.8.2 UI16_ARRAY, UI32_ARRAY, UI64_ARRAY (include/int_arrays.h) . . . 6
1.8.3 IZM and VX_SEG (include/iZ_toolkit.h) . . . 6
1.9 8. Toolkit Internals (for Extending) . . . 7
1.10 9. Usage Examples . . . 7
1.10.1 9.1 Prime sieving (library call) . . . 7
1.10.2 9.2 Range counting/streaming . . . 7
1.10.3 9.3 Random prime generation . . . 8
1.10.4 9.4 Build and run included examples . . . 8
1.11 10. Testing . . . 8
1.12 11. Benchmarking . . . 8
1.13 12. Doxygen Documentation . . . 9
1.14 13. Logging and Output Files . . . 9
1.15 14. Extension Guidelines . . . 9
1.16 15. Troubleshooting . . . 9
1.16.1 Linker errors for GMP/OpenSSL . . . 9
1.16.2 doxygen: command not found . . . 9
1.16.3 Slow or memory-heavy benchmark runs . . . 9
1.17 16. Future Work . . . 10
1.17.1 16.1 Segmented Sieve of Atkin (Segmented SoA) . . . 10

1.17.2 16.2 Sorenson's Pseudosquares Prime Sieve . . . 10
1.17.3 16.3 Pritchard-inspired Wheel Sieve Direction . . . 10
1.18 17. License . . . 10

2 iZprime User Manual 11
2.1 Scope . . . 11
2.2 Architecture Layers . . . 11
2.3 Algorithm Families . . . 11
2.4 Memory Ownership . . . 11
2.5 Extending the Library . . . 12
2.6 Building Documentation . . . 12

3 Topic Index 13
3.1 Topics . . . 13

4 Data Structure Index 15
4.1 Data Structures . . . 15

5 File Index 17
5.1 File List . . . 17

6 Topic Documentation 19
6.1 Bitmap Module . . . 19
6.1.1 Detailed Description . . . 20
6.1.2 Function Documentation . . . 21
6.1.2.1 bitmap_clear_all() . . . 21
6.1.2.2 bitmap_clear_bit() . . . 21
6.1.2.3 bitmap_clear_steps() . . . 22
6.1.2.4 bitmap_clear_steps_simd() . . . 22
6.1.2.5 bitmap_clone() . . . 23
6.1.2.6 bitmap_compute_hash() . . . 24
6.1.2.7 bitmap_flip_bit() . . . 24
6.1.2.8 bitmap_fread() . . . 25
6.1.2.9 bitmap_free() . . . 26
6.1.2.10 bitmap_fwrite() . . . 26
6.1.2.11 bitmap_get_bit() . . . 27
6.1.2.12 bitmap_init() . . . 28
6.1.2.13 bitmap_set_all() . . . 29
6.1.2.14 bitmap_set_bit() . . . 29
6.1.2.15 bitmap_validate_hash() . . . 30
6.1.2.16 TEST_BITMAP() . . . 31
6.2 Integer Arrays . . . 31
6.2.1 Detailed Description . . . 33
6.2.2 Function Documentation . . . 33
6.2.2.1 TEST_GENERIC_INT_ARRAYS() . . . 33

6.3 iZ Public API . . . 34
6.3.1 Detailed Description . . . 35
6.3.2 Macro Definition Documentation . . . 35
6.3.2.1 N_LIMIT . . . 35
6.3.3 Function Documentation . . . 36
6.3.3.1 iZ_next_prime() . . . 36
6.3.3.2 SiZ() . . . 37
6.3.3.3 SiZ_count() . . . 38
6.3.3.4 SiZ_stream() . . . 39
6.3.3.5 SiZm() . . . 40
6.3.3.6 SiZm_vy() . . . 41
6.3.3.7 SoA() . . . 41
6.3.3.8 SoE() . . . 42
6.3.3.9 SoEu() . . . 43
6.3.3.10 SoS() . . . 44
6.3.3.11 SSoE() . . . 45
6.3.3.12 vx_random_prime() . . . 45
6.3.3.13 vy_random_prime() . . . 46
6.4 Toolkit (iZ/iZm) . . . 47
6.4.1 Detailed Description . . . 49
6.4.2 Macro Definition Documentation . . . 49
6.4.2.1 MR_ROUNDS . . . 49
6.4.2.2 VX2 . . . 49
6.4.2.3 VX3 . . . 49
6.4.2.4 VX4 . . . 50
6.4.2.5 VX5 . . . 50
6.4.2.6 VX6 . . . 50
6.4.2.7 VX7 . . . 50
6.4.2.8 VX8 . . . 50
6.4.3 Function Documentation . . . 51
6.4.3.1 compute_l2_vx() . . . 51
6.4.3.2 compute_max_vx() . . . 51
6.4.3.3 compute_vx_k() . . . 52
6.4.3.4 get_root_primes() . . . 52
6.4.3.5 iZ() . . . 53
6.4.3.6 iZ_mpz() . . . 53
6.4.3.7 iZm_clone() . . . 54
6.4.3.8 iZm_construct_vx_base() . . . 55
6.4.3.9 iZm_free() . . . 55
6.4.3.10 iZm_init() . . . 56
6.4.3.11 iZm_solve_for_xp() . . . 56
6.4.3.12 iZm_solve_for_xp_mpz() . . . 57
6.4.3.13 iZm_solve_for_yp() . . . 59

6.4.3.14 process_iZ_bitmaps() . . . 60
6.4.3.15 vx_collect_p_gaps() . . . 61
6.4.3.16 vx_free() . . . 61
6.4.3.17 vx_full_sieve() . . . 61
6.4.3.18 vx_init() . . . 62
6.4.3.19 vx_search_prime() . . . 63
6.4.3.20 vx_stream_file() . . . 64
6.4.3.21 vy_search_prime() . . . 64
6.5 Logging . . . 65
6.5.1 Detailed Description . . . 66
6.5.2 Macro Definition Documentation . . . 66
6.5.2.1 LOG_DIR . . . 66
6.5.2.2 LOG_FILE . . . 67
6.5.2.3 LOG_MAX_SIZE . . . 67
6.5.2.4 LOGGER_FORMAT_PRINTF . . . 67
6.5.3 Enumeration Type Documentation . . . 67
6.5.3.1 LogLevel . . . 67
6.5.4 Function Documentation . . . 68
6.5.4.1 log_console() . . . 68
6.5.4.2 log_debug() . . . 68
6.5.4.3 log_error() . . . 68
6.5.4.4 log_fatal() . . . 69
6.5.4.5 log_info() . . . 69
6.5.4.6 log_init() . . . 69
6.5.4.7 log_level_to_string() . . . 70
6.5.4.8 log_message() . . . 70
6.5.4.9 log_message_extended() . . . 70
6.5.4.10 log_set_log_level() . . . 71
6.5.4.11 log_test() . . . 71
6.5.4.12 log_test_summary() . . . 72
6.5.4.13 log_warn() . . . 73
6.6 Tests and Benchmarks . . . 73
6.6.1 Detailed Description . . . 74
6.6.2 Typedef Documentation . . . 74
6.6.2.1 SIEVE_FN . . . 74
6.7 Utilities . . . 74
6.7.1 Detailed Description . . . 75
6.7.2 Macro Definition Documentation . . . 75
6.7.2.1 DIR_output . . . 75
6.7.2.2 MAX_CORES . . . 75
6.7.3 Function Documentation . . . 76
6.7.3.1 create_dir() . . . 76
6.7.3.2 gcd() . . . 76

6.7.3.3 get_cpu_cores_count() . . . 77
6.7.3.4 get_cpu_L2_cache_size_bits() . . . 77
6.7.3.5 gmp_seed_randstate() . . . 78
6.7.3.6 is_numeric_str() . . . 78
6.7.3.7 modular_inverse() . . . 79
6.7.3.8 print_centered_text() . . . 79
6.7.3.9 print_line() . . . 80
6.7.3.10 print_module_header() . . . 81
6.7.3.11 print_sha256_hash() . . . 81
6.7.3.12 print_test_header() . . . 81

7 Data Structure Documentation 83
7.1 BITMAP Struct Reference . . . 83
7.1.1 Detailed Description . . . 83
7.1.2 Field Documentation . . . 83
7.1.2.1 byte_size . . . 83
7.1.2.2 data . . . 84
7.1.2.3 sha256 . . . 84
7.1.2.4 size . . . 84
7.2 INPUT_SIEVE_RANGE Struct Reference . . . 85
7.2.1 Detailed Description . . . 85
7.2.2 Field Documentation . . . 85
7.2.2.1 filepath . . . 85
7.2.2.2 mr_rounds . . . 85
7.2.2.3 range . . . 86
7.2.2.4 start . . . 86
7.3 IZM Struct Reference . . . 86
7.3.1 Detailed Description . . . 86
7.3.2 Field Documentation . . . 87
7.3.2.1 base_x5 . . . 87
7.3.2.2 base_x7 . . . 87
7.3.2.3 k_vx . . . 87
7.3.2.4 root_primes . . . 87
7.3.2.5 vx . . . 87
7.4 SIEVE_LIMIT Struct Reference . . . 88
7.4.1 Detailed Description . . . 88
7.4.2 Field Documentation . . . 88
7.4.2.1 base . . . 88
7.4.2.2 exp . . . 88
7.5 SIEVE_MODEL Struct Reference . . . 88
7.5.1 Detailed Description . . . 89
7.5.2 Field Documentation . . . 89
7.5.2.1 function . . . 89

7.5.2.2 name . . . 89
7.6 UI16_ARRAY Struct Reference . . . 89
7.6.1 Detailed Description . . . 89
7.6.2 Field Documentation . . . 90
7.6.2.1 array . . . 90
7.6.2.2 capacity . . . 90
7.6.2.3 count . . . 90
7.6.2.4 sha256 . . . 90
7.7 UI32_ARRAY Struct Reference . . . 90
7.7.1 Detailed Description . . . 91
7.7.2 Field Documentation . . . 91
7.7.2.1 array . . . 91
7.7.2.2 capacity . . . 91
7.7.2.3 count . . . 91
7.7.2.4 sha256 . . . 91
7.8 UI64_ARRAY Struct Reference . . . 91
7.8.1 Detailed Description . . . 92
7.8.2 Field Documentation . . . 92
7.8.2.1 array . . . 92
7.8.2.2 capacity . . . 92
7.8.2.3 count . . . 92
7.8.2.4 sha256 . . . 93
7.9 VX_SEG Struct Reference . . . 93
7.9.1 Detailed Description . . . 93
7.9.2 Field Documentation . . . 93
7.9.2.1 bit_ops . . . 93
7.9.2.2 end_x . . . 94
7.9.2.3 is_large_limit . . . 94
7.9.2.4 mr_rounds . . . 94
7.9.2.5 p_count . . . 94
7.9.2.6 p_gaps . . . 94
7.9.2.7 p_test_ops . . . 95
7.9.2.8 root_limit . . . 95
7.9.2.9 start_x . . . 95
7.9.2.10 vx . . . 95
7.9.2.11 x5 . . . 95
7.9.2.12 x7 . . . 96
7.9.2.13 y . . . 96
7.9.2.14 yvx . . . 96

8 File Documentation 97
8.1 include/bitmap.h File Reference . . . 97
8.1.1 Detailed Description . . . 98

8.2 bitmap.h . . . 98
8.3 include/int_arrays.h File Reference . . . 99
8.3.1 Detailed Description . . . 101
8.4 int_arrays.h . . . 101
8.5 include/iZ_api.h File Reference . . . 102
8.5.1 Detailed Description . . . 103
8.6 iZ_api.h . . . 104
8.7 include/iZ_toolkit.h File Reference . . . 104
8.7.1 Detailed Description . . . 106
8.8 iZ_toolkit.h . . . 106
8.9 include/logger.h File Reference . . . 107
8.9.1 Detailed Description . . . 109
8.10 logger.h . . . 109
8.11 include/test_api.h File Reference . . . 110
8.11.1 Detailed Description . . . 111
8.12 test_api.h . . . 111
8.13 include/utils.h File Reference . . . 112
8.13.1 Detailed Description . . . 113
8.14 utils.h . . . 113
8.15 src/iZ_apps.c File Reference . . . 114
8.15.1 Detailed Description . . . 114
8.16 iZ_apps.c . . . 115
8.17 src/prime_sieve.c File Reference . . . 125
8.17.1 Detailed Description . . . 126
8.17.2 Function Documentation . . . 126
8.17.2.1 est_pi_n() . . . 126
8.17.2.2 process_eratosthenes_bitmap() . . . 126
8.18 prime_sieve.c . . . 127
8.19 src/toolkit/bitmap.c File Reference . . . 133
8.19.1 Detailed Description . . . 134
8.19.1.1 Implementation Notes . . . 134
8.19.1.2 Performance Characteristics . . . 134
8.20 bitmap.c . . . 135
8.21 src/toolkit/int_arrays.c File Reference . . . 139
8.21.1 Detailed Description . . . 139
8.21.1.1 Implementation Notes . . . 140
8.21.1.2 Performance Characteristics . . . 140
8.22 int_arrays.c . . . 141
8.23 src/toolkit/iZ_toolkit.c File Reference . . . 141
8.23.1 Detailed Description . . . 143
8.23.2 Function Documentation . . . 143
8.23.2.1 compute_k_vx() . . . 143
8.23.2.2 vx_det_sieve() . . . 143

8.23.2.3 vx_prob_sieve() 144
8.23.2.4 vx_set_base_values() 144
8.23.3 Variable Documentation 144
8.23.3.1 s_primes 144
8.23.3.2 s_primes_count 145
8.24 iZ_toolkit.c 145
8.25 src/toolkit/logger.c File Reference 155
8.25.1 Detailed Description 155
8.26 logger.c 155
8.27 src/toolkit/utils.c File Reference 158
8.27.1 Detailed Description 159
8.28 utils.c 159

9 Examples 163
9.1 /Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c 163

Index 169

Chapter 1

iZprime

iZprime is a performance-oriented C library for prime sieving and prime generation, for whom might be interested in prime sieving algorithms or algorithmic design in general.

At a glance, the library gives you:

- classical sieve implementations and comparison studies,
- the SiZ family (SiZ, SiZm, SiZm_vy) built on the 6x +/- 1 index space,
- cache-aware segmented execution for large-limit enumeration,
- multiprocessing range counting/streaming,
- optimized large random prime generation strategies for cryptographic applications.

The library is structured as an extensible toolkit, not just a set of endpoints: core data structures and iZ/iZm internals are available for research extensions.

The algorithm design and pseudocode are documented in docs/pseudocode.pdf and correspond to the implemented code paths.

1.1 Table of Contents

- iZprime
  - Table of Contents
  - 1. What the Library Provides
    - Classical sieve algorithms
    - SiZ family (iZ-space)
    - Range operations
    - Prime generation
  - 2. Core Design
    - iZ index space
    - iZm segmentation
    - Deterministic + probabilistic stages
  - 3. Project Layout

- 4. Dependencies
  - Required
  - Optional but recommended
  - Install (macOS, Homebrew)
  - Install (Ubuntu/Debian)
- 5. Build and Run
- 6. Public API Overview
  - Input limits
  - Return ownership rules
  - Range API structure
- 7. Data Structures
  - `BITMAP` (`include/bitmap.h`)
  - `UI16_ARRAY`, `UI32_ARRAY`, `UI64_ARRAY` (`include/int_arrays.h`)
  - `IZM` and `VX_SEG` (`include/iZ_toolkit.h`)
- 8. Toolkit Internals (for Extending)
- 9. Usage Examples
  - 9.1 Prime sieving (library call)
  - 9.2 Range counting/streaming
  - 9.3 Random prime generation
  - 9.4 Build and run included examples
- 10. Testing
- 11. Benchmarking
- 12. Doxygen Documentation
- 13. Logging and Output Files
- 14. Extension Guidelines
- 15. Troubleshooting
  - Linker errors for GMP/OpenSSL
  - `doxygen: command not found`
  - Slow or memory-heavy benchmark runs
- 16. Future Work
  - 16.1 Segmented Sieve of Atkin (Segmented SoA)
  - 16.2 Sorenson's Pseudosquares Prime Sieve
  - 16.3 Pritchard-inspired Wheel Sieve Direction
- 17. License

1.2 1. What the Library Provides

1.2.1 Classical sieve algorithms

The library provides modern implementations of several classic sieve algorithms:

- SoE : Classic Sieve of Eratosthenes
- SSoE : Segmented Sieve of Eratosthenes
- SoEu : Sieve of Euler (linear sieve)
- SoS : Sieve of Sundaram
- SoA : Sieve of Atkin

1.2.2 SiZ family (iZ-space)

The SiZ family is a fresh wheel-based optimization to the classic Sieve of Eratosthenes, it combines multiple wheel structures to optimize and reduce the primes search space. It includes:

- SiZ : Solid Sieve-iZ on 6x ± 1
- SiZm : Segmented Sieve-iZm (horizontal sieving)
- SiZm_vy : Semi-segmented Sieve-iZm (vertical sieving, unordered output)

1.2.3 Range operations

- SiZ_stream : stream primes in [start, start + range - 1] to file and return count
- SiZ_count : count primes in range using multiple processes

1.2.4 Prime generation

- vx_random_prime : random prime search via horizontal strategy
- vy_random_prime : random prime search via vertical strategy
- iZ_next_prime : next/previous prime near an arbitrary base value

1.3 2. Core Design

1.3.1 iZ index space

The SiZ family works in index space over values of form: 6x ± 1

This removes all multiples of 2 and 3 from the candidate domain and reduces work compared with integer-domain sieving.

From the pseudocode analysis (docs/pseudocode.pdf):

- SiZ uses two bitmaps for 6x-1 and 6x+1 lanes.
- Composite marking is driven by the Xp-Identity in index space.
- The resulting sieve space is approximately O(N/3) (vs O(N) for classic SoE).

1.3.2 iZm segmentation

SiZm applies segmented sieving with a VX segment width. VX is chosen from primorial-like products (VX2..VX8), and base segment bitmaps are pre-sieved then reused across segments.

This design is specifically intended to:

- keep active segments cache-resident (L2-aware VX sizing),
- pre-sieve small expensive primes once and amortize that work across segments,
- scale efficiently for high limits while controlling memory overhead.

1.3.3 Deterministic + probabilistic stages

For large numeric ranges, deterministic composite marking is followed by Miller-Rabin checks where needed.

1.4 3. Project Layout

```
include/
  iZ_api.h        # public API
  iZ_toolkit.h    # iZ/iZm internal toolkit
  bitmap.h        # packed bitmaps
  int_arrays.h    # dynamic integer arrays
  utils.h         # shared utilities
  logger.h        # logging API
  test_api.h      # test/benchmark entrypoints

src/
  prime_sieve.c   # classical + SiZ sieve implementations
  iZ_apps.c       # range APIs + random-prime APIs
  toolkit/        # bitmap/arrays/utils/logger/toolkit internals

examples/
  sieve_primes.c  # sieve CLI example
  SiZ_range.c     # range count/stream example
  p_genrators.c   # random-prime and next-prime example

test/
  test_all.c      # test runner CLI

docs/
  pseudocode.pdf  # algorithm pseudocode documentation
  ...
```

1.5 4. Dependencies

1.5.1 Required

- C compiler: GCC or Clang
- make
- GMP (gmp)
- OpenSSL 3 (ssl, crypto)

1.5.2 Optional but recommended

- pkg-config (Makefile prefers it for portable dependency discovery)
- Python + py_tools/requirements.txt for plotting benchmark outputs
- Doxygen for API/manual generation

1.5.3 Install (macOS, Homebrew)

```
brew install gcc make pkg-config gmp openssl@3 doxygen
```

1.5.4 Install (Ubuntu/Debian)

```
sudo apt-get update
sudo apt-get install -y build-essential make pkg-config libgmp-dev libssl-dev doxygen
```

1.6 5. Build and Run

From repository root:

```
make
```

Notes:

- make builds the main binary at build/src/iZ and then runs it.
- Main program output is appended to output/stdout.txt.

Useful build variants:

```
make debug
make release
make clean
make help
```

1.7 6. Public API Overview

Public header: include/iZ_api.h

1.7.1 Input limits

- Most sieve entry points expect: 10 < n <= 10^12.
- N_LIMIT is defined as 1000000000000ULL.

1.7.2 Return ownership rules

- Functions returning UI64_ARRAY * return heap-owned arrays.
- Caller must release with:

```
ui64_free(&arr);
```

1.7.3 Range API structure

```
typedef struct {
    char *start;       // decimal string
    uint64_t range;    // interval size
    int mr_rounds;     // Miller-Rabin rounds
    char *filepath;    // optional stream output path
} INPUT_SIEVE_RANGE;
```

- SiZ_stream(&input) uses filepath to stream primes.
- SiZ_count(&input, cores) ignores streaming and returns count only.

1.8 7. Data Structures

1.8.1 BITMAP (include/bitmap.h)

Packed bit-array with:

- size, byte_size, data
- optional SHA-256 checksum (sha256)

Key operations:

- single-bit set/get/flip/clear
- bulk set/clear
- stepped clearing for sieve marking (bitmap_clear_steps, SIMD variant)
- binary I/O + checksum verification

1.8.2 UI16_ARRAY, UI32_ARRAY, UI64_ARRAY (include/int_arrays.h)

Dynamic arrays with:

- capacity/count tracking
- push/pop/resize
- optional SHA-256 verification
- binary read/write helpers

1.8.3 IZM and VX_SEG (include/iZ_toolkit.h)

Internal SiZm state:

- IZM: base VX configuration, root primes, pre-sieved base bitmaps
- VX_SEG: one segment runtime state (bitmaps, boundaries, prime counts, operations, optional prime-gap vector)

1.9 8. Toolkit Internals (for Extending)

Internal toolkit header: include/iZ_toolkit.h

Key extension points:

- iZ mapping helpers: iZ, iZ_mpz
- VX selection: compute_vx_k, compute_l2_vx, compute_max_vx
- modular hit solvers:
  - iZm_solve_for_xp
  - iZm_solve_for_xp_mpz
  - iZm_solve_for_yp
- segment lifecycle:
  - vx_init
  - vx_full_sieve
  - vx_stream_file
  - vx_free

If you extend SiZ-family algorithms, this is the correct layer to reuse instead of re-implementing segment logic.

1.10 9. Usage Examples

1.10.1 9.1 Prime sieving (library call)

```
#include <iZ_api.h>

int main(void) {
    uint64_t n = 1000000ULL;
    UI64_ARRAY *primes = SiZm(n);
    if (!primes) return 1;

    printf("count=%d\n", primes->count);
    printf("last=%llu\n", (unsigned long long)primes->array[primes->count - 1]);

    ui64_free(&primes);
    return 0;
}
```

1.10.2 9.2 Range counting/streaming

```
#include <iZ_api.h>

int main(void) {
    INPUT_SIEVE_RANGE in = {
        .start = "1000000000000",
        .range = 1000000,
        .mr_rounds = MR_ROUNDS,
        .filepath = "output/iZ_stream.txt"
    };

    uint64_t count = SiZ_stream(&in);
    printf("count=%llu\n", (unsigned long long)count);
    return 0;
}
```

1.10.3 9.3 Random prime generation

```
#include <iZ_api.h>

int main(void) {
    mpz_t p;
    mpz_init(p);

    int ok = vx_random_prime(p, 2048, get_cpu_cores_count());
    if (ok) gmp_printf("%Zd\n", p);

    mpz_clear(p);
    return ok ? 0 : 1;
}
```

1.10.4 9.4 Build and run included examples

```
make examples
./build/examples/sieve_primes SiZm 10000000 10
./build/examples/SiZ_range 0 1000000
./build/examples/SiZ_range 1000000000000 1000000 output/iZ_stream.txt
./build/examples/p_genrators 1024 vx
```

Or use helper target:

```
make run-example EX=sieve_primes ARGS="SiZm 10000000 10"
```

1.11 10. Testing

Primary test runner target:

```
make test-all
```

Other targets:

```
make test-unit
make test-integration
```

Test options (Make variables):

```
make test-all VERBOSE=1
make test-all TEST_OUTPUT=output/test_all.txt
make test-all TEST_ARGS="--help"
```

Runner options (from test/test_all.c):

- --all
- --unit
- --integration
- --benchmark-p-sieve
- --benchmark-p-gen
- --save-results
- --verbose

1.12 11. Benchmarking

Dedicated benchmark targets:

```
make benchmark-p_sieve
make benchmark-p_gen
```

With result persistence (where supported):

```
make benchmark-p_sieve SAVE_RESULTS=1
make benchmark-p_gen SAVE_RESULTS=1
```

Plotting utilities are in py_tools/.

1.13 12. Doxygen Documentation

To generate docs/userManual.pdf, run this command from root directory:

```
make docs
```

1.14 13. Logging and Output Files

- Logging is enabled by default (LOGGING=1, -DENABLE_LOGGING).
- Log files are under logs/.
- Main program stdout is appended to output/stdout.txt.
- Many example/test flows write artifacts under output/.

1.15 14. Extension Guidelines

If you plan to add new algorithms for publication/research:

1. Add public declarations only to headers in include/ when API-stable.
2. Keep new internals inside src/toolkit/ or src/ as appropriate.
3. Reuse BITMAP, UI∗_ARRAY, IZM, and VX_SEG before introducing new containers.
4. Preserve ownership conventions (all heap-returning APIs must document free path).
5. Add tests to unit/integration suites and expose benchmark hooks if relevant.
6. Update Doxygen comments so doxygen Doxyfile remains coherent.

1.16 15. Troubleshooting

1.16.1 Linker errors for GMP/OpenSSL

- Install required dev packages (see Dependencies).
- Ensure pkg-config can resolve gmp and OpenSSL.
- On macOS Homebrew, verify openssl@3 is installed.

1.16.2 doxygen: command not found

Install Doxygen and re-run:

```
doxygen Doxyfile
```

1.16.3 Slow or memory-heavy benchmark runs

- Use smaller limits/bit sizes for local checks.
- Run benchmark targets explicitly (they are intentionally separated from test-all).

1.17 16. Future Work

The roadmap includes algorithmic additions that can expand both performance and research value:

1.17.1 16.1 Segmented Sieve of Atkin (Segmented SoA)

- Add a segmented SoA implementation for large-N runs where monolithic SoA memory behavior becomes limiting.
- Benchmark it against SSoE, SiZ, and SiZm to identify realistic crossover regions.

1.17.2 16.2 Sorenson's Pseudosquares Prime Sieve

- Explore integration of a pseudosquares-based sieve pipeline (as proposed by Sorenson) for fast prime filtering in selected ranges.
- Evaluate how it complements iZ-based generation workflows.

1.17.3 16.3 Pritchard-inspired Wheel Sieve Direction

- Prototype one high-impact direction from Pritchard's sieve work (for example dynamic wheel progression) inside the toolkit layer.
- Study whether hybridization with iZm segmentation can further improve constant factors in practice.

If you are interested in contributing to any of these, open an issue or discussion with an initial design/↩ benchmark plan.

1.18 17. License

This project is licensed under the MIT License. See LICENSE.

Chapter 2

iZprime User Manual

2.1 Scope

This manual summarizes the public API, toolkit internals, and extension conventions for the iZprime C library.

2.2 Architecture Layers

- iZ Public API provides stable entry points used by applications.
- iZ Toolkit covers iZ/iZm primitives and VX segment execution.
- Bitmap Toolkit and Integer Arrays provide storage utilities used by sieves.
- Utilities and Logging provide shared runtime helpers.

2.3 Algorithm Families

- Classical sieves: SoE, SSoE, SoEu, SoS, SoA.
- SiZ family: SiZ, SiZm, SiZm_vy.
- Range variants: SiZ_stream and SiZ_count.
- Prime search: vx_random_prime, vy_random_prime, iZ_next_prime.

Detailed pseudocode for the algorithms is available in docs/pseudocode.pdf.

2.4 Memory Ownership

- Functions returning UI64_ARRAY * transfer ownership to the caller.
- Release integer arrays with ui16_free, ui32_free, or ui64_free.
- Release bitmaps with bitmap_free.
- Release toolkit objects with iZm_free and vx_free.

2.5 Extending the Library

1. Add API declarations and Doxygen contracts in headers under include/.
2. Implement in src/ with sectioned comments that describe algorithm flow.
3. Keep iZ mapping invariants explicit (6x-1 and 6x+1 lines).
4. Add tests/benchmarks through the test API entry points.

2.6 Building Documentation

Run from repository root:

```
doxygen Doxyfile
```

Generated HTML is written under docs/api/html.

Chapter 3

Topic Index

3.1 Topics

Here is a list of all topics with brief descriptions:

Bitmap Module 19
Integer Arrays 31
iZ Public API 34
Toolkit (iZ/iZm) 47
Logging 65
Tests and Benchmarks 73
Utilities 74

Chapter 4

Data Structure Index

4.1 Data Structures

Here are the data structures with brief descriptions:

BITMAP
Packed bit-array with optional SHA-256 checksum 83
INPUT_SIEVE_RANGE
Input parameters for range sieving/counting 85
IZM
Precomputed iZm assets for repeated VX-segment sieving 86
SIEVE_LIMIT 88
SIEVE_MODEL 88
UI16_ARRAY
Dynamic array for uint16_t values 89
UI32_ARRAY
Dynamic array for uint32_t values 90
UI64_ARRAY
Dynamic array for uint64_t values 91
VX_SEG
Runtime state for one VX segment at a specific y 93

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

include/bitmap.h
Bitmap container and bit operations used by sieve implementations 97
include/int_arrays.h
Dynamic arrays for uint16_t, uint32_t, and uint64_t values 99
include/iZ_api.h
Public API for prime sieving, range scans, and prime generation 102
include/iZ_toolkit.h
Core iZ/iZm primitives used by SiZ-family algorithms 104
include/logger.h
Header file for the logging system 107
include/test_api.h
Public test/benchmark entry points for the iZ library 110
include/utils.h
Shared utilities and common includes for the iZ library 112
src/iZ_apps.c
High-level application routines built on top of the iZ API 114
src/prime_sieve.c
Implementations of classical and SiZ-family sieve algorithms 125
src/toolkit/bitmap.c
Implementation of the bitmap module for efficient bit array operations 133
src/toolkit/int_arrays.c
Implementation of dynamic integer array module for 16-bit, 32-bit, and 64-bit unsigned integers 139
src/toolkit/iZ_toolkit.c
Implementation of iZ index space helpers and iZm/VX segment machinery 141
src/toolkit/logger.c
Logging module 155
src/toolkit/utils.c
Implementations for the shared utility layer 158

Chapter 6

Topic Documentation

6.1 Bitmap Module

Packed bit-array primitives for sieve and toolkit modules.

Files

- file bitmap.c

  Implementation of the bitmap module for efficient bit array operations.

Data Structures

- struct BITMAP

  Packed bit-array with optional SHA-256 checksum.

Functions

- int TEST_BITMAP (int verbose)

  Run bitmap module tests.

Lifecycle

- BITMAP * bitmap_init (size_t size, int set_bits)

  Allocate a bitmap with size bits.
- void bitmap_free (BITMAP **bitmap)

  Free a bitmap and set the caller pointer to NULL.
- BITMAP * bitmap_clone (BITMAP *src)

  Deep-copy a bitmap, including data and checksum.

Single-bit Operations

- int bitmap_get_bit (BITMAP *bitmap, size_t idx)

  Read bit value at idx.
- void bitmap_set_bit (BITMAP *bitmap, size_t idx)

  Set bit at idx to 1.
- void bitmap_flip_bit (BITMAP *bitmap, size_t idx)

  Toggle bit at idx.
- void bitmap_clear_bit (BITMAP *bitmap, size_t idx)

  Clear bit at idx (set to 0).

Bulk Operations

- void bitmap_set_all (BITMAP *bitmap)

  Set all bits to 1.
- void bitmap_clear_all (BITMAP *bitmap)

  Clear all bits to 0.
- void bitmap_clear_steps (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)

  Clear every step-th bit from start_idx up to limit.
- void bitmap_clear_steps_simd (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)

  SIMD-accelerated variant of bitmap_clear_steps().

Integrity and I/O

- void bitmap_compute_hash (BITMAP *bitmap)

  Compute SHA-256 over bitmap data and store it in BITMAP::sha256.
- int bitmap_validate_hash (BITMAP *bitmap)

  Verify BITMAP::sha256 against current bitmap data.
- int bitmap_fwrite (BITMAP *bitmap, FILE *file)

  Write bitmap payload and checksum to a binary stream.
- BITMAP * bitmap_fread (FILE *file)

  Read bitmap payload and checksum from a binary stream.

6.1.1 Detailed Description

Packed bit-array primitives for sieve and toolkit modules.

6.1.2 Function Documentation

6.1.2.1 bitmap_clear_all()

```
void bitmap_clear_all (
            BITMAP * bitmap)
```

Clear all bits to 0.

Parameters

bitmap	Bitmap to modify.

Clear all bits to 0.

Parameters

bitmap	A pointer to the BITMAP structure.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 180 of file bitmap.c.

References BITMAP::data, and BITMAP::size.

6.1.2.2 bitmap_clear_bit()

```
void bitmap_clear_bit (
            BITMAP * bitmap,
            size_t idx)
```

Clear bit at idx (set to 0).

Parameters

bitmap	Bitmap to modify.
idx	Zero-based bit index.

Clear bit at idx (set to 0).

Parameters

bitmap	The BITMAP to modify.
idx	The index of the bit to clear.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 159 of file bitmap.c.

References BITMAP::data.

Referenced by iZm_construct_vx_base(), SoEu(), vx_prob_sieve(), and vx_stream_file().

6.1.2.3 bitmap_clear_steps()

```
void bitmap_clear_steps (
            BITMAP * bitmap,
            uint64_t step,
            uint64_t start_idx,
            uint64_t limit)
```

Clear every step-th bit from start_idx up to limit.

This is the core primitive for marking composite progressions in sieve code.

Parameters

bitmap	Bitmap to modify.
step	Index increment between cleared bits.
start_idx	First index to clear.
limit	Inclusive upper index bound.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 210 of file bitmap.c.

References BITMAP::data, and BITMAP::size.

Referenced by vx_search_prime().

6.1.2.4 bitmap_clear_steps_simd()

```
void bitmap_clear_steps_simd (
            BITMAP * bitmap,
            uint64_t step,
            uint64_t start_idx,
            uint64_t limit)
```

SIMD-accelerated variant of bitmap_clear_steps().

Parameters

bitmap	Bitmap to modify.
step	Index increment between cleared bits.
start_idx	First index to clear.
limit	Inclusive upper index bound.

SIMD-accelerated variant of bitmap_clear_steps().

This function uses SIMD instructions (AVX2, SSE2, or NEON) to clear bits at regular intervals in the bitmap. It processes multiple bits in parallel for improved performance on large bitmaps.

Parameters

bitmap	Pointer to the bitmap to modify

step	Interval between bits to clear (must be > 0)
start_idx	Starting bit index (inclusive)
limit	Upper bound for clearing (inclusive)

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 233 of file bitmap.c.

References BITMAP::data, and BITMAP::size.

Referenced by iZm_construct_vx_base(), process_eratosthenes_bitmap(), process_iZ_bitmaps(), SiZm(), SiZm_vy(), SoA(), SoS(), SSoE(), and vx_det_sieve().

6.1.2.5 bitmap_clone()

```
BITMAP * bitmap_clone (
            BITMAP * src)
```

Deep-copy a bitmap, including data and checksum.

Parameters

src	Source bitmap.

Returns

Cloned bitmap, or NULL on failure.

Deep-copy a bitmap, including data and checksum.

Allocates a new bitmap with the same size as the source and copies all data including the bit array and SHA-256 hash. The returned bitmap is completely independent of the source and must be freed separately.

Parameters

src	Pointer to the source bitmap to clone

Returns

Pointer to newly allocated cloned BITMAP, or NULL on failure

Return values

NULL	if src is NULL, src->size is 0, or memory allocation fails

Note

The clone includes a copy of the SHA-256 hash from the source
Caller must free the returned bitmap using bitmap_free()
Changes to the clone do not affect the source and vice versa

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 382 of file bitmap.c.

References bitmap_init(), BITMAP::byte_size, BITMAP::data, BITMAP::sha256, and BITMAP::size.

Referenced by iZm_clone(), SiZm(), and vx_init().

6.1.2.6 bitmap_compute_hash()

```
void bitmap_compute_hash (
            BITMAP * bitmap)
```

Compute SHA-256 over bitmap data and store it in BITMAP::sha256.

Parameters

bitmap	Bitmap to hash.

Compute SHA-256 over bitmap data and store it in BITMAP::sha256.

Parameters

bitmap	The BITMAP for which the SHA-256 hash is generated.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 398 of file bitmap.c.

References BITMAP::byte_size, BITMAP::data, and BITMAP::sha256.

Referenced by bitmap_fwrite().

6.1.2.7 bitmap_flip_bit()

```
void bitmap_flip_bit (
            BITMAP * bitmap,
            size_t idx)
```

Toggle bit at idx.

Parameters

bitmap	Bitmap to modify.

idx	Zero-based bit index.

Toggle bit at idx.

Parameters

bitmap	The BITMAP to modify.
idx	The index of the bit to flip.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 148 of file bitmap.c.

References BITMAP::data.

Referenced by SoA().

6.1.2.8 bitmap_fread()

```
BITMAP * bitmap_fread (
            FILE * file)
```

Read bitmap payload and checksum from a binary stream.

Parameters

file	Readable stream.

Returns

Newly allocated bitmap, or NULL on parse/verification failure.

Read bitmap payload and checksum from a binary stream.

Parameters

file	The file pointer to read from.

Returns

BITMAP* A pointer to the newly created BITMAP, or NULL on failure.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 492 of file bitmap.c.

References bitmap_free(), bitmap_init(), bitmap_validate_hash(), BITMAP::byte_size, BITMAP::data, log_error(), and BITMAP::sha256.

6.1.2.9 bitmap_free()

```
void bitmap_free (
            BITMAP ** bitmap)
```

Free a bitmap and set the caller pointer to NULL.

Parameters

bitmap	Address of a BITMAP pointer.

Free a bitmap and set the caller pointer to NULL.

This function safely deallocates the bitmap's data array and the structure itself, then sets the pointer to NULL to prevent use-after-free bugs and double-free errors. Safe to call with NULL pointers.

Parameters

bitmap	Double pointer to the BITMAP to free

Postcondition

*bitmap is set to NULL after successful deallocation

Safe to call multiple times on the same pointer

Note

No operation performed if bitmap == NULL or *bitmap == NULL

This is the only correct way to free a BITMAP; never use free() directly

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 103 of file bitmap.c.

Referenced by bitmap_fread(), get_root_primes(), iZm_free(), SiZ(), SiZm(), SiZm_vy(), SoA(), SoE(), SoEu(), SoS(), SSoE(), vx_free(), and vx_search_prime().

6.1.2.10 bitmap_fwrite()

```
int bitmap_fwrite (
            BITMAP * bitmap,
            FILE * file)
```

Write bitmap payload and checksum to a binary stream.

Parameters

bitmap	Bitmap to serialize.

file	Writable stream.

Returns

1 on success, otherwise 0.

Write bitmap payload and checksum to a binary stream.

Parameters

bitmap	The BITMAP to write.
file	The file pointer to write to.

Returns

int 1 on success, 0 on failure.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 440 of file bitmap.c.

References bitmap_compute_hash(), BITMAP::byte_size, BITMAP::data, log_error(), BITMAP::sha256, and BITMAP::size.

6.1.2.11 bitmap_get_bit()

```
int bitmap_get_bit (
            BITMAP * bitmap,
            size_t idx)
```

Read bit value at idx.

Parameters

bitmap	Bitmap to inspect.
idx	Zero-based bit index.

Returns

1 if set, otherwise 0.

Read bit value at idx.

Parameters

bitmap	The BITMAP to read from.

idx	The index of the bit to read.

Returns

int 1 if the bit is set, 0 if unset, -1 on error.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 137 of file bitmap.c.

References BITMAP::data.

Referenced by iZ_next_prime(), process_eratosthenes_bitmap(), process_iZ_bitmaps(), SiZm(), SiZm_vy(), SoA(), SoEu(), SoS(), SSoE(), vx_collect_p_gaps(), vx_det_sieve(), vx_prob_sieve(), vx_search_prime(), and vx_stream_file().

6.1.2.12 bitmap_init()

```
BITMAP * bitmap_init (
            size_t size,
            int set_bits)
```

Allocate a bitmap with size bits.

Parameters

size	Number of bits to allocate (must be > 0).
set_bits	Non-zero initializes all bits to 1, otherwise to 0.

Returns

Newly allocated bitmap, or NULL on allocation failure.

Allocate a bitmap with size bits.

Allocates memory for both the BITMAP structure and its underlying data array. The data array is allocated using calloc(), ensuring all bits start at 0. The SHA-256 hash field is initialized to all zeros.

Parameters

size	The number of bits in the bitmap (must be > 0)
set_bits	If non-zero, initializes all bits to 1; otherwise, all bits are 0.

Returns

Pointer to newly allocated BITMAP on success, NULL on failure

Return values

NULL	if size is 0

NULL	if memory allocation fails

Note

Memory allocated: sizeof(BITMAP) + size/8 bytes

On failure, any partially allocated memory is freed before returning

Warning

Caller must free returned bitmap using bitmap_free(), not free()

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 57 of file bitmap.c.

References BITMAP::byte_size, BITMAP::data, log_error(), BITMAP::sha256, and BITMAP::size.

Referenced by bitmap_clone(), bitmap_fread(), get_root_primes(), iZm_init(), SiZ(), SiZm(), SiZm_vy(), SoA(), SoE(), SoEu(), SoS(), SSoE(), and vx_search_prime().

6.1.2.13 bitmap_set_all()

```
void bitmap_set_all (
            BITMAP * bitmap)
```

Set all bits to 1.

Parameters

bitmap	Bitmap to modify.

Set all bits to 1.

Parameters

bitmap	The BITMAP to modify.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 170 of file bitmap.c.

References BITMAP::data, and BITMAP::size.

Referenced by iZm_construct_vx_base(), SiZm_vy(), and SSoE().

6.1.2.14 bitmap_set_bit()

```
void bitmap_set_bit (
            BITMAP * bitmap,
            size_t idx)
```

Set bit at idx to 1.

Parameters

bitmap	Bitmap to modify.

idx	Zero-based bit index.

Set bit at idx to 1.

Parameters

bitmap	The BITMAP to modify.
idx	The index of the bit to set.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 125 of file bitmap.c.

References BITMAP::data.

6.1.2.15 bitmap_validate_hash()

```
int bitmap_validate_hash (
            BITMAP * bitmap)
```

Verify BITMAP::sha256 against current bitmap data.

Parameters

bitmap	Bitmap to verify.

Returns

1 if checksum matches, otherwise 0.

Verify BITMAP::sha256 against current bitmap data.

Parameters

bitmap	The BITMAP whose hash is validated.

Returns

int 1 if the hash matches, 0 otherwise.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 412 of file bitmap.c.

References BITMAP::byte_size, BITMAP::data, log_error(), and BITMAP::sha256.

Referenced by bitmap_fread().

6.1.2.16 TEST_BITMAP()

int TEST_BITMAP (
int verbose)

Run bitmap module tests.

Parameters

verbose	Non-zero enables detailed logging.

Returns

1 when all tests pass, otherwise 0.

6.2 Integer Arrays

Dynamic integer containers used by sieve and toolkit layers.

Files

- file int_arrays.c
  Implementation of dynamic integer array module for 16-bit, 32-bit, and 64-bit unsigned integers.

Data Structures

- struct UI16_ARRAY
  Dynamic array for uint16_t values.
- struct UI32_ARRAY
  Dynamic array for uint32_t values.
- struct UI64_ARRAY
  Dynamic array for uint64_t values.

Functions

- int TEST_GENERIC_INT_ARRAYS (int verbose)
  Run generic dispatch tests for C11 _Generic helper macros.

UI16 API

- UI16_ARRAY * ui16_init (int capacity)

  Allocate a UI16 array with an initial capacity.
- void ui16_free (UI16_ARRAY **array)

  Free a UI16 array and null the caller pointer.
- void ui16_resize_to (UI16_ARRAY *array, int new_capacity)

  Resize UI16 storage to new_capacity (must be >= count).
- void ui16_resize_to_fit (UI16_ARRAY *array)

  Shrink UI16 storage so capacity equals count.
- void ui16_push (UI16_ARRAY *array, uint16_t element)

  Append a uint16 value, growing storage if needed.
- void ui16_pop (UI16_ARRAY *array)

  Remove the last element if the array is non-empty.
- void ui16_compute_hash (UI16_ARRAY *array)

  Compute SHA-256 checksum over active payload.
- int ui16_verify_hash (UI16_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui16_fwrite (UI16_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI16_ARRAY * ui16_fread (FILE *file)

  Deserialize a UI16 array from a binary stream.
- int TEST_UI16_ARRAY (int verbose)

  Execute UI16 test suite.

UI32 API

- UI32_ARRAY * ui32_init (int capacity)

  Allocate a UI32 array with an initial capacity.
- void ui32_free (UI32_ARRAY **array)

  Free a UI32 array and null the caller pointer.
- void ui32_resize_to (UI32_ARRAY *array, int new_capacity)

  Resize UI32 storage to new_capacity (must be >= count).
- void ui32_resize_to_fit (UI32_ARRAY *array)

  Shrink UI32 storage so capacity equals count.
- void ui32_push (UI32_ARRAY *array, uint32_t element)

  Append a uint32 value, growing storage if needed.
- void ui32_pop (UI32_ARRAY *array)

  Remove the last element if the array is non-empty.
- void ui32_compute_hash (UI32_ARRAY *array)

  Compute SHA-256 checksum over active payload.
- int ui32_verify_hash (UI32_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui32_fwrite (UI32_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI32_ARRAY * ui32_fread (FILE *file)

  Deserialize a UI32 array from a binary stream.
- int TEST_UI32_ARRAY (int verbose)

  Execute UI32 test suite.

UI64 API

- UI64_ARRAY * ui64_init (int capacity)

  Allocate a UI64 array with an initial capacity.
- void ui64_free (UI64_ARRAY **array)

  Free a UI64 array and null the caller pointer.
- void ui64_resize_to (UI64_ARRAY *array, int new_capacity)

  Resize UI64 storage to new_capacity (must be >= count).
- void ui64_resize_to_fit (UI64_ARRAY *array)

  Shrink UI64 storage so capacity equals count.
- void ui64_push (UI64_ARRAY *array, uint64_t element)

  Append a uint64 value, growing storage if needed.
- void ui64_pop (UI64_ARRAY *array)

  Remove the last element if the array is non-empty.
- void ui64_compute_hash (UI64_ARRAY *array)

  Compute SHA-256 checksum over active payload.
- int ui64_verify_hash (UI64_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui64_fwrite (UI64_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI64_ARRAY * ui64_fread (FILE *file)

  Deserialize a UI64 array from a binary stream.
- int TEST_UI64_ARRAY (int verbose)

  Execute UI64 test suite.

6.2.1 Detailed Description

Dynamic integer containers used by sieve and toolkit layers.

6.2.2 Function Documentation

6.2.2.1 TEST_GENERIC_INT_ARRAYS()

```
int TEST_GENERIC_INT_ARRAYS (
            int verbose)
```

Run generic dispatch tests for C11 _Generic helper macros.

Parameters

verbose	Non-zero enables detailed logging.

Returns

1 when all tests pass, otherwise 0.

6.3 iZ Public API

< iZ/iZm toolkit structures and helpers.

Files

- file iZ_apps.c
  High-level application routines built on top of the iZ API.
- file prime_sieve.c
  Implementations of classical and SiZ-family sieve algorithms.

Data Structures

- struct INPUT_SIEVE_RANGE
  Input parameters for range sieving/counting.

Macros

- #define N_LIMIT (1000000000000ULL)

Classic Prime Sieve Algorithms

Baseline sieves up to a numeric limit.

All classic sieve functions take a limit n and return an ascending list of primes <= n.

- UI64_ARRAY * SoE (uint64_t n)
  Optimized Sieve of Eratosthenes.
- UI64_ARRAY * SSoE (uint64_t n)
  Segmented Sieve of Eratosthenes.
- UI64_ARRAY * SoEu (uint64_t n)
  Euler (linear) sieve.
- UI64_ARRAY * SoS (uint64_t n)
  Sieve of Sundaram.
- UI64_ARRAY * SoA (uint64_t n)
  Sieve of Atkin.

iZ-based Sieve Algorithms

Sieve family operating in the 6x-1 / 6x+1 index space.

- UI64_ARRAY * SiZ (uint64_t n)
  Solid Sieve-iZ (wheel 6, iZ index space).
- UI64_ARRAY * SiZm (uint64_t n)
  Segmented Sieve-iZm (VX segmented, horizontal processing).
- UI64_ARRAY * SiZm_vy (uint64_t n)
  Segmented Sieve-iZm (vertical processing; faster, unordered output).

SiZ Range Variants

Count/stream primes over a numeric interval.

- uint64_t SiZ_stream (INPUT_SIEVE_RANGE *range)
  Stream primes in a range to filepath (and return the count).
- uint64_t SiZ_count (INPUT_SIEVE_RANGE *input_range, int cores_num)
  Count primes in a range using multiple worker processes.

Prime Generators

Probabilistic prime searches using iZm/VX machinery.

- int vy_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using vertical (vy) sieving.
- int vx_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using horizontal (vx) sieving.
- int iZ_next_prime (mpz_t p, mpz_t base, int forward)
  Advance to the next (or previous) prime from a base value.

6.3.1 Detailed Description

< iZ/iZm toolkit structures and helpers.

< Common utilities, types, and dependencies.

High-level entry points for sieves and prime generation.

The API is organized into:

- Classic sieves: operate directly on the integer domain.
- SiZ / SiZm: operate in iZ index space (6x-1 and 6x+1), optionally segmented.
- Range variants: count/stream primes over large intervals.
- Prime generators: probabilistic searches using the iZm/VX machinery.

6.3.2 Macro Definition Documentation

6.3.2.1 N_LIMIT

#define N_LIMIT (1000000000000ULL)

Maximum supported sieve limit for standard entry points (10^12).

Definition at line 28 of file iZ_api.h.

Referenced by SiZm_vy().

6.3.3 Function Documentation

6.3.3.1 iZ_next_prime()

```
int iZ_next_prime (
            mpz_t p,
            mpz_t base,
            int forward)
```

Advance to the next (or previous) prime from a base value.

Find the next prime number after a given base.

Parameters

p	Output prime (initialized by caller).

base	Starting value.
forward	Non-zero to search forward, 0 to search backward.

Returns

1 on success, 0 on failure.

@description: This function searches for the next/previous prime number after a given base using the iZ framework.

Parameters

p	The mpz_t variable to store the found prime number.
base	The base number to start the search from.
forward	If true, search for the next prime; if false, search for the previous prime.

Returns

1 if a prime is found, 0 otherwise.

Definition at line 824 of file iZ_apps.c.

References IZM::base_x5, IZM::base_x7, bitmap_get_bit(), iZ_mpz(), iZm_free(), iZm_init(), log_debug(), log_error(), MR_ROUNDS, VX5, and VX6.

6.3.3.2 SiZ()

```
UI64_ARRAY * SiZ (
            uint64_t n)
```

Solid Sieve-iZ (wheel 6, iZ index space).

Classic Sieve-iZ algorithm for prime generation up to n.

Parameters

n	Upper bound (inclusive).

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n is in (10, 10^12].

The Sieve-iZ algorithm operates in the iZ index space of numbers of the form 6x ± 1, excluding all multiples of 2 and 3. It maintains two bitmaps, x5 and x7, representing candidates of the form 6x − 1 (iZ−) and 6x + 1 (iZ+), respectively. For each index x whose bit remains set, the corresponding candidate is treated as prime and its composites are marked using the Xp identity in the iZ space.

This provides a wheel factorization of modulo 6, reducing the number of candidates by approximately a factor of 3 compared to a classical sieve over all integers. After sieving, any remaining set bits correspond to primes of the form 6x ± 1, which are mapped back to their integer values and appended to the output array alongside the primes 2 and 3.

Parameters

n	Upper bound (inclusive) for prime generation. Must satisfy 10 < n <= pow(10, 12).

Returns

Pointer to a UI64_ARRAY containing all primes <= n on success, or NULL if allocation fails.

Definition at line 446 of file prime_sieve.c.

References UI64_ARRAY::array, bitmap_free(), bitmap_init(), UI64_ARRAY::count, est_pi_n(), process_iZ_bitmaps(), ui64_free(), ui64_init(), ui64_pop(), ui64_push(), and ui64_resize_to_fit().

Referenced by compute_max_vx(), iZm_init(), SiZm(), and SiZm_vy().

6.3.3.3 SiZ_count()

```
uint64_t SiZ_count (
            INPUT_SIEVE_RANGE * input_range,
            int cores_num)
```

Count primes in a range using multiple worker processes.

Multi-process prime counting over a numeric range using Sieve-iZm.

Parameters

input_range	Range configuration (output disabled).
cores_num	Requested worker process count.

Returns

Prime count in the interval, or 0 on error.

This function parallelizes prime counting over the interval [Zs, Ze] (interpreted from input_range) by partitioning the corresponding iZ index space into VX segments and distributing contiguous blocks of segments across multiple processes. Each child process owns its own cloned IZM instance and VX segments, performs a full Sieve-iZm pass over its assigned segments, and returns a local prime count to the parent via a pipe.

The parent process aggregates all child counts, applies boundary corrections for endpoints that do not align exactly with 6x ± 1, and returns the total number of primes in [Zs, Ze]. Any failure to create pipes, fork children, or initialize per-process resources causes the function to log an error and return 0.

Parameters

input_range	Pointer to an INPUT_SIEVE_RANGE structure describing the numeric interval and Miller–Rabin configuration.

cores_num	Requested number of worker processes. The actual number used is clamped to the available CPU cores and the number of VX segments in the range.

Returns

The total number of primes found in [Zs, Ze] on success, or 0 on any error or child-process failure.

Definition at line 204 of file iZ_apps.c.

References UI64_ARRAY::array, UI64_ARRAY::count, get_cpu_cores_count(), iZ_mpz(), iZm_clone(), iZm_free(), iZm_init(), log_error(), INPUT_SIEVE_RANGE::mr_rounds, VX_SEG::p_count, INPUT_SIEVE_RANGE::range, SiZm(), INPUT_SIEVE_RANGE::start, ui64_free(), VX6, vx_free(), vx_full_sieve(), and vx_init().

6.3.3.4 SiZ_stream()

```
uint64_t SiZ_stream (
            INPUT_SIEVE_RANGE * input_range)
```

Stream primes in a range to filepath (and return the count).

Count or stream primes in an arbitrary numeric range using Sieve-iZm.

Parameters

range	Range configuration (must include filepath).

Returns

Prime count in the interval, or 0 on error.

This function counts primes in the interval [Zs, Ze], where Zs is provided as a decimal string and Ze = Zs + range - 1. It maps the numeric bounds into the iZ index space (6x ± 1), partitions that space into segments of length vx, and for each segment invokes the Sieve-iZm machinery via VX segment objects.

When an output file path is provided in input_range->filepath, the function additionally streams all primes in the requested interval to that file in ascending order, reconstructing them from per-segment prime gaps. Otherwise, it operates in counting mode only.

Parameters

input_range	Pointer to an INPUT_SIEVE_RANGE structure describing the start value (as a decimal string), the range length, the optional output filepath, and Miller–Rabin configuration.

Returns

The total number of primes found in [Zs, Ze] on success, or 0 on invalid input, allocation failure, or I/O error.

Definition at line 37 of file iZ_apps.c.

References UI64_ARRAY::array, UI64_ARRAY::count, INPUT_SIEVE_RANGE::filepath, iZm_free(), iZm_init(), log_error(), INPUT_SIEVE_RANGE::mr_rounds, VX_SEG::p_count, INPUT_SIEVE_RANGE::range, SiZm(), INPUT_SIEVE_RANGE::start, ui64_free(), VX6, vx_free(), vx_full_sieve(), vx_init(), and vx_stream_file().

6.3.3.5 SiZm()

```
UI64_ARRAY * SiZm (
            uint64_t n)
```

Segmented Sieve-iZm (VX segmented, horizontal processing).

Segmented Sieve-iZm algorithm for prime generation up to n.

Parameters

n	Upper bound (inclusive).

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n is in (10, 10^12].

@description: The Sieve-iZm algorithm is a segmented, cache-aware variant of the Sieve-iZ that operates in the iZ index space of numbers of the form 6x ± 1. It partitions the search interval into segments of length vx, where vx is a product of small primes, and constructs a pre-sieved base segment using those small primes once at initialization time.

For each segment, Sieve-iZm clones the base bitmaps and only marks composites of the remaining root primes that do not divide vx. This amortizes the cost of small-prime sieving across all segments and keeps the working set small and highly cache-resident. After marking, any remaining set bits correspond to primes of the form 6x ± 1 within that segment, which are then mapped back to their integer values and appended to the output array.

Compared to the basic Sieve-iZ, Sieve-iZm achieves the same O(n log log n) time complexity with effectively constant auxiliary memory (aside from the output) and significantly reduced constant factors, making it well-suited for enumerating primes up to large bounds.

Parameters

n	Upper bound (inclusive) for prime generation. Must satisfy 10 < n <= pow(10, 12).

Returns

Pointer to a UI64_ARRAY containing all primes <= n on success, or NULL on allocation or initialization failure.

Definition at line 519 of file prime_sieve.c.

References UI64_ARRAY::array, bitmap_clear_steps_simd(), bitmap_clone(), bitmap_free(), bitmap_get_bit(), bitmap_init(), BITMAP::byte_size, compute_l2_vx(), UI64_ARRAY::count, BITMAP::data, est_pi_n(), iZ(), iZm_construct_vx_base(), iZm_solve_for_xp(), process_iZ_bitmaps(), SiZ(), ui64_free(), ui64_init(), ui64_pop(), ui64_push(), and ui64_resize_to_fit().

Referenced by SiZ_count(), SiZ_stream(), and vx_search_prime().

6.3.3.6 SiZm_vy()

```
UI64_ARRAY * SiZm_vy (
            uint64_t n)
```

Segmented Sieve-iZm (vertical processing; faster, unordered output).

Segmented Sieve-iZm with vertical (vy) traversal order.

This variant emphasizes throughput and may not preserve ascending order of returned primes.

Parameters

n	Upper bound (inclusive).

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n is in (10, 10^12].

This variant prioritizes throughput and returns primes in non-sorted order.

Parameters

n	Inclusive upper bound for prime generation.

Returns

Heap-allocated list of primes <= n, or NULL on failure.

Definition at line 627 of file prime_sieve.c.

References UI64_ARRAY::array, bitmap_clear_steps_simd(), bitmap_free(), bitmap_get_bit(), bitmap_init(), bitmap_set_all(), UI64_ARRAY::count, gcd(), get_root_primes(), iZ(), iZm_solve_for_yp(), N_LIMIT, SiZ(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

6.3.3.7 SoA()

```
UI64_ARRAY * SoA (
            uint64_t n)
```

Sieve of Atkin.

Sieve of Atkin: Generates a list of prime numbers up to a given limit using the Sieve of Atkin algorithm.

Parameters

n	Upper bound (inclusive).

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n is in (10, 10^12].

@description: This function implements the Sieve of Atkin, an efficient algorithm to find all prime numbers up to a specified integer n. It initializes a bitmap to mark potential primes and applies the Atkin conditions to identify primes. The function also marks odd multiples of squares of primes as non-prime, then collects the remaining unmarked numbers as primes.

Parameters

n	The upper limit (inclusive) up to which prime numbers are to be found.

Returns

- UI64_ARRAY∗ A pointer to the UI64_ARRAY structure containing the list of primes up to n.
- NULL if memory allocation fails or if n is less than 10.

Definition at line 329 of file prime_sieve.c.

References bitmap_clear_steps_simd(), bitmap_flip_bit(), bitmap_free(), bitmap_get_bit(), bitmap_init(), est_pi_n(), ui64_free(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

6.3.3.8 SoE()

```
UI64_ARRAY * SoE (
            uint64_t n)
```

Optimized Sieve of Eratosthenes.

Optimized Sieve of Eratosthenes algorithm to find all primes up to n.

Parameters

n	Upper bound (inclusive).

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n is in (10, 10^12].

@description: This function applies common speedups to the Sieve of Eratosthenes algorithm. It works the same way as the classic sieve but skips checking or marking even numbers and starts with 3, incrementing by 2, it also starts the sieve from p∗p.

Parameters

n	The upper limit to find primes.

Returns

- A pointer to a UI64_ARRAY structure containing the list of prime numbers up to n,
- NULL if memory allocation fails or if n is less than 10.

Definition at line 69 of file prime_sieve.c.

References bitmap_free(), bitmap_init(), est_pi_n(), process_eratosthenes_bitmap(), ui64_free(), ui64_init(), and ui64_resize_to_fit().

6.3.3.9 SoEu()

```
UI64_ARRAY * SoEu (
            uint64_t n)
```

Euler (linear) sieve.

Sieve of Euler: Generates a list of prime numbers up to a given limit using the Euler Sieve algorithm.

Parameters

n	Upper bound (inclusive).

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n is in (10, 10^12].

@description: This function uses the Euler Sieve algorithm to find all prime numbers up to a specified limit n. It marks each composite only once, allowing for a more efficient sieve process. It initializes a bitmap to track prime numbers and iterates through the numbers, marking distinct multiples of each prime found. The function also handles memory allocation and resizing of the primes array as needed.

Parameters

n	The upper limit up to which prime numbers are to be found.

Returns

- UI64_ARRAY∗ A pointer to the UI64_ARRAY structure containing the list of primes up to n.
- NULL if memory allocation fails or if n is less than 10.

Definition at line 199 of file prime_sieve.c.

References UI64_ARRAY::array, bitmap_clear_bit(), bitmap_free(), bitmap_get_bit(), bitmap_init(), UI64_ARRAY::count, est_pi_n(), ui64_free(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

6.3.3.10 SoS()

```
UI64_ARRAY * SoS (
            uint64_t n)
```

Sieve of Sundaram.

Sieve of Sundaram: Generates a list of prime numbers up to a given limit using the Sieve of Sundaram algorithm.

Parameters

n	Upper bound (inclusive).

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n is in (10, 10^12].

@description: This function implements the Sieve of Sundaram algorithm to find all prime numbers up to a specified limit n. The Sieve of Sundaram works by eliminating numbers of the form i + j + 2ij from a list of integers from 1 to k = (n-1)/2. If an integer x in this range is not eliminated, then 2x+1 is a prime number. The prime number 2 is handled as a special case, as the sieve only generates odd primes.

Parameters

n	The upper limit (inclusive) up to which prime numbers are to be found.

Returns

- UI64_ARRAY∗ A pointer to the UI64_ARRAY structure containing the list of primes up to n.
- NULL if memory allocation fails or if n is less than 10.

Definition at line 264 of file prime_sieve.c.

References bitmap_clear_steps_simd(), bitmap_free(), bitmap_get_bit(), bitmap_init(), est_pi_n(), ui64_free(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

6.3.3.11 SSoE()

```
UI64_ARRAY * SSoE (
            uint64_t n)
```

Segmented Sieve of Eratosthenes.

Segmented Sieve of Eratosthenes algorithm to find all primes up to n.

Parameters

n	Upper bound (inclusive).

Returns

Heap-allocated prime list, or NULL on allocation failure.

Precondition

n is in (10, 10^12].

@description: This function implements the Segmented Sieve of Eratosthenes algorithm to find all prime numbers up to a given limit n. It divides the range into segments and uses a bitmap to mark prime numbers in each segment. The function returns a pointer to a UI64_ARRAY structure containing the list of prime numbers found. The function also handles memory allocation and resizing of the primes array as needed.

Parameters

n	The upper limit to find primes.

Returns

- A pointer to the UI64_ARRAY structure containing the list of primes up to n,
- NULL if memory allocation fails or if n is less than 1000.

Definition at line 112 of file prime_sieve.c.

References UI64_ARRAY::array, bitmap_clear_steps_simd(), bitmap_free(), bitmap_get_bit(), bitmap_init(), bitmap_set_all(), UI64_ARRAY::count, est_pi_n(), process_eratosthenes_bitmap(), ui64_free(), ui64_init(), ui64_push(), and ui64_resize_to_fit().

6.3.3.12 vx_random_prime()

```
int vx_random_prime (
            mpz_t p,
            int bit_size,
            int cores_num)
```

Search for a random prime using horizontal (vx) sieving.

Generates a random prime candidate using the vx_search_prime routine.

Parameters

p	Output prime (initialized by caller).

bit_size	Target size of p in bits.
cores_num	Requested worker processes.

Returns

1 on success, 0 on failure.

@description: This function generates a random prime of a given bit size using the vx_search_prime routine. It initializes the random base and sets up the search parameters. The function also allows for parallel processing of the search using multiple child processes. The generated prime is stored in the provided mpz_t p variable.

Parameters

p	The mpz_t variable to store the generated prime number.
bit_size	The target bit size of the prime.
cores_num	The number of cores to use for parallel processing.

Returns

1 if a prime is found, 0 otherwise.

Definition at line 697 of file iZ_apps.c.

References log_error(), VX5, VX6, and vx_search_prime().

6.3.3.13 vy_random_prime()

```
int vy_random_prime (
            mpz_t p,
            int bit_size,
            int cores_num)
```

Search for a random prime using vertical (vy) sieving.

Generates a random prime candidate using the vy_search_prime routine.

Parameters

p	Output prime (initialized by caller).
bit_size	Target size of p in bits.
cores_num	Requested worker processes.

Returns

1 on success, 0 on failure.

@description: This function generates a random prime of a given bit size using the vy_search_prime routine. It initializes the random base and sets up the search parameters. The function also allows for parallel processing of the search using multiple child processes. The generated prime is stored in the provided mpz_t p variable.

Parameters

p	The mpz_t variable to store the generated prime number.

bit_size	The target bit size of the prime.
cores_num	The number of cores to use for parallel processing.

Returns

1 if a prime is found, 0 otherwise.

Definition at line 550 of file iZ_apps.c.

References compute_max_vx(), log_error(), and vy_search_prime().

6.4 Toolkit (iZ/iZm)

Internal building blocks for SiZ and SiZm implementations.

Files

- file iZ_toolkit.c

  Implementation of iZ index space helpers and iZm/VX segment machinery.

Data Structures

- struct IZM

  Precomputed iZm assets for repeated VX-segment sieving.
- struct VX_SEG

  Runtime state for one VX segment at a specific y.

Macros

- #define MR_ROUNDS 25

Functions

- void iZm_construct_vx_base (uint64_t vx, BITMAP *base_x5, BITMAP *base_x7)

  Build pre-sieved base bitmaps for a VX segment.

iZ Mapping Helpers

- uint64_t iZ (uint64_t x, int i)

  Map iZ coordinates to an integer: 6*x + i.
- void iZ_mpz (mpz_t z, mpz_t x, int i)

  GMP variant of iZ().
- void process_iZ_bitmaps (UI64_ARRAY *primes, BITMAP *x5, BITMAP *x7, uint64_t x_limit)

  Traverse iZ bitmaps, emit surviving primes, and mark composites.
- void get_root_primes (UI64_ARRAY *primes, uint64_t limit)

  Generate primes up to limit for deterministic sieving.

IZM Lifecycle

- IZM * iZm_init (size_t vx)

  Allocate and initialize an IZM object for a given VX.
- IZM * iZm_clone (IZM *src)

  Deep-copy an IZM object for per-worker ownership.
- void iZm_free (IZM **iZm)

  Release an IZM object and set the caller pointer to NULL.

VX Selection Helpers

- uint64_t compute_vx_k (uint64_t n, int max_k)

  Compute VX by multiplying up to max_k small primes.
- uint64_t compute_l2_vx (uint64_t n)

  Choose VX using an L2-cache-aware heuristic.
- void compute_max_vx (mpz_t vx, int bit_size)

  Compute largest primorial-like VX below 2^bit_size.

Modular Hit Solvers

- uint64_t iZm_solve_for_xp (int m_id, uint64_t p, uint64_t vx, uint64_t y)

  Solve first x-hit of prime p for line m_id in segment y.
- uint64_t iZm_solve_for_xp_mpz (int m_id, uint64_t p, uint64_t vx, mpz_t y)

  GMP variant of iZm_solve_for_xp() for very large y.
- int64_t iZm_solve_for_yp (int m_id, uint64_t p, uint64_t vx, uint64_t x)

  Solve first y-hit for fixed x in vertical (vy) scanning.

VX Segment Lifecycle and Execution

- VX_SEG * vx_init (IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds)

  Initialize and deterministically sieve one VX segment.
- void vx_free (VX_SEG **vx_obj)

  Free a VX segment and all owned resources.
- void vx_collect_p_gaps (VX_SEG *vx_obj)

  Extract prime-gap encoding from a fully sieved segment.
- void vx_full_sieve (VX_SEG *vx_obj, int collect_p_gaps)

  Complete segment processing (probabilistic stage and optional gaps).
- void vx_stream_file (VX_SEG *vx_obj, FILE *output)

  Stream segment primes to a file.

Random Prime Search Routines

- int vx_search_prime (mpz_t p, int m_id, int vx, int bit_size)

  Horizontal iZm/VX random-prime search.
- int vy_search_prime (mpz_t p, int m_id, mpz_t vx)

  Vertical iZm/VY random-prime search.

Toolkit Tests

- int TEST_IZM (int verbose)

  Run IZM construction and solver tests.
- int TEST_VX_SEG (int verbose)

  Run VX segment tests.

Standard VX Sizes (primorial products excluding 2,3)

- #define VX2 (5 ∗ 7ULL)
- #define VX3 (VX2 ∗ 11ULL)
- #define VX4 (VX3 ∗ 13ULL)
- #define VX5 (VX4 ∗ 17ULL)
- #define VX6 (VX5 ∗ 19ULL)
- #define VX7 (VX6 ∗ 23ULL)
- #define VX8 (VX7 ∗ 29ULL)

6.4.1 Detailed Description

Internal building blocks for SiZ and SiZm implementations.

6.4.2 Macro Definition Documentation

6.4.2.1 MR_ROUNDS

#define MR_ROUNDS 25

Default Miller-Rabin rounds used by toolkit search/sieve helpers.

Definition at line 68 of file iZ_toolkit.h.

Referenced by iZ_next_prime(), vx_init(), vx_search_prime(), and vy_search_prime().

6.4.2.2 VX2

#define VX2 (5 ∗ 7ULL)

35

Definition at line 58 of file iZ_toolkit.h.

6.4.2.3 VX3

#define VX3 (VX2 ∗ 11ULL)

385

Definition at line 59 of file iZ_toolkit.h.

6.4.2.4 VX4

#define VX4 (VX3 ∗ 13ULL)

5005

Definition at line 60 of file iZ_toolkit.h.

6.4.2.5 VX5

#define VX5 (VX4 ∗ 17ULL)

85085

Definition at line 61 of file iZ_toolkit.h.

Referenced by iZ_next_prime(), and vx_random_prime().

6.4.2.6 VX6

#define VX6 (VX5 ∗ 19ULL)

1616615

Definition at line 62 of file iZ_toolkit.h.

Referenced by iZ_next_prime(), SiZ_count(), SiZ_stream(), and vx_random_prime().

6.4.2.7 VX7

#define VX7 (VX6 ∗ 23ULL)

37260615

Definition at line 63 of file iZ_toolkit.h.

6.4.2.8 VX8

#define VX8 (VX7 ∗ 29ULL)

1080558835

Definition at line 64 of file iZ_toolkit.h.

6.4.3 Function Documentation

6.4.3.1 compute_l2_vx()

```
uint64_t compute_l2_vx (
                uint64_t n)
```

Choose VX using an L2-cache-aware heuristic.

Choose a cache-aware VX size based on detected L2 capacity.

Parameters

n	Target numeric limit.

Returns

Selected VX size.

Parameters

n	Target numeric sieve bound.

Returns

Selected VX.

Definition at line 290 of file iZ_toolkit.c.

References get_cpu_L2_cache_size_bits(), and s_primes.

Referenced by SiZm().

6.4.3.2 compute_max_vx()

```
void compute_max_vx (
                mpz_t vx,
                int bit_size)
```

Compute largest primorial-like VX below 2^bit_size.

Compute the maximum vx such that vx < 2^bit_size.

Parameters

vx	Output mpz_t containing VX.
bit_size	Bit-size ceiling.

Parameters:

Parameters

vx	mpz_t variable to store the computed maximum vx value.

bit_size	Integer representing the bit size limit for vx.

Definition at line 315 of file iZ_toolkit.c.

References UI64_ARRAY::array, and SiZ().

Referenced by vy_random_prime().

6.4.3.3 compute_vx_k()

```
uint64_t compute_vx_k (
            uint64_t n,
            int max_k)
```

Compute VX by multiplying up to max_k small primes.

Calculate vx for a given range x_n by multiplying max_k small primes (> 3).

Parameters

n	Target numeric limit for sieving.
max↩ _k	Maximum number of multiplicative prime factors (>3).

Returns

Selected VX size.

Parameters:

Parameters

x_n	the number of bits to be vectorized
max↩ _k	the number of primes to be multiplied

Returns

vx the product of the first max_k primes (> 3) not exceeding x_n

Definition at line 268 of file iZ_toolkit.c.

References s_primes.

6.4.3.4 get_root_primes()

```
void get_root_primes (
            UI64_ARRAY * primes,
            uint64_t limit)
```

Generate primes up to limit for deterministic sieving.

Generate root primes up to limit using iZ bitmap traversal.

Parameters

primes	Output array, appended in ascending order.

limit	Numeric upper bound.
primes	Destination array.
limit	Numeric upper bound.

Definition at line 112 of file iZ_toolkit.c.

References bitmap_free(), bitmap_init(), process_iZ_bitmaps(), ui64_free(), and ui64_push().

Referenced by SiZm_vy().

6.4.3.5 iZ()

```
uint64_t iZ (
            uint64_t x,
            int i)   [inline]
```

Map iZ coordinates to an integer: 6∗x + i.

Parameters

x	iZ x-coordinate.
i	Matrix identifier, typically -1 or +1.

Returns

Mapped integer value.

Map iZ coordinates to an integer: 6∗x + i.

Parameters:

Parameters

x	(uint64_t) The value of x in 6x + i.
i	(int) The value of i in 6x + i.

Returns

The computed value 6x + i as a 64-bit unsigned integer.

Definition at line 24 of file iZ_toolkit.c.

Referenced by process_iZ_bitmaps(), SiZm(), and SiZm_vy().

6.4.3.6 iZ_mpz()

```
void iZ_mpz (
            mpz_t z,
            mpz_t x,
            int i)
```

GMP variant of iZ().

Parameters

z	Output value.

x	Input x-coordinate.
i	Matrix identifier, typically -1 or +1.

GMP variant of iZ().

Parameters:

Parameters

z	(mpz_t) The result of the calculation 6x + i.
x	(mpz_t) The input value of x in 6x + i.
i	(int) The value of i in 6x + i.

Definition at line 37 of file iZ_toolkit.c.

Referenced by iZ_next_prime(), SiZ_count(), vx_prob_sieve(), vx_search_prime(), vx_set_base_values(), vx_stream_file(), and vy_search_prime().

6.4.3.7 iZm_clone()

```
IZM * iZm_clone (
            IZM * src)
```

Deep-copy an IZM object for per-worker ownership.

Deep-copy an IZM object for independent worker usage.

Parameters

src	Source IZM instance.

Returns

Newly allocated clone, or NULL on failure.

Parameters

src	Source IZM object.

Returns

Cloned IZM object, or NULL on failure.

Definition at line 209 of file iZ_toolkit.c.

References UI64_ARRAY::array, IZM::base_x5, IZM::base_x7, bitmap_clone(), UI64_ARRAY::capacity, UI64_ARRAY::count, iZm_free(), IZM::k_vx, log_error(), IZM::root_primes, ui64_init(), and IZM::vx.

Referenced by SiZ_count().

6.4.3.8 iZm_construct_vx_base()

```
void iZm_construct_vx_base (
            uint64_t vx,
            BITMAP * base_x5,
            BITMAP * base_x7)
```

Build pre-sieved base bitmaps for a VX segment.

Constructs a pre-sieved iZm base segment of size vx.

Parameters

vx	Segment width.
base_x5	Output bitmap for 6x-1 candidates.
base_x7	Output bitmap for 6x+1 candidates.

@description: This function constructs a pre-sieved iZm base segment of size vx. It marks all composites of small primes that divide vx in the bitmaps base_x5 (iZ-) and base_x7 (iZ+). This pre-sieved base can then be used as a template for sieving all vx segments.

Parameters:

Parameters

iZm	Pointer to the IZM structure containing the base segment bitmaps.

Definition at line 347 of file iZ_toolkit.c.

References bitmap_clear_bit(), bitmap_clear_steps_simd(), bitmap_set_all(), s_primes, and s_primes_count.

Referenced by iZm_init(), and SiZm().

6.4.3.9 iZm_free()

```
void iZm_free (
            IZM ** iZm)
```

Release an IZM object and set the caller pointer to NULL.

Free the memory allocated for an IZM structure.

Parameters

iZm	Address of an IZM pointer.

Parameters:

Parameters

iZm	A pointer to the IZM structure to be freed.

Definition at line 243 of file iZ_toolkit.c.

References bitmap_free(), and ui64_free().

Referenced by iZ_next_prime(), iZm_clone(), iZm_init(), SiZ_count(), and SiZ_stream().

6.4.3.10 iZm_init()

```
IZM * iZm_init (
            size_t vx)
```

Allocate and initialize an IZM object for a given VX.

Initialize an iZm structure for a given vx size.

Parameters

vx	Segment width; must be odd, not divisible by 3, and >= 35.

Returns

Initialized IZM object, or NULL on failure.

This function initializes an iZm structure for a specified vx size. It allocates memory for the structure, generates root primes for deterministic sieving, initializes base bitmaps, and constructs pre-sieved base segments for iZm5 and iZm7.

Parameters:

Parameters

vx	The size of the segment for the iZm structure.

Returns

A pointer to the initialized IZM structure.

Definition at line 161 of file iZ_toolkit.c.

References IZM::base_x5, IZM::base_x7, bitmap_init(), compute_k_vx(), iZm_construct_vx_base(), iZm_free(), IZM::k_vx, log_error(), IZM::root_primes, SiZ(), ui64_free(), and IZM::vx.

Referenced by iZ_next_prime(), SiZ_count(), and SiZ_stream().

6.4.3.11 iZm_solve_for_xp()

```
uint64_t iZm_solve_for_xp (
            int m_id,
            uint64_t p,
            uint64_t vx,
            uint64_t y)
```

Solve first x-hit of prime p for line m_id in segment y.

Compute the first composite hit of p in the yth vx segment in iZm index space.

Parameters

m_id	Line id (-1 for x5, +1 for x7).

p	Prime used for marking.
vx	Segment width.
y	Segment index.

Returns

First x index to clear for this prime/line.

@description: This function solves for the smallest x that satisfies: (x + vx ∗ y) x_p (mod p), where x_p is normalized based on the m_id and p to either x_p or p - x_p, then computes x and returns it.

Parameters:

Parameters

m_id	int indicating the matrix type (-1 for iZm5, 1 for iZm7).
p	Unsigned 64-bit integer parameter.
vx	size_t parameter representing the vx value.
y	Unsigned 64-bit integer parameter.

Returns

The computed x value as a 64-bit unsigned integer.

Definition at line 399 of file iZ_toolkit.c.

Referenced by SiZm(), and vx_det_sieve().

6.4.3.12 iZm_solve_for_xp_mpz()

```
uint64_t iZm_solve_for_xp_mpz (
            int m_id,
            uint64_t p,
            uint64_t vx,
            mpz_t y)
```

GMP variant of iZm_solve_for_xp() for very large y.

Solve for x given m_id, p, vx, and y using GMP. Same as above but using GMP. Suitable for arbitrary y values.

Parameters

m_id	Line id (-1 for x5, +1 for x7).
p	Prime used for marking.
vx	Segment width.
y	Segment index as mpz.

Returns

First x index to clear for this prime/line.

Parameters:

Parameters

m_id	int indicating the target matrix (-1 for iZm5, 1 for iZm7).

p	Unsigned 64-bit integer parameter.
vx	Size_t parameter representing the vx value.
y	mpz_t parameter representing the y value.

Returns

The computed x value as a 64-bit unsigned integer.

Definition at line 433 of file iZ_toolkit.c.

Referenced by vx_det_sieve(), and vx_search_prime().

6.4.3.13 iZm_solve_for_yp()

```
int64_t iZm_solve_for_yp (
            int m_id,
            uint64_t p,
            uint64_t vx,
            uint64_t x)
```

Solve first y-hit for fixed x in vertical (vy) scanning.

Compute the first composite hit of p in the xth vy segment in iZm of width vx.

Parameters

m_id	Line id (-1 for x5, +1 for x7).
p	Prime used for marking.
vx	Segment width.
x	Fixed x-coordinate.

Returns

y index on success, -1 when no modular solution exists.

@description: This function solves for the smallest y that satisfies the equation (x + vx ∗ y) x_p (mod p), where x_p is normalized based on the m_id and p. The function checks if p and vx are co-primes, and if not, it returns -1 indicating no solution can be found. If they are co-primes, it calculates the multiplicative inverse of vx modulo p and computes y using the formula y = (delta ∗ vx_inv) % p, where delta is the difference between x_p and x modulo p.

Parameters:

Parameters

m_id	Integer indicating the matrix type (-1 for iZm5, 1 for iZm7).
p	Unsigned 64-bit integer parameter.

vx	Size_t parameter representing the vx value.
x	Unsigned 64-bit integer parameter.

Returns

The computed y value as a 64-bit unsigned integer.

Definition at line 474 of file iZ_toolkit.c.

References gcd(), and modular_inverse().

Referenced by SiZm_vy().

6.4.3.14 process_iZ_bitmaps()

```
void process_iZ_bitmaps (
            UI64_ARRAY * primes,
            BITMAP * x5,
            BITMAP * x7,
            uint64_t x_limit)
```

Traverse iZ bitmaps, emit surviving primes, and mark composites.

Consume iZ candidate bitmaps, emit primes, and mark root composites.

Parameters

primes	Output prime array.
x5	Bitmap for 6x-1 candidates.
x7	Bitmap for 6x+1 candidates.
x_limit	Exclusive x upper bound.
primes	Destination array for discovered primes.
x5	Bitmap for 6x-1 candidates.
x7	Bitmap for 6x+1 candidates.
x_limit	Exclusive x upper bound.

Definition at line 70 of file iZ_toolkit.c.

References bitmap_clear_steps_simd(), bitmap_get_bit(), iZ(), and ui64_push().

Referenced by get_root_primes(), SiZ(), and SiZm().

6.4.3.15 vx_collect_p_gaps()

```
void vx_collect_p_gaps (
            VX_SEG * vx_obj)
```

Extract prime-gap encoding from a fully sieved segment.

Convert survivor bits into compact prime-gap encoding.

Parameters

vx_obj	Segment object.

This routine must run after deterministic/probabilistic sieving is complete. It populates VX_SEG::p_gaps for downstream streaming and gap traversal.

Parameters

vx_obj	Segment object with validated prime survivors.

Definition at line 773 of file iZ_toolkit.c.

References bitmap_get_bit(), VX_SEG::is_large_limit, VX_SEG::p_count, VX_SEG::p_gaps, VX_SEG::start_x, ui16_init(), ui16_push(), vx_full_sieve(), VX_SEG::x5, VX_SEG::x7, and VX_SEG::y.

Referenced by vx_full_sieve().

6.4.3.16 vx_free()

```
void vx_free (
            VX_SEG ** vx_obj)
```

Free a VX segment and all owned resources.

Free all memory owned by a VX segment object.

Parameters

vx_obj	Address of a VX_SEG pointer.
vx_obj	Address of the VX_SEG pointer to release.

Definition at line 745 of file iZ_toolkit.c.

References bitmap_free(), and ui16_free().

Referenced by SiZ_count(), and SiZ_stream().

6.4.3.17 vx_full_sieve()

```
void vx_full_sieve (
            VX_SEG * vx_obj,
            int collect_p_gaps)
```

Complete segment processing (probabilistic stage and optional gaps).

This function performs the sieve process on a given vx and y defined in the VX_SEG structure, and stores the primes gaps in the vx_obj->p_gaps array.

Parameters

vx_obj	Segment object.

collect_p_gaps	Non-zero to populate VX_SEG::p_gaps.

@description: This function combines deterministic sieving and probabilistic primality tests to identify prime candidates in a standard VX segment of a specific y in the iZ-Matrix. It could be used to only count primes in the vx segment, and optionally populates the vx_obj->p_gaps array with prime gaps between consecutive primes detected in the segment.

Parameters

vx_obj	The VX_SEG to be processed.
collect_p_gaps	0 only counts primes and doesn't store prime gaps, other non-zero int values populates vx_obj->p_gaps with detected prime gaps.

Definition at line 833 of file iZ_toolkit.c.

References VX_SEG::is_large_limit, vx_collect_p_gaps(), and vx_prob_sieve().

Referenced by SiZ_count(), SiZ_stream(), and vx_collect_p_gaps().

6.4.3.18 vx_init()

```
VX_SEG * vx_init (
            IZM * iZm,
            int start_x,
            int end_x,
            char * y_str,
            int mr_rounds)
```

Initialize and deterministically sieve one VX segment.

Initialize the members of the VX_SEG structure with the given parameters and defaults.

Parameters

iZm	Initialized toolkit context.
start_x	Inclusive x start index.
end_x	Inclusive x end index.
y_str	Segment index y as a decimal string.
mr_rounds	Miller-Rabin rounds (0 uses default).

Returns

Initialized and partially processed segment, or NULL on failure.

@description: This function allocates memory for a VX_SEG structure and initializes its members. The count is initialized to 0, and the p_gaps array is allocated with an initial size of (vx/2).

Parameters:

Parameters

y_str	A character pointer representing a numeric string.

Returns

VX_SEG* A pointer to the initialized VX_SEG structure. NULL if memory allocation fails or y_str is not a numeric string.

Definition at line 697 of file iZ_toolkit.c.

References IZM::base_x5, IZM::base_x7, VX_SEG::bit_ops, bitmap_clone(), VX_SEG::end_x, log_error(), MR_ROUNDS, VX_SEG::mr_rounds, VX_SEG::p_count, VX_SEG::p_gaps, VX_SEG::p_test_ops, VX_SEG::start_x, IZM::vx, VX_SEG::vx, vx_det_sieve(), vx_set_base_values(), VX_SEG::x5, and VX_SEG::x7.

Referenced by SiZ_count(), and SiZ_stream().

6.4.3.19 vx_search_prime()

```
int vx_search_prime (
            mpz_t p,
            int m_id,
            int vx,
            int bit_size)
```

Horizontal iZm/VX random-prime search.

horizontal search routine for generating a random prime.

Parameters

p	Output prime.
m_id	Requested line id (-1, +1, or random when other value).
vx	Segment width.
bit_size	Target bit size.

Returns

1 on success, otherwise 0.

@description: This function searches for a prime number using the given parameters. It combines the iZ-Matrix filtering techniques with the Miller-Rabin primality test. The search is performed by finding suitable x values that do not correspond to composites of primes that divide vx. Then, it iterates over y values in the equation p = iZ(x + vx * y) until a prime is found.

Parameters

p	The prime number found in the search.
m_id	The identifier (1 or -1) for the iZ matrix.
vx	The horizontal vector of the iZ matrix.
bit_size	The target bit size of the prime.

Returns

1 if a prime is found, 0 otherwise.

Definition at line 946 of file iZ_toolkit.c.

References UI64_ARRAY::array, bitmap_clear_steps(), bitmap_free(), bitmap_get_bit(), bitmap_init(), UI64_ARRAY::count, gmp_seed_randstate(), iZ_mpz(), iZm_solve_for_xp_mpz(), log_error(), MR_ROUNDS, SiZm(), and ui64_free().

Referenced by vx_random_prime().

6.4.3.20 vx_stream_file()

```
void vx_stream_file (
            VX_SEG * vx_obj,
            FILE * output)
```

Stream segment primes to a file.

Stream segment primes to an output file in traversal order.

Parameters

vx_obj	Segment object.
output	Writable output stream.
vx_obj	Segment object.
output	Destination stream.

Definition at line 852 of file iZ_toolkit.c.

References bitmap_clear_bit(), bitmap_get_bit(), VX_SEG::is_large_limit, iZ_mpz(), VX_SEG::mr_rounds, VX_SEG::p_count, VX_SEG::p_test_ops, VX_SEG::start_x, VX_SEG::x5, VX_SEG::x7, and VX_SEG::yvx.

Referenced by SiZ_stream().

6.4.3.21 vy_search_prime()

```
int vy_search_prime (
            mpz_t p,
            int m_id,
            mpz_t vx)
```

Vertical iZm/VY random-prime search.

vertical search routine for generating a random prime.

Parameters

p	Output prime.

m_id	Requested line id (-1, +1, or random when other value).
vx	Segment width.

Returns

1 on success, otherwise 0.

@description: This function searches for a prime number using the given parameters. It combines the iZ-Matrix filtering techniques with the Miller-Rabin primality test. The search is performed by finding a suitable x value that does not correspond to a composite of a prime that divides vx. Then, it iterates over y value in the equation p = iZ(x + vx ∗ y, m_id) until a prime is found.

Parameters

p	The prime number found in the search.
m_id	The identifier (1 or -1) for the iZ matrix.
vx	The horizontal vector of the iZ matrix.

Returns

1 if a prime is found, 0 otherwise.

Definition at line 1043 of file iZ_toolkit.c.

References gmp_seed_randstate(), iZ_mpz(), and MR_ROUNDS.

Referenced by vy_random_prime().

6.5 Logging

Thread-safe logging and test-output helpers.

Files

- file logger.c
  Logging module.

Macros

- #define LOGGER_FORMAT_PRINTF(fmt_idx, arg_idx)
  Compiler attribute wrapper for printf-style format checking.
- #define LOG_DIR "logs/"
  Directory where logs are stored.
- #define LOG_FILE LOG_DIR "log.txt"
  Default log file.
- #define LOG_MAX_SIZE 1024 ∗ 1024 ∗ 5
  Maximum log file size (5 MB).

Enumerations

- enum LogLevel {
  LOG_DEBUG , LOG_INFO , LOG_WARNING , LOG_ERROR ,
  LOG_FATAL }

  Enumeration of log levels.

Functions

- const char * log_level_to_string (LogLevel level)

  Returns a string representation of the log level.
- void log_init (const char *log_file)

  Initializes the logging system.
- void log_shutdown (void)

  Shuts down the logging system and cleans up resources.
- void log_set_log_level (LogLevel level)

  Sets the current log level. Messages below this level will not be logged.
- void log_message (LogLevel level, const char *format,...) LOGGER_FORMAT_PRINTF(2

  Logs a formatted message at the given log level.
- void void log_message_extended (LogLevel level, const char *file_name, int line_number, const char *format,...) LOGGER_FORMAT_PRINTF(4

  Logs a formatted message with extended information (file name, line number).
- void void void log_console (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a timestamped message to the console only, without requiring a log level.
- void void void void log_test (int result, int test_id, const char *unit_name, const char *format,...) LOGGER_FORMAT_PRINTF(4

  Convenience function for logging test results.
- void void void void void log_test_summary (char *module_name, int passed, int failed, int verbose)

  Logs a summary of test results for a module.
- void log_debug (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a debug message.
- void void log_info (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs an info message.
- void void void log_warn (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a warning message.
- void void void void log_error (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs an error message.
- void void void void void log_fatal (const char *format,...) LOGGER_FORMAT_PRINTF(1

  Logs a fatal message.

6.5.1 Detailed Description

Thread-safe logging and test-output helpers.

6.5.2 Macro Definition Documentation

6.5.2.1 LOG_DIR

#define LOG_DIR "logs/"

Directory where logs are stored.

Definition at line 54 of file logger.h.

6.5.2.2 LOG_FILE

#define LOG_FILE LOG_DIR "log.txt"

Default log file.

Definition at line 55 of file logger.h.

6.5.2.3 LOG_MAX_SIZE

#define LOG_MAX_SIZE 1024 * 1024 * 5

Maximum log file size (5 MB).

Definition at line 56 of file logger.h.

6.5.2.4 LOGGER_FORMAT_PRINTF

```
#define LOGGER_FORMAT_PRINTF(
            fmt_idx,
            arg_idx)
```

Compiler attribute wrapper for printf-style format checking.

Definition at line 50 of file logger.h.

Referenced by log_console(), log_debug(), log_error(), log_fatal(), log_info(), log_message(), log_message_extended(), log_test(), and log_warn().

6.5.3 Enumeration Type Documentation

6.5.3.1 LogLevel

enum LogLevel

Enumeration of log levels.

This enumeration defines the different log levels that can be used in the logging system. Each level corresponds to a severity of importance.

Enumerator

LOG_DEBUG	Debug level.
LOG_INFO	Info level.
LOG_WARNING	Warning level.
LOG_ERROR	Error level.
LOG_FATAL	Fatal level.

Definition at line 65 of file logger.h.

6.5.4 Function Documentation

6.5.4.1 log_console()

```
void void void log_console (
            const char * format,
            ...)
```

Logs a timestamped message to the console only, without requiring a log level.

This can be used for debugging or tracking specific points in the code.

Parameters

format	The format string for the log message.

References log_console(), and LOGGER_FORMAT_PRINTF.

Referenced by log_console().

6.5.4.2 log_debug()

```
void log_debug (
            const char * format,
            ...)
```

Logs a debug message.

Parameters

format	The format string for the log message.

References log_debug(), and LOGGER_FORMAT_PRINTF.

Referenced by iZ_next_prime(), and log_debug().

6.5.4.3 log_error()

```
void void void void log_error (
            const char * format,
            ...)
```

Logs an error message.

Parameters

format	The format string for the log message.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

References log_error(), and LOGGER_FORMAT_PRINTF.

Referenced by bitmap_fread(), bitmap_fwrite(), bitmap_init(), bitmap_validate_hash(), create_dir(), iZ_next_prime(), iZm_clone(), iZm_init(), log_error(), SiZ_count(), SiZ_stream(), vx_init(), vx_random_prime(), vx_search_prime(), vx_set_base_values(), and vy_random_prime().

6.5.4.4 log_fatal()

```
void void void void void log_fatal (
            const char * format,
            ...)
```

Logs a fatal message.

Parameters

format	The format string for the log message.

References log_fatal(), and LOGGER_FORMAT_PRINTF.

Referenced by log_fatal().

6.5.4.5 log_info()

```
void void log_info (
            const char * format,
            ...)
```

Logs an info message.

Parameters

format	The format string for the log message.

References log_info(), and LOGGER_FORMAT_PRINTF.

Referenced by log_info(), and vx_prob_sieve().

6.5.4.6 log_init()

```
void log_init (
            const char * log_file)
```

Initializes the logging system.

Creates the log directory if it doesn't exist and rotates logs if necessary.

Parameters

log_file	The path to the log file.

6.5.4.7 log_level_to_string()

```
const char * log_level_to_string (
            LogLevel level)
```

Returns a string representation of the log level.

Parameters

level	The log level.

Returns

A string corresponding to the log level.

6.5.4.8 log_message()

```
void log_message (
            LogLevel level,
            const char * format,
            ...)
```

Logs a formatted message at the given log level.

This function logs messages to the log file in a thread-safe manner.

Parameters

level	The log level for the message.
format	The format string for the log message.

References LOGGER_FORMAT_PRINTF.

6.5.4.9 log_message_extended()

```
void void log_message_extended (
            LogLevel level,
            const char * file_name,
            int line_number,
            const char * format,
            ...)
```

Logs a formatted message with extended information (file name, line number).

This function logs messages to the log file in a thread-safe manner.

Parameters

level	The log level for the message.

file_name	The source file where the log was generated.
line_number	The line number in the source file.
format	The format string for the log message.

References log_message_extended(), and LOGGER_FORMAT_PRINTF.

Referenced by log_message_extended().

6.5.4.10 log_set_log_level()

```
void log_set_log_level (
            LogLevel level)
```

Sets the current log level. Messages below this level will not be logged.

Parameters

level	The log level to set.

6.5.4.11 log_test()

```
void void void void log_test (
            int result,
            int test_id,
            const char * unit_name,
            const char * format,
            ...)
```

Convenience function for logging test results.

This function logs test results to the console, indicating pass or fail status.

Parameters

result	The result of the test (1 for pass, 0 for fail).
test_id	The identifier for the test.
unit_↩ name	The name of the unit being tested.
format	The format string for the log message.

References log_test(), and LOGGER_FORMAT_PRINTF.

Referenced by log_test().

6.5.4.12 log_test_summary()

```
void void void void void log_test_summary (
            char * module_name,
            int passed,
            int failed,
            int verbose)
```

Logs a summary of test results for a module.

This function prints a summary of the number of passed and failed tests for a given module to the console.

Parameters

module_name	The name of the module being tested.

passed	The number of tests that passed.
failed	The number of tests that failed.
verbose	If non-zero, it suggests verbose mode if there are errors.

References log_test_summary().

Referenced by log_test_summary().

6.5.4.13 log_warn()

```
void void void log_warn (
            const char * format,
            ...)
```

Logs a warning message.

Parameters

format	The format string for the log message.

References log_warn(), and LOGGER_FORMAT_PRINTF.

Referenced by log_warn().

6.6 Tests and Benchmarks

Test harness entry points.

Data Structures

- struct SIEVE_LIMIT
- struct SIEVE_MODEL

Typedefs

- typedef UI64_ARRAY *(* SIEVE_FN) (uint64_t)

Sieve model tests/benchmarks

- int TEST_SIEVE_MODELS_INTEGRITY (int verbose)

  Run correctness checks across all registered sieve models.
- void BENCHMARK_SIEVE_MODELS (int save_results)

  Benchmark registered sieve models and optionally persist results.

Range sieve tests/benchmarks

- int TEST_SiZ_stream (int verbose)

  Validate SiZ_stream behavior and output formatting.
- int TEST_SiZ_count (int verbose)

  Validate SiZ_count correctness across worker counts.

Prime generation tests/benchmarks

- int TEST_iZ_next_prime (int verbose)

  Validate iZ_next_prime for forward/backward traversal cases.
- int TEST_vy_random_prime (int verbose)

  Validate vy_random_prime for primality and bit-length constraints.
- int TEST_vx_random_prime (int verbose)

  Validate vx_random_prime for primality and bit-length constraints.
- int BENCHMARK_P_GEN_ALGORITHMS (int bit_size, int test_rounds, int save_results)

  Benchmark random-prime generation routines over repeated trials.

6.6.1 Detailed Description

Test harness entry points.

6.6.2 Typedef Documentation

6.6.2.1 SIEVE_FN

typedef UI64_ARRAY *(* SIEVE_FN) (uint64_t)

Sieve function type: takes a limit and returns a prime list.

Definition at line 20 of file test_api.h.

6.7 Utilities

Cross-cutting helpers used across modules.

Files

- file utils.c

  Implementations for the shared utility layer.

Macros

- #define DIR_output "./output"
- #define MAX_CORES get_cpu_cores_count()

Functions

- int create_dir (const char *dir)

  Create a directory if it does not exist.
- int is_numeric_str (const char *str)

  Return non-zero if str contains only ASCII digits.
- uint64_t gcd (uint64_t a, uint64_t b)

  Compute the greatest common divisor of a and b.
- uint64_t modular_inverse (uint64_t a, uint64_t m)

  Compute the modular inverse of a modulo m.
- void print_line (int length, char fill_char)

  Print a repeated character line to stdout.
- void print_centered_text (const char *text, int line_length, char fill_char)

  Print text centered within a filled line.
- void print_sha256_hash (const unsigned char *hash)

  Print a SHA-256 digest as hex to stdout.
- void gmp_seed_randstate (gmp_randstate_t state)

  Seed a GMP random state.
- int get_cpu_cores_count (void)

  Get the number of online CPU cores (>= 1).
- int get_cpu_L2_cache_size_bits (void)

  Return the CPU L2 cache size in bits (best effort).
- void print_module_header (char *module_name)

  Print a test-suite header banner for a module.
- void print_test_header (void)

  Print the generic test runner header.

6.7.1 Detailed Description

Cross-cutting helpers used across modules.

6.7.2 Macro Definition Documentation

6.7.2.1 DIR_output

#define DIR_output "./output"

Default directory for output artifacts produced by examples/tests.

Definition at line 46 of file utils.h.

6.7.2.2 MAX_CORES

#define MAX_CORES get_cpu_cores_count()

Convenience macro: number of online CPU cores.

Definition at line 48 of file utils.h.

6.7.3 Function Documentation

6.7.3.1 create_dir()

```
int create_dir (
            const char * dir)
```

Create a directory if it does not exist.

Parameters

dir	Directory path.

Returns

0 on success (including already exists), -1 on failure.

Parameters

dir	The directory path.

Returns

int 0 if the directory already exists or was successfully created, -1 otherwise.

Definition at line 89 of file utils.c.

References log_error().

6.7.3.2 gcd()

```
uint64_t gcd (
            uint64_t a,
            uint64_t b)
```

Compute the greatest common divisor of a and b.

Compute the greatest common divisor of a and b.

Parameters

a	The first integer.
b	The second integer.

Returns

int The GCD of a and b.

Definition at line 112 of file utils.c.

Referenced by iZm_solve_for_yp(), and SiZm_vy().

6.7.3.3 get_cpu_cores_count()

```
int get_cpu_cores_count (
                void )
```

Get the number of online CPU cores (>= 1).

Get the number of online CPU cores (>= 1).

Returns

int The number of CPU cores.

Definition at line 196 of file utils.c.

Referenced by SiZ_count().

6.7.3.4 get_cpu_L2_cache_size_bits()

```
int get_cpu_L2_cache_size_bits (
                void )
```

Return the CPU L2 cache size in bits (best effort).

Returns

Cache size in bits, or a conservative default.

Return the CPU L2 cache size in bits (best effort).

@description: This function retrieves the L2 cache size using platform-specific methods:

- Linux: Reads from sysfs (/sys/devices/system/cpu/cpu0/cache/index2/size)
- macOS/BSD: Uses sysctl to query hw.l2cachesize
- Fallback: Returns 256 KB as a conservative default

The function attempts multiple detection methods to ensure robustness across different architectures and operating systems.

Returns

int The L2 cache size in bits, or 256 KB if unable to determine.

Definition at line 218 of file utils.c.

Referenced by compute_l2_vx().

6.7.3.5 gmp_seed_randstate()

```
void gmp_seed_randstate (
            gmp_randstate_t state)
```

Seed a GMP random state.

Attempts to seed from /dev/urandom, falling back to time-based seeding.

Seed a GMP random state.

@description: This function seeds the GMP random state using /dev/urandom.

Parameters

state	The GMP random state.

Definition at line 173 of file utils.c.

Referenced by vx_search_prime(), and vy_search_prime().

6.7.3.6 is_numeric_str()

```
int is_numeric_str (
            const char * str)
```

Return non-zero if str contains only ASCII digits.

Parameters

str	Input string.

Returns

1 if numeric, 0 otherwise.

Return non-zero if str contains only ASCII digits.

Parameters

str	Pointer to the string.

Returns

int 1 if the string is numeric, 0 otherwise.

Definition at line 15 of file utils.c.

6.7.3.7 modular_inverse()

```
uint64_t modular_inverse (
            uint64_t a,
            uint64_t m)
```

Compute the modular inverse of a modulo m.

Returns

The inverse in [0, m-1] if it exists; otherwise an implementation-defined value.

Compute the modular inverse of a modulo m.

@description: This function computes the modular inverse of a modulo m using the Extended Euclidean Algorithm. The function takes two integers a and m as input and returns the modular inverse of a modulo m. If a and m are not co-prime, the function returns -1 indicating no modular inverse exists. The function also handles the case where m is 1, returning 0 as the modular inverse.

Parameters:

Parameters

a	The integer for which the modular inverse is to be computed.
m	The modulus.

Returns

The modular inverse of a modulo m, or -1 if no inverse exists.

Definition at line 140 of file utils.c.

Referenced by iZm_solve_for_yp().

6.7.3.8 print_centered_text()

```
void print_centered_text (
            const char * text,
            int line_length,
            char fill_char)
```

Print text centered within a filled line.

Print text centered within a filled line.

Parameters

text	Text payload.
line_length	Total line width.
fill_char	Padding character.

Definition at line 51 of file utils.c.

6.7.3.9 print_line()

```
void print_line (
            int length,
            char fill_char)
```

Print a repeated character line to stdout.

Print a repeated character line to stdout.

Parameters

length	Number of characters to print.

fill_char	Character used to fill the line ('-' when zero).

Definition at line 34 of file utils.c.

Referenced by print_module_header(), and print_test_header().

6.7.3.10 print_module_header()

```
void print_module_header (
            char * module_name)
```

Print a test-suite header banner for a module.

Print a test-suite header banner for a module.

Parameters

module_name	Display name of the tested module.

Definition at line 273 of file utils.c.

References print_line().

6.7.3.11 print_sha256_hash()

```
void print_sha256_hash (
            const unsigned char * hash)
```

Print a SHA-256 digest as hex to stdout.

Print a SHA-256 digest as hex to stdout.

Parameters

hash	Pointer to the SHA-256 hash array.

Definition at line 75 of file utils.c.

6.7.3.12 print_test_header()

```
void print_test_header (
            void )
```

Print the generic test runner header.

Print the generic test runner header.

Definition at line 283 of file utils.c.

References print_line().

Chapter 7

Data Structure Documentation

7.1 BITMAP Struct Reference

Packed bit-array with optional SHA-256 checksum.

#include <bitmap.h>

Data Fields

- size_t size
- size_t byte_size
- unsigned char * data
- unsigned char sha256 [SHA256_DIGEST_LENGTH]

7.1.1 Detailed Description

Packed bit-array with optional SHA-256 checksum.

size is measured in bits and byte_size is the backing storage in bytes. The checksum is maintained explicitly via bitmap_compute_hash() and verified via bitmap_validate_hash().

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 26 of file bitmap.h.

7.1.2 Field Documentation

7.1.2.1 byte_size

size_t BITMAP::byte_size

Number of bytes in data.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 29 of file bitmap.h.

Referenced by bitmap_clone(), bitmap_compute_hash(), bitmap_fread(), bitmap_fwrite(), bitmap_init(), bitmap_validate_hash(), and SiZm().

7.1.2.2 data

unsigned char* BITMAP::data

Packed bits (LSB-first per byte).

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 30 of file bitmap.h.

Referenced by bitmap_clear_all(), bitmap_clear_bit(), bitmap_clear_steps(), bitmap_clear_steps_simd(), bitmap_clone(), bitmap_compute_hash(), bitmap_flip_bit(), bitmap_fread(), bitmap_fwrite(), bitmap_get_bit(), bitmap_init(), bitmap_set_all(), bitmap_set_bit(), bitmap_validate_hash(), and SiZm().

7.1.2.3 sha256

unsigned char BITMAP::sha256[SHA256_DIGEST_LENGTH]

Cached SHA-256 checksum.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 31 of file bitmap.h.

Referenced by bitmap_clone(), bitmap_compute_hash(), bitmap_fread(), bitmap_fwrite(), bitmap_init(), and bitmap_validate_hash().

7.1.2.4 size

size_t BITMAP::size

Number of addressable bits.

Examples

/Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/bitmap.c.

Definition at line 28 of file bitmap.h.

Referenced by bitmap_clear_all(), bitmap_clear_steps(), bitmap_clear_steps_simd(), bitmap_clone(), bitmap_fwrite(), bitmap_init(), and bitmap_set_all().

The documentation for this struct was generated from the following file:

- include/bitmap.h

7.2 INPUT_SIEVE_RANGE Struct Reference

Input parameters for range sieving/counting.

#include <iZ_api.h>

Data Fields

- char * start
  Start of range as a base-10 numeric string.
- uint64_t range
  Interval size (number of integers to cover).
- int mr_rounds
  Miller–Rabin rounds for large primality checks.
- char * filepath
  Output path for streaming primes (NULL to disable output).

7.2.1 Detailed Description

Input parameters for range sieving/counting.

The interval is interpreted as:

- Start Z_s from the decimal string start.
- End $Z_e = Z_s + range - 1$.

Definition at line 127 of file iZ_api.h.

7.2.2 Field Documentation

7.2.2.1 filepath

char* INPUT_SIEVE_RANGE::filepath

Output path for streaming primes (NULL to disable output).

Definition at line 132 of file iZ_api.h.

Referenced by SiZ_stream().

7.2.2.2 mr_rounds

int INPUT_SIEVE_RANGE::mr_rounds

Miller–Rabin rounds for large primality checks.

Definition at line 131 of file iZ_api.h.

Referenced by SiZ_count(), and SiZ_stream().

7.2.2.3 range

uint64_t INPUT_SIEVE_RANGE::range

Interval size (number of integers to cover).

Definition at line 130 of file iZ_api.h.

Referenced by SiZ_count(), and SiZ_stream().

7.2.2.4 start

char* INPUT_SIEVE_RANGE::start

Start of range as a base-10 numeric string.

Definition at line 129 of file iZ_api.h.

Referenced by SiZ_count(), and SiZ_stream().

The documentation for this struct was generated from the following file:

- include/iZ_api.h

7.3 IZM Struct Reference

Precomputed iZm assets for repeated VX-segment sieving.

#include <iZ_toolkit.h>

Data Fields

- int vx
- int k_vx
- BITMAP * base_x5
- BITMAP * base_x7
- UI64_ARRAY * root_primes

7.3.1 Detailed Description

Precomputed iZm assets for repeated VX-segment sieving.

Definition at line 73 of file iZ_toolkit.h.

7.3.2 Field Documentation

7.3.2.1 base_x5

BITMAP* IZM::base_x5

Pre-sieved base bitmap for 6x-1 line.

Definition at line 77 of file iZ_toolkit.h.

Referenced by iZ_next_prime(), iZm_clone(), iZm_init(), and vx_init().

7.3.2.2 base_x7

BITMAP* IZM::base_x7

Pre-sieved base bitmap for 6x+1 line.

Definition at line 78 of file iZ_toolkit.h.

Referenced by iZ_next_prime(), iZm_clone(), iZm_init(), and vx_init().

7.3.2.3 k_vx

int IZM::k_vx

Count of small primes dividing vx (excluding 2,3).

Definition at line 76 of file iZ_toolkit.h.

Referenced by iZm_clone(), iZm_init(), and vx_det_sieve().

7.3.2.4 root_primes

UI64_ARRAY* IZM::root_primes

Root primes used for deterministic marking.

Definition at line 79 of file iZ_toolkit.h.

Referenced by iZm_clone(), iZm_init(), and vx_det_sieve().

7.3.2.5 vx

int IZM::vx

Segment width in iZ x-units.

Definition at line 75 of file iZ_toolkit.h.

Referenced by compute_k_vx(), iZm_clone(), iZm_init(), and vx_init().

The documentation for this struct was generated from the following file:

- include/iZ_toolkit.h

7.4 SIEVE_LIMIT Struct Reference

#include <test_api.h>

Data Fields

- int base
- int exp

7.4.1 Detailed Description

Limit specification as $base^{exp}$.

Definition at line 23 of file test_api.h.

7.4.2 Field Documentation

7.4.2.1 base

int SIEVE_LIMIT::base

Definition at line 25 of file test_api.h.

7.4.2.2 exp

int SIEVE_LIMIT::exp

Definition at line 26 of file test_api.h.

The documentation for this struct was generated from the following file:

- include/test_api.h

7.5 SIEVE_MODEL Struct Reference

#include <test_api.h>

Data Fields

- SIEVE_FN function
- const char name [32]

7.5.1 Detailed Description

Pair a sieve function with a human-readable name (for reports/benchmarks).

Definition at line 30 of file test_api.h.

7.5.2 Field Documentation

7.5.2.1 function

SIEVE_FN SIEVE_MODEL::function

Definition at line 32 of file test_api.h.

7.5.2.2 name

const char SIEVE_MODEL::name[32]

Definition at line 33 of file test_api.h.

The documentation for this struct was generated from the following file:

- include/test_api.h

7.6 UI16_ARRAY Struct Reference

Dynamic array for uint16_t values.

#include <int_arrays.h>

Data Fields

- int capacity
- int count
- uint16_t * array
- unsigned char sha256 [SHA256_DIGEST_LENGTH]

7.6.1 Detailed Description

Dynamic array for uint16_t values.

Definition at line 19 of file int_arrays.h.

7.6.2 Field Documentation

7.6.2.1 array

uint16_t* UI16_ARRAY::array

Contiguous element storage.

Definition at line 23 of file int_arrays.h.

7.6.2.2 capacity

int UI16_ARRAY::capacity

Allocated element capacity.

Definition at line 21 of file int_arrays.h.

7.6.2.3 count

int UI16_ARRAY::count

Number of valid elements.

Definition at line 22 of file int_arrays.h.

7.6.2.4 sha256

unsigned char UI16_ARRAY::sha256[SHA256_DIGEST_LENGTH]

SHA-256 over used payload.

Definition at line 24 of file int_arrays.h.

The documentation for this struct was generated from the following file:

- include/int_arrays.h

7.7 UI32_ARRAY Struct Reference

Dynamic array for uint32_t values.

#include <int_arrays.h>

Data Fields

- int capacity
- int count
- uint32_t * array
- unsigned char sha256 [SHA256_DIGEST_LENGTH]

7.7.1 Detailed Description

Dynamic array for uint32_t values.

Definition at line 28 of file int_arrays.h.

7.7.2 Field Documentation

7.7.2.1 array

uint32_t* UI32_ARRAY::array

Contiguous element storage.

Definition at line 32 of file int_arrays.h.

7.7.2.2 capacity

int UI32_ARRAY::capacity

Allocated element capacity.

Definition at line 30 of file int_arrays.h.

7.7.2.3 count

int UI32_ARRAY::count

Number of valid elements.

Definition at line 31 of file int_arrays.h.

7.7.2.4 sha256

unsigned char UI32_ARRAY::sha256[SHA256_DIGEST_LENGTH]

SHA-256 over used payload.

Definition at line 33 of file int_arrays.h.

The documentation for this struct was generated from the following file:

- include/int_arrays.h

7.8 UI64_ARRAY Struct Reference

Dynamic array for uint64_t values.

#include <int_arrays.h>

Data Fields

- int capacity
- int count
- uint64_t * array
- unsigned char sha256 [SHA256_DIGEST_LENGTH]

7.8.1 Detailed Description

Dynamic array for uint64_t values.

Definition at line 37 of file int_arrays.h.

7.8.2 Field Documentation

7.8.2.1 array

uint64_t* UI64_ARRAY::array

Contiguous element storage.

Definition at line 41 of file int_arrays.h.

Referenced by compute_max_vx(), iZm_clone(), SiZ(), SiZ_count(), SiZ_stream(), SiZm(), SiZm_vy(), SoEu(), SSoE(), vx_det_sieve(), and vx_search_prime().

7.8.2.2 capacity

int UI64_ARRAY::capacity

Allocated element capacity.

Definition at line 39 of file int_arrays.h.

Referenced by iZm_clone().

7.8.2.3 count

int UI64_ARRAY::count

Number of valid elements.

Definition at line 40 of file int_arrays.h.

Referenced by iZm_clone(), SiZ(), SiZ_count(), SiZ_stream(), SiZm(), SiZm_vy(), SoEu(), SSoE(), vx_det_sieve(), and vx_search_prime().

7.8.2.4 sha256

unsigned char UI64_ARRAY::sha256[SHA256_DIGEST_LENGTH]

SHA-256 over used payload.

Definition at line 42 of file int_arrays.h.

The documentation for this struct was generated from the following file:

- include/int_arrays.h

7.9 VX_SEG Struct Reference

Runtime state for one VX segment at a specific y.

#include <iZ_toolkit.h>

Data Fields

- int vx
- mpz_t y
- mpz_t yvx
- mpz_t root_limit
- int is_large_limit
- int mr_rounds
- int start_x
- int end_x
- BITMAP * x5
- BITMAP * x7
- int p_count
- UI16_ARRAY * p_gaps
- int bit_ops
- int p_test_ops

7.9.1 Detailed Description

Runtime state for one VX segment at a specific y.

Definition at line 174 of file iZ_toolkit.h.

7.9.2 Field Documentation

7.9.2.1 bit_ops

int VX_SEG::bit_ops

Approximate deterministic mark operations.

Definition at line 188 of file iZ_toolkit.h.

Referenced by vx_det_sieve(), and vx_init().

7.9.2.2 end_x

int VX_SEG::end_x

Inclusive end x for this segment.

Definition at line 183 of file iZ_toolkit.h.

Referenced by vx_det_sieve(), vx_init(), and vx_prob_sieve().

7.9.2.3 is_large_limit

int VX_SEG::is_large_limit

Non-zero => requires probabilistic primality checks.

Definition at line 180 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), vx_full_sieve(), vx_prob_sieve(), vx_set_base_values(), and vx_stream_file().

7.9.2.4 mr_rounds

int VX_SEG::mr_rounds

Miller-Rabin rounds when probabilistic checks are used.

Definition at line 181 of file iZ_toolkit.h.

Referenced by vx_init(), vx_prob_sieve(), and vx_stream_file().

7.9.2.5 p_count

int VX_SEG::p_count

Count of primes found in this segment.

Definition at line 186 of file iZ_toolkit.h.

Referenced by SiZ_count(), SiZ_stream(), vx_collect_p_gaps(), vx_det_sieve(), vx_init(), vx_prob_sieve(), and vx_stream_file().

7.9.2.6 p_gaps

UI16_ARRAY* VX_SEG::p_gaps

Optional prime-gap encoding for streamed output.

Definition at line 187 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), and vx_init().

7.9.2.7 p_test_ops

int VX_SEG::p_test_ops

Probabilistic primality tests executed.

Definition at line 189 of file iZ_toolkit.h.

Referenced by vx_init(), vx_prob_sieve(), and vx_stream_file().

7.9.2.8 root_limit

mpz_t VX_SEG::root_limit

sqrt(iZ(yvx + vx, +1)).

Definition at line 179 of file iZ_toolkit.h.

Referenced by vx_det_sieve(), and vx_set_base_values().

7.9.2.9 start_x

int VX_SEG::start_x

Inclusive start x for this segment.

Definition at line 182 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), vx_init(), vx_prob_sieve(), and vx_stream_file().

7.9.2.10 vx

int VX_SEG::vx

Segment width.

Definition at line 176 of file iZ_toolkit.h.

Referenced by vx_det_sieve(), vx_init(), and vx_set_base_values().

7.9.2.11 x5

BITMAP* VX_SEG::x5

Candidate bitmap for 6x-1.

Definition at line 184 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), vx_init(), vx_prob_sieve(), and vx_stream_file().

7.9.2.12 x7

BITMAP* VX_SEG::x7

Candidate bitmap for 6x+1.

Definition at line 185 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), vx_init(), vx_prob_sieve(), and vx_stream_file().

7.9.2.13 y

mpz_t VX_SEG::y

Segment index y.

Definition at line 177 of file iZ_toolkit.h.

Referenced by vx_collect_p_gaps(), vx_det_sieve(), and vx_set_base_values().

7.9.2.14 yvx

mpz_t VX_SEG::yvx

Cached product y*vx.

Definition at line 178 of file iZ_toolkit.h.

Referenced by vx_prob_sieve(), vx_set_base_values(), and vx_stream_file().

The documentation for this struct was generated from the following file:

- include/iZ_toolkit.h

Chapter 8

File Documentation

8.1 include/bitmap.h File Reference

Bitmap container and bit operations used by sieve implementations.

#include <utils.h>

Data Structures

- struct BITMAP
  Packed bit-array with optional SHA-256 checksum.

Functions

- int TEST_BITMAP (int verbose)
  Run bitmap module tests.

Lifecycle

- BITMAP * bitmap_init (size_t size, int set_bits)
  Allocate a bitmap with size bits.
- void bitmap_free (BITMAP **bitmap)
  Free a bitmap and set the caller pointer to NULL.
- BITMAP * bitmap_clone (BITMAP *src)
  Deep-copy a bitmap, including data and checksum.

Single-bit Operations

- int bitmap_get_bit (BITMAP *bitmap, size_t idx)
  Read bit value at idx.
- void bitmap_set_bit (BITMAP *bitmap, size_t idx)
  Set bit at idx to 1.
- void bitmap_flip_bit (BITMAP *bitmap, size_t idx)
  Toggle bit at idx.
- void bitmap_clear_bit (BITMAP *bitmap, size_t idx)
  Clear bit at idx (set to 0).

Bulk Operations

- void bitmap_set_all (BITMAP *bitmap)
  Set all bits to 1.
- void bitmap_clear_all (BITMAP *bitmap)
  Clear all bits to 0.
- void bitmap_clear_steps (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
  Clear every step-th bit from start_idx up to limit.
- void bitmap_clear_steps_simd (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
  SIMD-accelerated variant of bitmap_clear_steps().

Integrity and I/O

- void bitmap_compute_hash (BITMAP *bitmap)
  Compute SHA-256 over bitmap data and store it in BITMAP::sha256.
- int bitmap_validate_hash (BITMAP *bitmap)
  Verify BITMAP::sha256 against current bitmap data.
- int bitmap_fwrite (BITMAP *bitmap, FILE *file)
  Write bitmap payload and checksum to a binary stream.
- BITMAP * bitmap_fread (FILE *file)
  Read bitmap payload and checksum from a binary stream.

8.1.1 Detailed Description

Bitmap container and bit operations used by sieve implementations.

The bitmap API is intentionally minimal and fast: a packed bit-array plus helpers for per-bit operations, stepped clearing (for sieving), hashing, and binary serialization.

Definition in file bitmap.h.

8.2 bitmap.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef BITMAP_H
00011 #define BITMAP_H
00012
00013 #include <utils.h>
00014
00018
00026 typedef struct
00027 {
00028     size_t size;
00029     size_t byte_size;
00030     unsigned char *data;
00031     unsigned char sha256[SHA256_DIGEST_LENGTH];
00032 } BITMAP;
00033
00042 BITMAP *bitmap_init(size_t size, int set_bits);
00043
00048 void bitmap_free(BITMAP **bitmap);
00049
00055 BITMAP *bitmap_clone(BITMAP *src);
00057
00066 int bitmap_get_bit(BITMAP *bitmap, size_t idx);
00067
00073 void bitmap_set_bit(BITMAP *bitmap, size_t idx);
00074
00080 void bitmap_flip_bit(BITMAP *bitmap, size_t idx);
00081
```

```
void bitmap_clear_bit(BITMAP *bitmap, size_t idx);

void bitmap_set_all(BITMAP *bitmap);

void bitmap_clear_all(BITMAP *bitmap);

void bitmap_clear_steps(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit);

void bitmap_clear_steps_simd(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit);

void bitmap_compute_hash(BITMAP *bitmap);

int bitmap_validate_hash(BITMAP *bitmap);

int bitmap_fwrite(BITMAP *bitmap, FILE *file);

BITMAP *bitmap_fread(FILE *file);

int TEST_BITMAP(int verbose);


#endif // BITMAP_H
```

8.3 include/int_arrays.h File Reference

Dynamic arrays for uint16_t, uint32_t, and uint64_t values.

#include <utils.h>

Data Structures

- struct UI16_ARRAY
  Dynamic array for uint16_t values.
- struct UI32_ARRAY
  Dynamic array for uint32_t values.
- struct UI64_ARRAY
  Dynamic array for uint64_t values.

Functions

- int TEST_GENERIC_INT_ARRAYS (int verbose)
  Run generic dispatch tests for C11 _Generic helper macros.

UI16 API

- UI16_ARRAY * ui16_init (int capacity)
  Allocate a UI16 array with an initial capacity.
- void ui16_free (UI16_ARRAY **array)
  Free a UI16 array and null the caller pointer.
- void ui16_resize_to (UI16_ARRAY *array, int new_capacity)
  Resize UI16 storage to new_capacity (must be >= count).
- void ui16_resize_to_fit (UI16_ARRAY *array)
  Shrink UI16 storage so capacity equals count.
- void ui16_push (UI16_ARRAY *array, uint16_t element)
  Append a uint16 value, growing storage if needed.
- void ui16_pop (UI16_ARRAY *array)
  Remove the last element if the array is non-empty.
- void ui16_compute_hash (UI16_ARRAY *array)

Compute SHA-256 checksum over active payload.
- int ui16_verify_hash (UI16_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui16_fwrite (UI16_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI16_ARRAY * ui16_fread (FILE *file)

  Deserialize a UI16 array from a binary stream.
- int TEST_UI16_ARRAY (int verbose)

  Execute UI16 test suite.

UI32 API

- UI32_ARRAY * ui32_init (int capacity)

  Allocate a UI32 array with an initial capacity.
- void ui32_free (UI32_ARRAY **array)

  Free a UI32 array and null the caller pointer.
- void ui32_resize_to (UI32_ARRAY *array, int new_capacity)

  Resize UI32 storage to new_capacity (must be >= count).
- void ui32_resize_to_fit (UI32_ARRAY *array)

  Shrink UI32 storage so capacity equals count.
- void ui32_push (UI32_ARRAY *array, uint32_t element)

  Append a uint32 value, growing storage if needed.
- void ui32_pop (UI32_ARRAY *array)

  Remove the last element if the array is non-empty.
- void ui32_compute_hash (UI32_ARRAY *array)

  Compute SHA-256 checksum over active payload.
- int ui32_verify_hash (UI32_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui32_fwrite (UI32_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI32_ARRAY * ui32_fread (FILE *file)

  Deserialize a UI32 array from a binary stream.
- int TEST_UI32_ARRAY (int verbose)

  Execute UI32 test suite.

UI64 API

- UI64_ARRAY * ui64_init (int capacity)

  Allocate a UI64 array with an initial capacity.
- void ui64_free (UI64_ARRAY **array)

  Free a UI64 array and null the caller pointer.
- void ui64_resize_to (UI64_ARRAY *array, int new_capacity)

  Resize UI64 storage to new_capacity (must be >= count).
- void ui64_resize_to_fit (UI64_ARRAY *array)

  Shrink UI64 storage so capacity equals count.
- void ui64_push (UI64_ARRAY *array, uint64_t element)

  Append a uint64 value, growing storage if needed.
- void ui64_pop (UI64_ARRAY *array)

  Remove the last element if the array is non-empty.
- void ui64_compute_hash (UI64_ARRAY *array)

  Compute SHA-256 checksum over active payload.
- int ui64_verify_hash (UI64_ARRAY *array)

  Verify the stored checksum against current payload.
- int ui64_fwrite (UI64_ARRAY *array, FILE *file)

  Serialize count, payload, and checksum to a binary stream.
- UI64_ARRAY * ui64_fread (FILE *file)

  Deserialize a UI64 array from a binary stream.
- int TEST_UI64_ARRAY (int verbose)

  Execute UI64 test suite.

8.3.1 Detailed Description

Dynamic arrays for uint16_t, uint32_t, and uint64_t values.

The module provides uniform typed arrays with append/pop operations, deterministic resizing, optional SHA-256 integrity checks, and binary I/O.

Definition in file int_arrays.h.

8.4 int_arrays.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef INT_ARRAYS_H
00010 #define INT_ARRAYS_H
00011
00012 #include <utils.h>
00013
00017
00019 typedef struct
00020 {
00021     int capacity;
00022     int count;
00023     uint16_t *array;
00024     unsigned char sha256[SHA256_DIGEST_LENGTH];
00025 } UI16_ARRAY;
00026
00028 typedef struct
00029 {
00030     int capacity;
00031     int count;
00032     uint32_t *array;
00033     unsigned char sha256[SHA256_DIGEST_LENGTH];
00034 } UI32_ARRAY;
00035
00037 typedef struct
00038 {
00039     int capacity;
00040     int count;
00041     uint64_t *array;
00042     unsigned char sha256[SHA256_DIGEST_LENGTH];
00043 } UI64_ARRAY;
00044
00048 UI16_ARRAY *ui16_init(int capacity);
00050 void ui16_free(UI16_ARRAY **array);
00052 void ui16_resize_to(UI16_ARRAY *array, int new_capacity);
00054 void ui16_resize_to_fit(UI16_ARRAY *array);
00056 void ui16_push(UI16_ARRAY *array, uint16_t element);
00058 void ui16_pop(UI16_ARRAY *array);
00060 void ui16_compute_hash(UI16_ARRAY *array);
00062 int ui16_verify_hash(UI16_ARRAY *array);
00064 int ui16_fwrite(UI16_ARRAY *array, FILE *file);
00066 UI16_ARRAY *ui16_fread(FILE *file);
00068 int TEST_UI16_ARRAY(int verbose);
00070
00074 UI32_ARRAY *ui32_init(int capacity);
00076 void ui32_free(UI32_ARRAY **array);
00078 void ui32_resize_to(UI32_ARRAY *array, int new_capacity);
00080 void ui32_resize_to_fit(UI32_ARRAY *array);
00082 void ui32_push(UI32_ARRAY *array, uint32_t element);
00084 void ui32_pop(UI32_ARRAY *array);
00086 void ui32_compute_hash(UI32_ARRAY *array);
00088 int ui32_verify_hash(UI32_ARRAY *array);
00090 int ui32_fwrite(UI32_ARRAY *array, FILE *file);
00092 UI32_ARRAY *ui32_fread(FILE *file);
00094 int TEST_UI32_ARRAY(int verbose);
00096
00100 UI64_ARRAY *ui64_init(int capacity);
00102 void ui64_free(UI64_ARRAY **array);
00104 void ui64_resize_to(UI64_ARRAY *array, int new_capacity);
00106 void ui64_resize_to_fit(UI64_ARRAY *array);
00108 void ui64_push(UI64_ARRAY *array, uint64_t element);
00110 void ui64_pop(UI64_ARRAY *array);
00112 void ui64_compute_hash(UI64_ARRAY *array);
00114 int ui64_verify_hash(UI64_ARRAY *array);
00116 int ui64_fwrite(UI64_ARRAY *array, FILE *file);
```

```
UI64_ARRAY *ui64_fread(FILE *file);
int TEST_UI64_ARRAY(int verbose);

int TEST_GENERIC_INT_ARRAYS(int verbose);

#if __STDC_VERSION__ >= 201112L

#define int_array_free(arr) _Generic((arr), \
    UI16_ARRAY * *: ui16_free,              \
    UI32_ARRAY * *: ui32_free,              \
    UI64_ARRAY * *: ui64_free)(arr)

#define int_array_resize_to(arr, cap) _Generic((arr), \
    UI16_ARRAY *: ui16_resize_to,                     \
    UI32_ARRAY *: ui32_resize_to,                     \
    UI64_ARRAY *: ui64_resize_to)(arr, cap)

#define int_array_resize_to_fit(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_resize_to_fit,                \
    UI32_ARRAY *: ui32_resize_to_fit,                \
    UI64_ARRAY *: ui64_resize_to_fit)(arr)

#define int_array_push(arr, val) _Generic((arr), \
    UI16_ARRAY *: ui16_push,                     \
    UI32_ARRAY *: ui32_push,                     \
    UI64_ARRAY *: ui64_push)(arr, val)

#define int_array_pop(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_pop,                \
    UI32_ARRAY *: ui32_pop,                \
    UI64_ARRAY *: ui64_pop)(arr)

#define int_array_compute_hash(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_compute_hash,                \
    UI32_ARRAY *: ui32_compute_hash,                \
    UI64_ARRAY *: ui64_compute_hash)(arr)

#define int_array_verify_hash(arr) _Generic((arr), \
    UI16_ARRAY *: ui16_verify_hash,                \
    UI32_ARRAY *: ui32_verify_hash,                \
    UI64_ARRAY *: ui64_verify_hash)(arr)

#define int_array_fwrite(arr, file) _Generic((arr), \
    UI16_ARRAY *: ui16_fwrite,                      \
    UI32_ARRAY *: ui32_fwrite,                      \
    UI64_ARRAY *: ui64_fwrite)(arr, file)

#endif // __STDC_VERSION__ >= 201112L


#endif // INT_ARRAYS_H
```

8.5 include/iZ_api.h File Reference

Public API for prime sieving, range scans, and prime generation.

#include <utils.h>
#include <iZ_toolkit.h>

Data Structures

- struct INPUT_SIEVE_RANGE

  Input parameters for range sieving/counting.

Macros

- #define N_LIMIT (1000000000000ULL)

Functions

Classic Prime Sieve Algorithms

Baseline sieves up to a numeric limit.

All classic sieve functions take a limit n and return an ascending list of primes <= n.

- UI64_ARRAY * SoE (uint64_t n)
  Optimized Sieve of Eratosthenes.
- UI64_ARRAY * SSoE (uint64_t n)
  Segmented Sieve of Eratosthenes.
- UI64_ARRAY * SoEu (uint64_t n)
  Euler (linear) sieve.
- UI64_ARRAY * SoS (uint64_t n)
  Sieve of Sundaram.
- UI64_ARRAY * SoA (uint64_t n)
  Sieve of Atkin.

iZ-based Sieve Algorithms

Sieve family operating in the 6x-1 / 6x+1 index space.

- UI64_ARRAY * SiZ (uint64_t n)
  Solid Sieve-iZ (wheel 6, iZ index space).
- UI64_ARRAY * SiZm (uint64_t n)
  Segmented Sieve-iZm (VX segmented, horizontal processing).
- UI64_ARRAY * SiZm_vy (uint64_t n)
  Segmented Sieve-iZm (vertical processing; faster, unordered output).

SiZ Range Variants

Count/stream primes over a numeric interval.

- uint64_t SiZ_stream (INPUT_SIEVE_RANGE *range)
  Stream primes in a range to filepath (and return the count).
- uint64_t SiZ_count (INPUT_SIEVE_RANGE *input_range, int cores_num)
  Count primes in a range using multiple worker processes.

Prime Generators

Probabilistic prime searches using iZm/VX machinery.

- int vy_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using vertical (vy) sieving.
- int vx_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using horizontal (vx) sieving.
- int iZ_next_prime (mpz_t p, mpz_t base, int forward)
  Advance to the next (or previous) prime from a base value.

8.5.1 Detailed Description

Public API for prime sieving, range scans, and prime generation.

This header provides the stable, high-level entry points for the iZ library. Most functions return a heap-allocated UI64_ARRAY* that must be released with ui64_free().

Definition in file iZ_api.h.

8.6 iZ_api.h

Go to the documentation of this file.

```
#ifndef IZ_API_H
#define IZ_API_H

#include <utils.h>
#include <iZ_toolkit.h>

#define N_LIMIT (1000000000000ULL)

UI64_ARRAY *SoE(uint64_t n);

UI64_ARRAY *SSoE(uint64_t n);

UI64_ARRAY *SoEu(uint64_t n);

UI64_ARRAY *SoS(uint64_t n);

UI64_ARRAY *SoA(uint64_t n);


UI64_ARRAY *SiZ(uint64_t n);

UI64_ARRAY *SiZm(uint64_t n);

UI64_ARRAY *SiZm_vy(uint64_t n);


typedef struct
{
    char *start;
    uint64_t range;
    int mr_rounds;
    char *filepath;
} INPUT_SIEVE_RANGE;

uint64_t SiZ_stream(INPUT_SIEVE_RANGE *range);

uint64_t SiZ_count(INPUT_SIEVE_RANGE *input_range, int cores_num);


int vy_random_prime(mpz_t p, int bit_size, int cores_num);

int vx_random_prime(mpz_t p, int bit_size, int cores_num);

int iZ_next_prime(mpz_t p, mpz_t base, int forward);


#endif // IZ_API_H
```

8.7 include/iZ_toolkit.h File Reference

Core iZ/iZm primitives used by SiZ-family algorithms.

#include <utils.h>
#include <int_arrays.h>
#include <bitmap.h>

Data Structures

- struct IZM

  Precomputed iZm assets for repeated VX-segment sieving.
- struct VX_SEG

  Runtime state for one VX segment at a specific y.

Macros

- #define MR_ROUNDS 25

Standard VX Sizes (primorial products excluding 2,3)

- #define VX2 (5 ∗ 7ULL)
- #define VX3 (VX2 ∗ 11ULL)
- #define VX4 (VX3 ∗ 13ULL)
- #define VX5 (VX4 ∗ 17ULL)
- #define VX6 (VX5 ∗ 19ULL)
- #define VX7 (VX6 ∗ 23ULL)
- #define VX8 (VX7 ∗ 29ULL)

Functions

- void iZm_construct_vx_base (uint64_t vx, BITMAP ∗base_x5, BITMAP ∗base_x7)
  Build pre-sieved base bitmaps for a VX segment.

iZ Mapping Helpers

- uint64_t iZ (uint64_t x, int i)
  Map iZ coordinates to an integer: 6∗x + i.
- void iZ_mpz (mpz_t z, mpz_t x, int i)
  GMP variant of iZ().
- void process_iZ_bitmaps (UI64_ARRAY ∗primes, BITMAP ∗x5, BITMAP ∗x7, uint64_t x↩_limit)
  Traverse iZ bitmaps, emit surviving primes, and mark composites.
- void get_root_primes (UI64_ARRAY ∗primes, uint64_t limit)
  Generate primes up to limit for deterministic sieving.

IZM Lifecycle

- IZM ∗ iZm_init (size_t vx)
  Allocate and initialize an IZM object for a given VX.
- IZM ∗ iZm_clone (IZM ∗src)
  Deep-copy an IZM object for per-worker ownership.
- void iZm_free (IZM ∗∗iZm)
  Release an IZM object and set the caller pointer to NULL.

VX Selection Helpers

- uint64_t compute_vx_k (uint64_t n, int max_k)
  Compute VX by multiplying up to max_k small primes.
- uint64_t compute_l2_vx (uint64_t n)
  Choose VX using an L2-cache-aware heuristic.
- void compute_max_vx (mpz_t vx, int bit_size)
  Compute largest primorial-like VX below 2^bit_size.

Modular Hit Solvers

- uint64_t iZm_solve_for_xp (int m_id, uint64_t p, uint64_t vx, uint64_t y)
  Solve first x-hit of prime p for line m_id in segment y.
- uint64_t iZm_solve_for_xp_mpz (int m_id, uint64_t p, uint64_t vx, mpz_t y)
  GMP variant of iZm_solve_for_xp() for very large y.

- int64_t iZm_solve_for_yp (int m_id, uint64_t p, uint64_t vx, uint64_t x)
  Solve first y-hit for fixed x in vertical (vy) scanning.

VX Segment Lifecycle and Execution

- VX_SEG * vx_init (IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds)
  Initialize and deterministically sieve one VX segment.
- void vx_free (VX_SEG **vx_obj)
  Free a VX segment and all owned resources.
- void vx_collect_p_gaps (VX_SEG *vx_obj)
  Extract prime-gap encoding from a fully sieved segment.
- void vx_full_sieve (VX_SEG *vx_obj, int collect_p_gaps)
  Complete segment processing (probabilistic stage and optional gaps).
- void vx_stream_file (VX_SEG *vx_obj, FILE *output)
  Stream segment primes to a file.

Random Prime Search Routines

- int vx_search_prime (mpz_t p, int m_id, int vx, int bit_size)
  Horizontal iZm/VX random-prime search.
- int vy_search_prime (mpz_t p, int m_id, mpz_t vx)
  Vertical iZm/VY random-prime search.

Toolkit Tests

- int TEST_IZM (int verbose)
  Run IZM construction and solver tests.
- int TEST_VX_SEG (int verbose)
  Run VX segment tests.

8.7.1 Detailed Description

Core iZ/iZm primitives used by SiZ-family algorithms.

This layer exposes index-space mapping helpers, VX sizing utilities, pre-sieved iZm base construction, per-segment sieve objects, and internal prime-search routines used by high-level APIs in iZ_api.h.

Definition in file iZ_toolkit.h.

8.8 iZ_toolkit.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef IZ_TOOLKIT_H
00011 #define IZ_TOOLKIT_H
00012
00013 #include <utils.h>      // Common utilities, types, and dependencies.
00014 #include <int_arrays.h> // Integer array containers.
00015 #include <bitmap.h>     // Packed bit-array utilities.
00016
00020
00029 uint64_t iZ(uint64_t x, int i);
00030
00037 void iZ_mpz(mpz_t z, mpz_t x, int i);
00038
00046 void process_iZ_bitmaps(UI64_ARRAY *primes, BITMAP *x5, BITMAP *x7, uint64_t x_limit);
```

```
00047
00053 void get_root_primes(UI64_ARRAY *primes, uint64_t limit);
00055
00058 #define VX2 (5 * 7ULL)
00059 #define VX3 (VX2 * 11ULL)
00060 #define VX4 (VX3 * 13ULL)
00061 #define VX5 (VX4 * 17ULL)
00062 #define VX6 (VX5 * 19ULL)
00063 #define VX7 (VX6 * 23ULL)
00064 #define VX8 (VX7 * 29ULL)
00066
00068 #define MR_ROUNDS 25
00069
00073 typedef struct
00074 {
00075     int vx;
00076     int k_vx;
00077     BITMAP *base_x5;
00078     BITMAP *base_x7;
00079     UI64_ARRAY *root_primes;
00080 } IZM;
00081
00089 IZM *iZm_init(size_t vx);
00090
00096 IZM *iZm_clone(IZM *src);
00097
00102 void iZm_free(IZM **iZm);
00104
00113 uint64_t compute_vx_k(uint64_t n, int max_k);
00114
00120 uint64_t compute_l2_vx(uint64_t n);
00121
00127 void compute_max_vx(mpz_t vx, int bit_size);
00129
00136 void iZm_construct_vx_base(uint64_t vx, BITMAP *base_x5, BITMAP *base_x7);
00137
00148 uint64_t iZm_solve_for_xp(int m_id, uint64_t p, uint64_t vx, uint64_t y);
00149
00158 uint64_t iZm_solve_for_xp_mpz(int m_id, uint64_t p, uint64_t vx, mpz_t y);
00159
00168 int64_t iZm_solve_for_yp(int m_id, uint64_t p, uint64_t vx, uint64_t x);
00170
00174 typedef struct
00175 {
00176     int vx;
00177     mpz_t y;
00178     mpz_t yvx;
00179     mpz_t root_limit;
00180     int is_large_limit;
00181     int mr_rounds;
00182     int start_x;
00183     int end_x;
00184     BITMAP *x5;
00185     BITMAP *x7;
00186     int p_count;
00187     UI16_ARRAY *p_gaps;
00188     int bit_ops;
00189     int p_test_ops;
00190 } VX_SEG;
00191
00203 VX_SEG *vx_init(IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds);
00204
00209 void vx_free(VX_SEG **vx_obj);
00210
00215 void vx_collect_p_gaps(VX_SEG *vx_obj);
00216
00222 void vx_full_sieve(VX_SEG *vx_obj, int collect_p_gaps);
00223
00229 void vx_stream_file(VX_SEG *vx_obj, FILE *output);
00231
00242 int vx_search_prime(mpz_t p, int m_id, int vx, int bit_size);
00243
00251 int vy_search_prime(mpz_t p, int m_id, mpz_t vx);
00253
00257 int TEST_IZM(int verbose);
00259 int TEST_VX_SEG(int verbose);
00261
00263
00264 #endif // IZ_TOOLKIT_H
```

8.9 include/logger.h File Reference

Header file for the logging system.

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

Macros

- #define LOGGER_FORMAT_PRINTF(fmt_idx, arg_idx)

  Compiler attribute wrapper for printf-style format checking.
- #define LOG_DIR "logs/"

  Directory where logs are stored.
- #define LOG_FILE LOG_DIR "log.txt"

  Default log file.
- #define LOG_MAX_SIZE 1024 ∗ 1024 ∗ 5

  Maximum log file size (5 MB).

Enumerations

- enum LogLevel {
  LOG_DEBUG , LOG_INFO , LOG_WARNING , LOG_ERROR ,
  LOG_FATAL }

  Enumeration of log levels.

Functions

- const char ∗ log_level_to_string (LogLevel level)

  Returns a string representation of the log level.
- void log_init (const char ∗log_file)

  Initializes the logging system.
- void log_shutdown (void)

  Shuts down the logging system and cleans up resources.
- void log_set_log_level (LogLevel level)

  Sets the current log level. Messages below this level will not be logged.
- void log_message (LogLevel level, const char ∗format,...) LOGGER_FORMAT_PRINTF(2

  Logs a formatted message at the given log level.
- void void log_message_extended (LogLevel level, const char ∗file_name, int line_number, const char ∗format,...) LOGGER_FORMAT_PRINTF(4

  Logs a formatted message with extended information (file name, line number).
- void void void log_console (const char ∗format,...) LOGGER_FORMAT_PRINTF(1

  Logs a timestamped message to the console only, without requiring a log level.
- void void void void log_test (int result, int test_id, const char ∗unit_name, const char ∗format,...) LOGGER_FORMAT_PRINTF(4

  Convenience function for logging test results.
- void void void void void log_test_summary (char ∗module_name, int passed, int failed, int verbose)

  Logs a summary of test results for a module.
- void log_debug (const char ∗format,...) LOGGER_FORMAT_PRINTF(1

  Logs a debug message.
- void void log_info (const char ∗format,...) LOGGER_FORMAT_PRINTF(1

  Logs an info message.
- void void void log_warn (const char ∗format,...) LOGGER_FORMAT_PRINTF(1

  Logs a warning message.
- void void void void log_error (const char ∗format,...) LOGGER_FORMAT_PRINTF(1

  Logs an error message.
- void void void void void log_fatal (const char ∗format,...) LOGGER_FORMAT_PRINTF(1

  Logs a fatal message.

8.9.1 Detailed Description

Header file for the logging system.

This file contains function declarations and macros for a logging system that provides various logging levels (DEBUG, INFO, WARNING, ERROR, FATAL). The logging system supports thread-safe logging, log rotation, and console output. It also includes convenience functions for logging messages with extended information (file name, line number).

Note

> This module is part of a larger project that includes various components requiring logging functionality. It is designed to be flexible and easy to use, allowing developers to log messages at different levels and with different formats.

API:

- log_init: Initializes the logging system.
- log_shutdown: Shuts down the logging system.
- log_set_log_level: Sets the current log level.
- log_message: Logs a formatted message at the given log level.
- log_message_extended: Logs a formatted message with extended information.
- log_console: Logs a formatted message to the console.
- log_debug: Logs a debug message.
- log_info: Logs an info message.
- log_warn: Logs a warning message.
- log_error: Logs an error message.
- log_fatal: Logs a fatal message.

Definition in file logger.h.

8.10 logger.h

Go to the documentation of this file.

```
00031
00032 #ifndef LOGGER_H
00033 #define LOGGER_H
00034
00035 #include <stdio.h>
00036 #include <stdlib.h>
00037 #include <string.h>
00038
00042
00047 #if defined(__GNUC__) || defined(__clang__)
00048 #define LOGGER_FORMAT_PRINTF(fmt_idx, arg_idx) __attribute__((format(printf, fmt_idx, arg_idx)))
00049 #else
00050 #define LOGGER_FORMAT_PRINTF(fmt_idx, arg_idx)
00051 #endif
00052
00053 // Log directory and file configuration
00054 #define LOG_DIR "logs/"
00055 #define LOG_FILE LOG_DIR "log.txt"
```

```
#define LOG_MAX_SIZE 1024 * 1024 * 5

typedef enum
{
    LOG_DEBUG,
    LOG_INFO,
    LOG_WARNING,
    LOG_ERROR,
    LOG_FATAL
} LogLevel;

// Function declarations

const char *log_level_to_string(LogLevel level);

void log_init(const char *log_file);

void log_shutdown(void);

void log_set_log_level(LogLevel level);

void log_message(LogLevel level, const char *format, ...) LOGGER_FORMAT_PRINTF(2, 3);

void log_message_extended(LogLevel level, const char *file_name, int line_number, const char *format, ...)
    LOGGER_FORMAT_PRINTF(4, 5);

void log_console(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);

void log_test(int result, int test_id, const char *unit_name, const char *format, ...) LOGGER_FORMAT_PRINTF(4, 5);

void log_test_summary(char *module_name, int passed, int failed, int verbose);

// Convenience logging functions for different log levels
void log_debug(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);

void log_info(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);

void log_warn(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);

void log_error(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);

void log_fatal(const char *format, ...) LOGGER_FORMAT_PRINTF(1, 2);


#endif // LOGGER_H
```

8.11 include/test_api.h File Reference

Public test/benchmark entry points for the iZ library.

#include <iZ_api.h>

Data Structures

- struct SIEVE_LIMIT
- struct SIEVE_MODEL

Typedefs

- typedef UI64_ARRAY *(* SIEVE_FN) (uint64_t)

Functions

Sieve model tests/benchmarks

- int TEST_SIEVE_MODELS_INTEGRITY (int verbose)
  Run correctness checks across all registered sieve models.
- void BENCHMARK_SIEVE_MODELS (int save_results)
  Benchmark registered sieve models and optionally persist results.

Range sieve tests/benchmarks

- int TEST_SiZ_stream (int verbose)
  Validate SiZ_stream behavior and output formatting.
- int TEST_SiZ_count (int verbose)
  Validate SiZ_count correctness across worker counts.

Prime generation tests/benchmarks

- int TEST_iZ_next_prime (int verbose)
  Validate iZ_next_prime for forward/backward traversal cases.
- int TEST_vy_random_prime (int verbose)
  Validate vy_random_prime for primality and bit-length constraints.
- int TEST_vx_random_prime (int verbose)
  Validate vx_random_prime for primality and bit-length constraints.
- int BENCHMARK_P_GEN_ALGORITHMS (int bit_size, int test_rounds, int save_results)
  Benchmark random-prime generation routines over repeated trials.

8.11.1 Detailed Description

Public test/benchmark entry points for the iZ library.

Doxygen can include this header when documenting the project to expose the test harness API and benchmark entry points.

Definition in file test_api.h.

8.12 test_api.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef TESTS_H
00010 #define TESTS_H
00011
00016
00017 #include <iZ_api.h>
00018
00020 typedef UI64_ARRAY *(*SIEVE_FN)(uint64_t);
00021
00023 typedef struct
00024 {
00025     int base;
00026     int exp;
00027 } SIEVE_LIMIT;
00028
00030 typedef struct
00031 {
00032     SIEVE_FN function;
00033     const char name[32];
00034 } SIEVE_MODEL;
```

```
00035
00038
00039 int TEST_SIEVE_MODELS_INTEGRITY(int verbose);
00041 void BENCHMARK_SIEVE_MODELS(int save_results);
00043
00046
00047 int TEST_SiZ_stream(int verbose);
00049 int TEST_SiZ_count(int verbose);
00051
00054
00055 int TEST_iZ_next_prime(int verbose);
00057 int TEST_vy_random_prime(int verbose);
00059 int TEST_vx_random_prime(int verbose);
00060
00062 int BENCHMARK_P_GEN_ALGORITHMS(int bit_size, int test_rounds, int save_results);
00064
00066
00067 #endif // TESTS_H
```

8.13 include/utils.h File Reference

Shared utilities and common includes for the iZ library.

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stddef.h>
#include <string.h>
#include <assert.h>
#include <math.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <signal.h>
#include <unistd.h>
#include <time.h>
#include <sys/time.h>
#include <sys/wait.h>
#include <gmp.h>
#include <openssl/sha.h>
#include <logger.h>

Macros

- #define DIR_output "./output"
- #define MAX_CORES get_cpu_cores_count()

Functions

- int create_dir (const char *dir)

  Create a directory if it does not exist.
- int is_numeric_str (const char *str)

  Return non-zero if str contains only ASCII digits.
- uint64_t gcd (uint64_t a, uint64_t b)

  Compute the greatest common divisor of a and b.
- uint64_t modular_inverse (uint64_t a, uint64_t m)

  Compute the modular inverse of a modulo m.
- void print_line (int length, char fill_char)

Print a repeated character line to stdout.

- void print_centered_text (const char *text, int line_length, char fill_char)

  Print text centered within a filled line.
- void print_sha256_hash (const unsigned char *hash)

  Print a SHA-256 digest as hex to stdout.
- void gmp_seed_randstate (gmp_randstate_t state)

  Seed a GMP random state.
- int get_cpu_cores_count (void)

  Get the number of online CPU cores (>= 1).
- int get_cpu_L2_cache_size_bits (void)

  Return the CPU L2 cache size in bits (best effort).
- void print_module_header (char *module_name)

  Print a test-suite header banner for a module.
- void print_test_header (void)

  Print the generic test runner header.

8.13.1 Detailed Description

Shared utilities and common includes for the iZ library.

This header centralizes cross-cutting helpers (string checks, arithmetic, formatting, GMP helpers, and system queries).

Definition in file utils.h.

8.14 utils.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef UTILS_H
00010 #define UTILS_H
00011
00012 // necessary includes across files
00013
00014 #include <stdio.h>    // For printf, FILE, fopen, fwrite, fread, etc.
00015 #include <stdlib.h>   // For malloc, free, etc.
00016 #include <stdint.h>   // For fixed-width integer types like uint64_t
00017 #include <stddef.h>   // For size_t
00018 #include <string.h>   // For string manipulation functions like snprintf
00019 #include <assert.h>   // For assertions
00020 #include <math.h>     // For math functions like sqrt
00021 #include <sys/stat.h> // For creating directories (mkdir)
00022 #include <fcntl.h>    // For open
00023 #include <signal.h>   // For kill
00024 #include <unistd.h>   // For close
00025 #include <time.h>     // For time
00026 #include <sys/time.h> // For gettimeofday
00027 #include <sys/wait.h> // For waitpid
00028
00029 // Platform-specific includes for sysctl (macOS/BSD)
00030 #if defined(__APPLE__) || defined(__FreeBSD__) || defined(__OpenBSD__) || defined(__NetBSD__)
00031 #include <sys/types.h>
00032 #include <sys/sysctl.h>
00033 #endif
00034
00035 #include <gmp.h>         // GMP library for arbitrary precision arithmetic
00036 #include <openssl/sha.h> // For SHA-256 hashing
00037
00038 #include <logger.h> // Logger module for logging timestamped messages.
00039
00044
00046 #define DIR_output "./output"
00048 #define MAX_CORES get_cpu_cores_count()
```

```
// file utilities
int create_dir(const char *dir);

// string utilities
int is_numeric_str(const char *str);

// math utilities
uint64_t gcd(uint64_t a, uint64_t b);
uint64_t modular_inverse(uint64_t a, uint64_t m);

// print utilities
void print_line(int length, char fill_char);
void print_centered_text(const char *text, int line_length, char fill_char);
void print_sha256_hash(const unsigned char *hash);

// gmp utilities
void gmp_seed_randstate(gmp_randstate_t state);

// system utilities
int get_cpu_cores_count(void);
int get_cpu_L2_cache_size_bits(void);

// test utils
void print_module_header(char *module_name);
void print_test_header(void);


#endif // UTILS_H
```

8.15 src/iZ_apps.c File Reference

High-level application routines built on top of the iZ API.

#include <iZ_api.h>

Functions

SiZ Range Variants

Count/stream primes over a numeric interval.

- uint64_t SiZ_stream (INPUT_SIEVE_RANGE *range)
  Stream primes in a range to filepath (and return the count).
- uint64_t SiZ_count (INPUT_SIEVE_RANGE *input_range, int cores_num)
  Count primes in a range using multiple worker processes.

Prime Generators

Probabilistic prime searches using iZm/VX machinery.

- int vy_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using vertical (vy) sieving.
- int vx_random_prime (mpz_t p, int bit_size, int cores_num)
  Search for a random prime using horizontal (vx) sieving.
- int iZ_next_prime (mpz_t p, mpz_t base, int forward)
  Advance to the next (or previous) prime from a base value.

8.15.1 Detailed Description

High-level application routines built on top of the iZ API.

This module contains range-oriented entry points (stream/count) and probabilistic prime generation wrappers.

Definition in file iZ_apps.c.

8.16 iZ_apps.c

Go to the documentation of this file.

```
#include <iZ_api.h>

// ============================================================
// * SiZ Range Variants
// ============================================================

uint64_t SiZ_stream(INPUT_SIEVE_RANGE *input_range)
{
    assert(input_range && input_range->start && "Invalid INPUT_SIEVE_RANGE passed to SiZ_stream.");
    assert(input_range->filepath && "Output filepath is NULL in SiZ_stream.");
    input_range->range = MAX(input_range->range, 100); // enforce minimum range of 100

    FILE *output = fopen(input_range->filepath, "w");
    if (output == NULL)
    {
        log_error("Failed to open output file: %s", input_range->filepath);
        return 0;
    }

    uint64_t total = 0; // output: total prime count

    int vx = VX6; // Use VX6 segment size (1,616,615) for optimal results
    // Miller-Rabin rounds, bounded [5, 50]
    int mr_rounds = MIN(MAX(input_range->mr_rounds, 5), 50);

    // Parse numeric bounds into mpz
    mpz_t Zs, Ze, Xs, Xe, Ys, Ye;
    mpz_init(Zs);
    mpz_init(Ze);
    mpz_init(Xs);
    mpz_init(Xe);
    mpz_init(Ys);
    mpz_init(Ye);

    // read start value Zs
    if (mpz_set_str(Zs, input_range->start, 10) != 0)
    {
        log_error("Invalid numeric string for start.");
        mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
        if (output)
            fclose(output);
        return 0;
    }

    // Ze = Zs + range
    mpz_add_ui(Ze, Zs, input_range->range);

    // Convert bounds to iZ index space:
    mpz_fdiv_q_ui(Xs, Zs, 6); // Xs = Zs/6
    mpz_fdiv_q_ui(Xe, Ze, 6); // Xe = Ze/6

    // Ys = Xs / vx, Ye = Xe / vx
    mpz_fdiv_q_ui(Ys, Xs, vx);
    mpz_fdiv_q_ui(Ye, Xe, vx);

    // start_x = (Xs % vx), end_x = (Xe % vx)
    int start_x = mpz_fdiv_ui(Xs, vx);
    int end_x = mpz_fdiv_ui(Xe, vx);

    // Map modulo 0 to vx to stay in the inclusive x-index range [1..vx]
    if (start_x == 0)
        start_x = 1;
    if (end_x == 0)
        end_x = vx;

    // if Ys = 0, use SiZm for the first segment
    if (mpz_cmp_ui(Ys, 0) == 0)
    {
        uint64_t limit = mpz_cmp_ui(Ye, 0) > 0 ? vx : end_x;
        UI64_ARRAY *primes = SiZm(limit * 6 + 1);
        uint64_t s = mpz_get_ui(Zs);

        for (int i = 0; i < primes->count; i++)
        {
            // only primes > Zs
            if (primes->array[i] > s)
            {
                total++;
                // output prime
                if (output)
```

```
                fprintf(output, "%llu ", primes->array[i]);
            }
        }
        start_x = 1;            // next segment starts at x=1
        mpz_add_ui(Ys, Ys, 1); // increment Ys for the next segment

        ui64_free(&primes);
    }

    // If Ye = 0, we are done
    if (mpz_cmp_ui(Ye, 0) == 0)
    {
        // cleanup
        mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
        if (output)
            fclose(output);
        return total;
    }

    // Initialize iZm structure for vx segments
    IZM *iZm = iZm_init(vx);
    if (!iZm)
    {
        // check logs for errors
        mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
        return 0;
    }

    // Process remaining segments for y in (Ys:Ye)
    int y_range = mpz_get_ui(Ye) - mpz_get_ui(Ys);
    mpz_t prime_z;
    mpz_init(prime_z);

    for (int i = 0; i <= y_range; i++)
    {
        int seg_start_x = (i == 0) ? start_x : 1;
        int seg_end_x = (i == y_range) ? end_x : vx;
        VX_SEG *vx_obj = vx_init(iZm, seg_start_x, seg_end_x, mpz_get_str(NULL, 10, Ys), mr_rounds);
        if (!vx_obj)
        {
            // check logs for errors
            iZm_free(&iZm);
            mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, prime_z, NULL);
            if (output)
                fclose(output);
            return 0;
        }

        if (output)
            vx_stream_file(vx_obj, output);
        else
            vx_full_sieve(vx_obj, 0);
        total += vx_obj->p_count; // accumulate prime count

        vx_free(&vx_obj);
        mpz_add_ui(Ys, Ys, 1); // increment Ys for the next segment
    }

    // Cleanup
    iZm_free(&iZm);
    mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, prime_z, NULL);

    if (output)
        fclose(output);

    return total;
}

uint64_t SiZ_count(INPUT_SIEVE_RANGE *input_range, int cores_num)
{
    assert(input_range && input_range->start && input_range->range > 100 &&
           "Invalid INPUT_SIEVE_RANGE passed to SiZ_count.");

    uint64_t total = 0;
    int vx = VX6;           // use VX6 segment size for optimal results
    int vx_interval = 6 * vx; // covers a range of 9,699,690
    cores_num = MIN(cores_num, get_cpu_cores_count());

    uint64_t range = input_range->range;
    int total_segments = (range / vx_interval) + 1;

    // Parse numeric bounds into mpz
    mpz_t Zs, Ze, Xs, Xe, Ys, Ye;
    mpz_init(Zs);
    mpz_init(Ze);
    mpz_init(Xs);
    mpz_init(Xe);
```

```
    mpz_init(Ys);
    mpz_init(Ye);

    if (mpz_set_str(Zs, input_range->start, 10) != 0)
    {
        log_error("Invalid numeric string for start.");
        mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
        return 0;
    }

    // Convert bounds to iZ index space:
    // Ze = Zs + range - 1
    mpz_add_ui(Ze, Zs, input_range->range - 1);
    mpz_fdiv_q_ui(Xs, Zs, 6); // Xs = Zs/6
    // if Zs % 6 > 1, increment Xs
    if (mpz_fdiv_ui(Zs, 6) > 1)
        mpz_add_ui(Xs, Xs, 1);

    mpz_fdiv_q_ui(Xe, Ze, 6); // Xe = Ze/6

    // Ys = Xs / vx, Ye = Xe / vx
    mpz_fdiv_q_ui(Ys, Xs, vx);
    mpz_fdiv_q_ui(Ye, Xe, vx);

    // start_x = (Xs % vx), end_x = (Xe % vx)
    int start_x = mpz_fdiv_ui(Xs, vx);
    int end_x = mpz_fdiv_ui(Xe, vx);

    // if Ys = 0, use sieve_iZm for the first segment
    if (mpz_cmp_ui(Ys, 0) == 0)
    {
        uint64_t limit = mpz_cmp_ui(Ye, 0) > 0 ? vx : end_x;
        UI64_ARRAY *primes = SiZm(limit * 6 + 1);
        total += primes->count;
        uint64_t s = mpz_get_ui(Zs);

        for (int i = 0; i < primes->count; i++)
        {
            // decrement primes < Zs
            if (primes->array[i] < s)
            {
                total--;
                continue;
            }
        }

        ui64_free(&primes);
        mpz_add_ui(Ys, Ys, 1); // increment Ys for the next segment
        total_segments--;
    }

    IZM *iZm = iZm_init(vx);
    if (!iZm)
    {
        // check logs for errors
        mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
        return 0;
    }

    // Handle edge cases:
    mpz_t prime_z;
    mpz_init(prime_z);
    // if Ys > 0 and Zs % 6 <= 1
    if (mpz_cmp_ui(Ys, 0) > 0 && mpz_fdiv_ui(Zs, 6) <= 1)
    {
        // if iZ(Xs, -1) < Zs and prime, decrement total
        iZ_mpz(prime_z, Xs, -1);
        if (mpz_cmp(prime_z, Zs) < 0)
        {
            if (mpz_probab_prime_p(prime_z, 25))
            {
                total--;
            }
        }
    }
    // if Ye > 0 and Ze % 6 <= 1
    if (mpz_cmp_ui(Ye, 0) > 0 && mpz_fdiv_ui(Ze, 6) <= 1)
    {
        // if iZ(Xe, 1) > Ze and prime, decrement total
        iZ_mpz(prime_z, Xe, 1);
        if (mpz_cmp(prime_z, Ze) > 0)
        {
            if (mpz_probab_prime_p(prime_z, 25))
            {
                total--;
            }
        }
```

```
    }
    mpz_clear(prime_z);

    // Single-process processing of all segments
    if (cores_num == 1)
    {
        for (int i = 0; i < total_segments; i++)
        {
            int seg_start_x = (i == 0) ? start_x : 1;
            int seg_end_x = (i == total_segments - 1) ? end_x : vx;
            VX_SEG *vx_obj = vx_init(iZm, seg_start_x, seg_end_x, mpz_get_str(NULL, 10, Ys),
input_range->mr_rounds);
            if (!vx_obj)
            {
                mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
                iZm_free(&iZm);
                return 0;
            }

            vx_full_sieve(vx_obj, 0);
            total += vx_obj->p_count;

            vx_free(&vx_obj);
            mpz_add_ui(Ys, Ys, 1);
        }
        mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
        iZm_free(&iZm);

        return total;
    }

    // Multi-process processing of remaining segments
    // if total_segments < cores_num, adjust cores_num
    if (total_segments < cores_num)
    {
        cores_num = total_segments;
    }

    int segments_per_core = total_segments / cores_num;
    int remainder_segments = total_segments % cores_num;

    // uint64_t root_limit = mpz_cmp_ui(Ze, INT64_MAX) < 0 ? sqrt(mpz_get_ui(Ze)) + 1 : INT32_MAX;

    int pipe_fds[cores_num][2];
    pid_t pids[cores_num];
    for (int i = 0; i < cores_num; i++)
    {
        pids[i] = -1;
        pipe_fds[i][0] = -1;
        pipe_fds[i][1] = -1;
    }

    for (int core = 0; core < cores_num; core++)
    {
        if (pipe(pipe_fds[core]) == -1)
        {
            log_error("SiZ_count: Failed to create pipe for core %d. Aborting.", core);
            // cleanup and abort on any pipe error
            for (int j = 0; j <= core; j++)
            {
                if (pipe_fds[j][0] != -1)
                    close(pipe_fds[j][0]);
                if (pipe_fds[j][1] != -1)
                    close(pipe_fds[j][1]);
            }
            mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
            iZm_free(&iZm);
            return 0;
        }

        pids[core] = fork();
        if (pids[core] < 0)
        {
            log_error("SiZ_count: Failed to fork process for core %d. Aborting.", core);
            // close this core's pipe and any previous ones, then exit
            for (int j = 0; j <= core; j++)
            {
                if (pipe_fds[j][0] != -1)
                    close(pipe_fds[j][0]);
                if (pipe_fds[j][1] != -1)
                    close(pipe_fds[j][1]);
            }
            mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
            iZm_free(&iZm);
            return 0;
        }

```

```
        if (pids[core] == 0)
        {
            // Child process
            // Close unrelated pipes
            for (int j = 0; j < cores_num; j++)
            {
                if (j != core && pipe_fds[j][0] != -1)
                    close(pipe_fds[j][0]);
                if (j != core && pipe_fds[j][1] != -1)
                    close(pipe_fds[j][1]);
            }
            close(pipe_fds[core][0]); // close read end

            // Compute this child's starting Y and number of segments
            int offset = core * segments_per_core + (core < remainder_segments ? core : remainder_segments);
            int local_segments = segments_per_core + (core < remainder_segments ? 1 : 0);

            uint64_t child_total = 0;

            if (local_segments > 0)
            {
                mpz_t local_Ys;
                mpz_init(local_Ys);
                mpz_set(local_Ys, Ys);
                mpz_add_ui(local_Ys, local_Ys, offset);

                // Each child has its own IZM to avoid data races
                IZM *iZm_local = iZm_clone(iZm);
                if (!iZm_local)
                {
                    mpz_clear(local_Ys);
                    close(pipe_fds[core][1]);
                    exit(1);
                }

                for (int i = 0; i < local_segments; i++)
                {
                    int seg_start_x = (core == 0 && i == 0) ? start_x : 1;
                    int seg_end_x = (core == cores_num - 1 && i == local_segments - 1) ? end_x : vx;
                    VX_SEG *vx_obj = vx_init(iZm_local, seg_start_x, seg_end_x, mpz_get_str(NULL, 10, local_Ys),
input_range->mr_rounds);
                    if (!vx_obj)
                    {
                        iZm_free(&iZm_local);
                        mpz_clear(local_Ys);
                        close(pipe_fds[core][1]);
                        exit(1);
                    }

                    vx_full_sieve(vx_obj, 0);
                    child_total += vx_obj->p_count;

                    vx_free(&vx_obj);
                    mpz_add_ui(local_Ys, local_Ys, 1);
                }

                iZm_free(&iZm_local);
                mpz_clear(local_Ys);
            }

            // Send result to parent
            ssize_t written = write(pipe_fds[core][1], &child_total, sizeof(child_total));
            if (written != sizeof(child_total))
            {
                log_error("SiZ_count: Child %d failed to write result.", core);
            }

            close(pipe_fds[core][1]);
            exit(0);
        }
        else
        {
            // Parent process
            close(pipe_fds[core][1]); // close write end
            pipe_fds[core][1] = -1;
        }
    }

    // Collect results
    for (int core = 0; core < cores_num; core++)
    {
        if (pids[core] > 0 && pipe_fds[core][0] != -1)
        {
            uint64_t child_total = 0;
            ssize_t bytes_read = read(pipe_fds[core][0], &child_total, sizeof(child_total));
            if (bytes_read == sizeof(child_total))
            {
```

```
            total += child_total;
        }
        else if (bytes_read == 0)
        {
            log_error("SiZ_count: Child %d closed pipe without sending result.", core);
        }
        else if (bytes_read < 0)
        {
            log_error("SiZ_count: Failed to read result from child %d.", core);
        }
        else
        {
            log_error("SiZ_count: Partial read from child %d (got %zd bytes).", core, bytes_read);
        }

        close(pipe_fds[core][0]);
        pipe_fds[core][0] = -1;

        int status;
        waitpid(pids[core], &status, 0);
        if (WIFEXITED(status) && WEXITSTATUS(status) != 0)
        {
            log_error("SiZ_count: Child %d exited with status %d.", core, WEXITSTATUS(status));
            // propagate child failure as an overall error
            mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
            iZm_free(&iZm);
            return 0;
        }
        else if (WIFSIGNALED(status))
        {
            log_error("SiZ_count: Child %d terminated by signal %d.", core, WTERMSIG(status));
            mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
            iZm_free(&iZm);
            return 0;
        }
    }
    else if (pipe_fds[core][0] != -1)
    {
        close(pipe_fds[core][0]);
        pipe_fds[core][0] = -1;
    }
}

// Cleanup
mpz_clears(Zs, Ze, Xs, Xe, Ys, Ye, NULL);
iZm_free(&iZm);

return total;
}

// ============================================================
// * Random Prime Generation
// ============================================================

int vy_random_prime(mpz_t p, int bit_size, int cores_num)
{
    int found = 0;              // flag to indicate if a prime was found
    bit_size = MAX(bit_size, 10); // Set minimum bit size

    // 1. Compute max vx for the given bit-size
    mpz_t vx;
    mpz_init(vx);
    compute_max_vx(vx, bit_size);

    // 2. If < 2 cores, run the search in-process
    if (cores_num < 2)
    {
        found = vy_search_prime(p, 0, vx);
        mpz_clear(vx);
        return found;
    }

    // 3. Else, fork multiple processes to search for a prime
    // Create a pipe for inter-process communication.
    int fd[2];
    if (pipe(fd) == -1)
    {
        log_error("Failed to create pipe in vy_random_prime. Falling back to in-process search.");
        found = vy_search_prime(p, 0, vx);
        mpz_clear(vx);
        return found;
    }

    pid_t pids[cores_num];
    int forks_created = 0;

    // Fork child processes.
```

```
for (int i = 0; i < cores_num; i++)
{
    pid_t pid = fork();
    if (pid < 0)
    {
        log_error("Failed to fork process %d in vy_random_prime", i);
        continue; // Skip this fork failure and continue creating other forks
    }
    else if (pid == 0)
    {
        // Child process: close the read-end.
        close(fd[0]);

        mpz_t local_p; // local prime candidate
        mpz_init(local_p);

        // Search for a candidate prime
        found = vy_search_prime(local_p, 0, vx);

        // If a candidate is found, send it via the pipe.
        if (found)
        {
            char *p_str = mpz_get_str(NULL, 10, local_p);
            if (p_str)
            {
                write(fd[1], p_str, strlen(p_str) + 1);
                free(p_str);
            }
        }
        mpz_clear(local_p);

        close(fd[1]); // Close the write-end.
        exit(0);
    }
    else
    {
        // Parent process saves child's PID.
        pids[i] = pid;
        forks_created++;
    }
}

// 4. Parent reads first result from the pipe.
close(fd[1]); // Close the write-end.

if (forks_created == 0)
{
    log_error("No child processes were created in vy_random_prime, falling back to in-process search");
    close(fd[0]);
    found = vy_search_prime(p, 0, vx);
    mpz_clear(vx);
    return found;
}

// Allocate a sufficiently large buffer to hold the prime string + null terminator
// bit_size * 0.302 is approx digits, add extra padding for safety
size_t buf_size = (size_t)(bit_size * 0.4);
char *buf = malloc(buf_size);
if (!buf)
{
    log_error("Failed to allocate buffer in vy_random_prime");
    found = 0;
}
else
{
    ssize_t n = read(fd[0], buf, buf_size - 1);
    if (n == -1)
    {
        log_error("Failed to read from pipe in vx_random_prime");
        found = 0;
    }
    else if (n > 0)
    {
        buf[n] = '\0'; // Ensure null-termination
        if (mpz_set_str(p, buf, 10) == 0)
        {
            found = 1;
        }
        else
        {
            log_error("Failed to set prime from buffer in vx_random_prime");
            found = 0;
        }
    }
    free(buf);
}
close(fd[0]); // Close the read-end of the pipe.
```

```
    // 5. Terminate all child processes
    for (int i = 0; i < forks_created; i++)
    {
        kill(pids[i], SIGTERM);    // Terminate child process
        waitpid(pids[i], NULL, 0); // Wait for child process to terminate
    }

    mpz_clear(vx);

    return found;
}

int vx_random_prime(mpz_t p, int bit_size, int cores_num)
{
    int found = 0;
    bit_size = MAX(bit_size, 10);
    int vx = bit_size <= 2048 ? VX5 : VX6;

    // 2. If < 2 cores, run the search in-process
    if (cores_num < 2)
    {
        found = vx_search_prime(p, 0, vx, bit_size);
        return found;
    }

    // 3. Multi-core: fork processes
    int fd[2];
    if (pipe(fd) == -1)
    {
        log_error("Failed to create pipe in vx_random_prime. Falling back to in-process search.");
        found = vx_search_prime(p, 0, vx, bit_size);
        return found;
    }

    pid_t pids[cores_num];
    int forks_created = 0;

    for (int i = 0; i < cores_num; i++)
    {
        pid_t pid = fork();
        if (pid < 0)
        {
            log_error("Failed to fork process %d in vx_random_prime", i);
            continue;
        }
        else if (pid == 0)
        {
            // CHILD PROCESS: Initialize everything fresh
            close(fd[0]);

            // Search for prime with child's own objects
            mpz_t local_p;
            mpz_init(local_p);
            int child_found = vx_search_prime(local_p, 0, vx, bit_size);

            if (child_found)
            {
                char *p_str = mpz_get_str(NULL, 10, local_p);
                if (p_str != NULL)
                {
                    write(fd[1], p_str, strlen(p_str) + 1);
                    free(p_str);
                }
            }

            // Cleanup child resources
            mpz_clear(local_p);
            close(fd[1]);
            exit(0);
        }
        else
        {
            pids[i] = pid;
            forks_created++;
        }
    }

    // 4. Parent process reads result
    close(fd[1]);

    if (forks_created == 0)
    {
        log_error("No child processes were created in vx_random_prime, falling back to in-process search");
        close(fd[0]);
        // fallback to in-process search
        found = vx_search_prime(p, 0, vx, bit_size);
```

```
        return found;
    }

    // Allocate a sufficiently large buffer to hold the prime string + null terminator
    size_t buf_size = (size_t)(bit_size * 0.4);
    char *buf = malloc(buf_size);
    if (!buf)
    {
        log_error("Failed to allocate buffer in vx_random_prime");
        found = 0;
    }
    else
    {
        ssize_t n = read(fd[0], buf, buf_size - 1);
        if (n > 0)
        {
            buf[n] = '\0'; // Ensure null-termination
            if (mpz_set_str(p, buf, 10) == 0)
            {
                found = 1;
            }
            else
            {
                log_error("Failed to set prime from buffer in vx_random_prime");
                found = 0;
            }
        }
        free(buf);
    }

    close(fd[0]); // Close the read-end of the pipe.

    // Terminate children
    for (int i = 0; i < forks_created; i++)
    {
        kill(pids[i], SIGTERM);
        waitpid(pids[i], NULL, 0);
    }

    return found;
}

int iZ_next_prime(mpz_t p, mpz_t base, int forward)
{
    // 1. Initialization
    int found = 0; // flag to indicate if a prime was found

    // tmp variable to hold the value until a prime is found
    mpz_t z;
    mpz_init_set(z, base); // set tmp = base

    // a. Edge cases:
    // if forward and base is iZ-, check next iZ+
    if (mpz_fdiv_ui(z, 6) == 5 && forward)
    {
        mpz_add_ui(z, z, 2); // increment tmp by 2
        if (mpz_probab_prime_p(z, MR_ROUNDS))
        {
            mpz_set(p, z); // set p = tmp + 2
            mpz_clear(z);
            return 1;
        }
    }
    // if backward and base is iZ+, check previous iZ-
    else if (mpz_fdiv_ui(z, 6) == 1 && !forward)
    {
        mpz_sub_ui(z, z, 2); // decrement tmp by 2
        if (mpz_probab_prime_p(z, MR_ROUNDS))
        {
            mpz_set(p, z); // set p = tmp - 2
            mpz_clear(z);
            return 1;
        }
    }

    int vx = (mpz_sizeinbase(base, 2) > 2048) ? VX6 : VX5;
    IZM *iZm = iZm_init(vx);
    if (!iZm)
    {
        log_error("iZm initialization failed in iZ_next_prime.");
        mpz_clear(z);
        return 0;
    }

    // c. Initialize and set y and yvx
    mpz_t y, yvx, x_p;
    mpz_init(y);
```

```
mpz_init(yvx);
mpz_init(x_p);

mpz_div_ui(y, base, 6 * vx); // compute y = base / 6 * vx
mpz_mul_ui(yvx, y, vx);      // compute yvx = y * vx
mpz_div_ui(x_p, z, 6);       // compute x_p = tmp / 6

// 2. Iterate over the x5 and x7 bitmaps to find a prime
// set start_x = x_p % vx +/- 1
int step = forward ? 1 : -1;
int start_x = mpz_fdiv_ui(x_p, vx) + step;
int end_x = forward ? vx : 1;

int i = 0; // segment counter

while (!found)
{
    if (forward)
    {
        if (i > 0)
            start_x = 1; // start from the beginning of the bitmap

        for (int x = start_x; x <= end_x; x++)
        {
            // check if x5[x] is set
            if (bitmap_get_bit(iZm->base_x5, x))
            {
                mpz_add_ui(x_p, yvx, x); // set x_p = yvx + x
                iZ_mpz(z, x_p, -1);      // compute p = iZ(x_p, -1)
                // check if tmp is prime
                found = mpz_probab_prime_p(z, MR_ROUNDS);

                if (found)
                    break;
            }

            // check if x7[x] is set
            if (bitmap_get_bit(iZm->base_x7, x))
            {
                mpz_add_ui(x_p, yvx, x); // set x_p = yvx + x
                iZ_mpz(z, x_p, 1);       // compute tmp = iZ(x_p, 1)
                // check if tmp is prime
                found = mpz_probab_prime_p(z, MR_ROUNDS);

                if (found)
                    break;
            }
        }

        mpz_add_ui(yvx, yvx, vx); // increment yvx by vx for next segment
    }
    else // backward search
    {
        if (i > 0)
            start_x = vx; // start from the end of the bitmaps

        // check iZ+ first if backward
        for (int x = start_x; x >= end_x; x--)
        {
            // check if x7[x] is set
            if (bitmap_get_bit(iZm->base_x7, x))
            {
                mpz_add_ui(x_p, yvx, x); // set x_p = yvx + x
                iZ_mpz(z, x_p, 1);       // compute tmp = iZ(x_p, 1)
                // check if tmp is prime
                found = mpz_probab_prime_p(z, MR_ROUNDS);

                if (found)
                    break;
            }

            // check iZ-
            if (bitmap_get_bit(iZm->base_x5, x))
            {
                mpz_add_ui(x_p, yvx, x); // set x_p = yvx + x
                iZ_mpz(z, x_p, -1);      // compute p = iZ(x_p, -1)
                // check if tmp is prime
                found = mpz_probab_prime_p(z, MR_ROUNDS);

                if (found)
                    break;
            }
        }

        mpz_sub_ui(yvx, yvx, vx); // decrement yvx by vx for next segment
    }

```

```
        i++; // increment segment counter
    }

    // 3. Set the found prime
    if (found)
        mpz_set(p, z); // set p = tmp
    else
        log_debug("No prime found :/");

    // cleanup
    iZm_free(&iZm);
    mpz_clears(y, yvx, x_p, z, NULL);

    return found;
}
```

8.17 src/prime_sieve.c File Reference

Implementations of classical and SiZ-family sieve algorithms.

#include <iZ_api.h>

Functions

- static uint64_t est_pi_n (int64_t n)

  Estimate the number of primes up to n using the prime number theorem.
- static void process_eratosthenes_bitmap (UI64_ARRAY *primes, BITMAP *sieve_bitmap, uint64_t limit)

  Shared odd-only Eratosthenes pass used by SoE and SSoE setup.

Classic Prime Sieve Algorithms

Baseline sieves up to a numeric limit.

All classic sieve functions take a limit n and return an ascending list of primes <= n.

- UI64_ARRAY * SoE (uint64_t n)

  Optimized Sieve of Eratosthenes.
- UI64_ARRAY * SSoE (uint64_t n)

  Segmented Sieve of Eratosthenes.
- UI64_ARRAY * SoEu (uint64_t n)

  Euler (linear) sieve.
- UI64_ARRAY * SoS (uint64_t n)

  Sieve of Sundaram.
- UI64_ARRAY * SoA (uint64_t n)

  Sieve of Atkin.

iZ-based Sieve Algorithms

Sieve family operating in the 6x-1 / 6x+1 index space.

- UI64_ARRAY * SiZ (uint64_t n)

  Solid Sieve-iZ (wheel 6, iZ index space).
- UI64_ARRAY * SiZm (uint64_t n)

  Segmented Sieve-iZm (VX segmented, horizontal processing).
- UI64_ARRAY * SiZm_vy (uint64_t n)

  Segmented Sieve-iZm (vertical processing; faster, unordered output).

8.17.1 Detailed Description

Implementations of classical and SiZ-family sieve algorithms.

Classical routines (SoE/SSoE/SoEu/SoS/SoA) are provided as baseline implementations. SiZ-family routines (SiZ/SiZm/SiZm_vy) operate in iZ index space over numbers of the form 6x-1 and 6x+1.

Definition in file prime_sieve.c.

8.17.2 Function Documentation

8.17.2.1 est_pi_n()

```
uint64_t est_pi_n (
            int64_t n)   [static]
```

Estimate the number of primes up to n using the prime number theorem.

Parameters

n	The input value.

Returns

uint64_t The result of the prime counting function (n/ln(n)) ∗ 1.4.

Definition at line 20 of file prime_sieve.c.

Referenced by SiZ(), SiZm(), SoA(), SoE(), SoEu(), SoS(), and SSoE().

8.17.2.2 process_eratosthenes_bitmap()

```
void process_eratosthenes_bitmap (
            UI64_ARRAY * primes,
            BITMAP * sieve_bitmap,
            uint64_t limit)   [static]
```

Shared odd-only Eratosthenes pass used by SoE and SSoE setup.

Parameters

primes	Output prime array.
sieve_bitmap	Candidate bitmap.
limit	Inclusive numeric upper bound represented in sieve_bitmap.

Definition at line 35 of file prime_sieve.c.

References bitmap_clear_steps_simd(), bitmap_get_bit(), and ui64_push().

Referenced by SoE(), and SSoE().

8.18 prime_sieve.c

Go to the documentation of this file.

```
#include <iZ_api.h>

static uint64_t est_pi_n(int64_t n)
{
    return (n / log(n)) * 1.4; // 40% over-estimation to avoid reallocs
}

// ============================================================
// * Classic Sieve Algorithms
// ============================================================

static void process_eratosthenes_bitmap(UI64_ARRAY *primes, BITMAP *sieve_bitmap, uint64_t limit)
{
    ui64_push(primes, 2); // Add 2 to primes, we will skip even numbers in the loop
    uint64_t n_sqrt = sqrt(limit);

    // Sieve logic: collect unmarked primes and mark their multiples if p <= sqrt(n),
    // skipping even numbers
    for (uint64_t i = 3; i <= limit; i += 2)
    {
        if (bitmap_get_bit(sieve_bitmap, i))
        {
            ui64_push(primes, i);
            if (i <= n_sqrt)
            {
                bitmap_clear_steps_simd(sieve_bitmap, 2 * i, i * i, limit + 1);
            }
        }
    }
}

UI64_ARRAY *SoE(uint64_t n)
{
    // Bound check:
    assert(n > 10 && n <= pow(10, 12) && "Input must be in the range (10, 10^12).");

    // Initialize the primes object with an estimated capacity
    UI64_ARRAY *primes = ui64_init(est_pi_n(n));
    assert(primes && "Memory allocation failed for primes array.");

    // Create a bitmap to mark prime numbers
    BITMAP *sieve = bitmap_init(n + 1, 1);
    if (!sieve)
    {
        ui64_free(&primes);
        return NULL;
    }

    // Sieve logic
    process_eratosthenes_bitmap(primes, sieve, n);

    // cleanup and finalize
    bitmap_free(&sieve);        // free bitmap
    ui64_resize_to_fit(primes); // Trim excess memory in primes array

    return primes;
}

UI64_ARRAY *SSoE(uint64_t n)
{
    // Bound check:
    assert(n > 10 && n <= pow(10, 12) && "Input must be in the range (10, 10^12).");

    // Initialize UI64_ARRAY with an estimated capacity
    UI64_ARRAY *primes = ui64_init(est_pi_n(n));
    assert(primes && "Memory allocation failed for primes array.");

    // Define the segment size; can be tuned based on memory constraints
    uint64_t segment_size = (uint64_t)sqrt(n);

    // * Step 1: Sieve small primes up to sqrt(n) using the traditional sieve
    BITMAP *sieve = bitmap_init(segment_size + 8, 1);
    if (!sieve)
    {
        ui64_free(&primes);
        return NULL;
    }

    // process first segment to collect root primes
    process_eratosthenes_bitmap(primes, sieve, segment_size);
```

```
    // * Step 2: Segmented sieve
    uint64_t low = segment_size + 1;
    uint64_t high = low + segment_size - 1;

    // Iterate over segments
    while (low <= n)
    {
        bitmap_set_all(sieve); // Reset segment bitmap
        uint64_t root_limit = sqrt(high);

        // Sieve the current segment using primes <= sqrt(high)
        for (int i = 1; i < primes->count; i++) // skip 2
        {
            uint64_t p = primes->array[i];
            if (p > root_limit)
                break;

            // Find the minimum number in [low, high] that is a multiple of p
            uint64_t start = (low / p) * p;
            start += (start < low) ? p : 0;     // ensure start >= low
            start += (start % 2 == 0) ? p : 0; // ensure start is odd
            start = MAX(p * p, start);          // start from p^2 or higher

            // Mark multiples of p within the segment
            bitmap_clear_steps_simd(sieve, 2 * p, start - low, high - low + 1);
        }

        // Collect primes from the current segment
        uint64_t i = (low % 2 == 0) ? low + 1 : low;
        for (; i <= high; i += 2) // skip even numbers
        {
            if (bitmap_get_bit(sieve, i - low))
                ui64_push(primes, i);
        }

        // Move to the next segment
        low += segment_size;
        high += segment_size;
        if (high > n)
            high = n;
    }

    // * Step 3: Cleanup and finalize
    bitmap_free(&sieve);        // free bitmap
    ui64_resize_to_fit(primes); // Trim excess memory in primes array

    return primes;
}

UI64_ARRAY *SoEu(uint64_t n)
{
    // Bound check:
    assert(n > 10 && n <= pow(10, 12) && "Input must be in the range (10, 10^12).");

    // initialization
    UI64_ARRAY *primes = ui64_init(est_pi_n(n));
    assert(primes && "Memory allocation failed for primes array.");

    BITMAP *sieve = bitmap_init(n + 1, 1);
    if (sieve == NULL)
    {
        ui64_free(&primes);
        return NULL;
    }

    // starting the prime list with 2 to skip reading even numbers
    ui64_push(primes, 2);

    // sieve logic: iterate through odd numbers, marking composites by clearing multiples of primes
    for (uint64_t i = 3; i <= n; i += 2)
    {
        if (bitmap_get_bit(sieve, i))
            ui64_push(primes, i);

        // Mark multiples of the current prime
        for (int j = 1; j < primes->count; ++j)
        {
            uint64_t p = primes->array[j];

            if (p * i > n)
                break;

            bitmap_clear_bit(sieve, p * i);

            if (i % p == 0)
                break;
```

```
        }
    }

    // cleanup
    bitmap_free(&sieve);

    // Resize primes array to fit the exact number of primes found
    ui64_resize_to_fit(primes);

    return primes;
}

UI64_ARRAY *SoS(uint64_t n)
{
    // Bound check:
    assert(n > 10 && n <= pow(10, 12) && "Input must be in the range (10, 10^12).");

    // Calculate k as the odd numbers up to n.
    uint64_t k = (n - 1) / 2 + 1;

    // Initialize the primes object with an estimated capacity
    UI64_ARRAY *primes = ui64_init(est_pi_n(n));
    assert(primes && "Memory allocation failed for primes array.");

    ui64_push(primes, 2);

    // Create a bitmap with size k+1.
    BITMAP *sieve = bitmap_init(k + 8, 1);
    if (sieve == NULL)
    {
        ui64_free(&primes);
        return NULL;
    }

    uint64_t n_sqrt = sqrt(n) + 1;

    // iterate through odd numbers
    for (uint64_t i = 1; i < k; ++i)
    {
        if (bitmap_get_bit(sieve, i))
        {
            uint64_t p = 2 * i + 1;
            ui64_push(primes, p);
            if (p < n_sqrt)
            {
                // First composite mark xp in the bitmap is given by:
                // xp = 2 * i * i + 2 * i = p * i + i, corresponding to p^2 in the odd set
                uint64_t xp = p * i + i;
                // Mark composites of p additively in the bitmap using step size p
                bitmap_clear_steps_simd(sieve, p, xp, k);
            }
        }
    }

    // Cleanup
    bitmap_free(&sieve);

    // Resize primes array to fit the exact number of primes found
    ui64_resize_to_fit(primes);

    return primes;
}

UI64_ARRAY *SoA(uint64_t n)
{
    // Bound check:
    assert(n > 10 && n <= pow(10, 12) && "Input must be in the range (10, 10^12).");

    UI64_ARRAY *primes = ui64_init(est_pi_n(n));
    assert(primes && "Memory allocation failed for primes array.");

    // Create a bitmap to mark potential primes
    BITMAP *sieve = bitmap_init(n + 1, 0);
    if (sieve == NULL)
    {
        ui64_free(&primes);
        return NULL;
    }

    // Add 2 and 3 to primes
    ui64_push(primes, 2);
    ui64_push(primes, 3);

    // 1. Mark potential primes in the bitmap using the Atkin conditions
    // Condition 1: for all 4x^2 + y^2 <= n
    for (uint64_t x = 1; 4 * x * x < n; x++)
    {
```

```
        uint64_t a = 4 * x * x;
        for (uint64_t y = 1; a + y * y <= n; ++y)
        {
            uint64_t b = a + y * y;
            // if 4x^2 + y^2   1 or 5 (mod 12), flip the bit
            if (b % 12 == 1 || b % 12 == 5)
                bitmap_flip_bit(sieve, b);
        }
    }

    // Condition 2: for all 3x^2 + y^2 <= n
    for (uint64_t x = 1; 3 * x * x < n; x++)
    {
        uint64_t a = 3 * x * x;
        for (uint64_t y = 1; a + y * y <= n; ++y)
        {
            uint64_t b = a + y * y;
            // if 3x^2 + y^2   7 (mod 12), flip the bit
            if (b % 12 == 7)
                bitmap_flip_bit(sieve, b);
        }
    }

    // Condition 3: for all 3x^2 - y^2 <= n and x > y
    for (uint64_t x = 1; 2 * x * x < n; x++) // Approximation for loop bound
    {
        uint64_t a = 3 * x * x;
        for (uint64_t y = x - 1; y > 0; y--)
        {
            uint64_t b = a - y * y;
            if (b > n)
                break; // b increases as y decreases

            // if 3x^2 - y^2   11 (mod 12), flip the bit
            if (b % 12 == 11)
                bitmap_flip_bit(sieve, b);
        }
    }

    // 2. Remove remaining composites by sieving out multiples of squares of root primes
    uint64_t n_sqrt = sqrt(n);
    for (uint64_t p = 5; p <= n_sqrt; p += 2)
    {
        if (bitmap_get_bit(sieve, p))
        {
            // Mark odd multiples of p^2 as non-prime
            // starting at p^2 with step size 2p^2
            bitmap_clear_steps_simd(sieve, 2 * p * p, p * p, n + 1);
        }
    }

    // 3. Collect primes from the bitmap
    for (uint64_t p = 5; p <= n; p += 2)
    {
        if (bitmap_get_bit(sieve, p))
            ui64_push(primes, p);
    }

    // cleanup
    bitmap_free(&sieve);

    // Resize primes array to fit the exact number of primes found
    ui64_resize_to_fit(primes);

    return primes;
}

// ============================================================
// * Sieve-iZ Algorithms
// ============================================================

UI64_ARRAY *SiZ(uint64_t n)
{
    // Sanity check:
    assert(n > 10 && n <= pow(10, 12) && "Input must be in the range (10, 10^12).");

    // Initialize primes object with enough initial estimation
    UI64_ARRAY *primes = ui64_init(est_pi_n(n));
    assert(primes && "Memory allocation failed for primes array.");

    // Add 2, 3 to primes
    ui64_push(primes, 2);
    ui64_push(primes, 3);

    // Calculate x_n, max x value in iZ space for given n
    uint64_t x_n = n / 6 + 1;

```

```
    // Create bitmap X-Arrays x5, x7, each of size x_n + 1 bits
    BITMAP *x5 = bitmap_init(x_n + 1, 1);
    BITMAP *x7 = bitmap_init(x_n + 1, 1);

    // Memory allocation failed, check logs
    if (!x5 || !x7)
    {
        ui64_free(&primes);
        return NULL;
    }

    // Sieve logic
    process_iZ_bitmaps(primes, x5, x7, x_n);

    // Cleanup: free memory of x5, x7
    bitmap_free(&x5);
    bitmap_free(&x7);

    // Handle edge case: if last prime > n, remove it
    if (primes->array[primes->count - 1] > n)
        ui64_pop(primes);

    // Trim unused memory in primes object
    ui64_resize_to_fit(primes);

    return primes;
}

UI64_ARRAY *SiZm(uint64_t n)
{
    // Bounds check:
    assert(n > 10 && n <= pow(10, 12) && "Input must be in the range (10, 10^12).");

    // if n < 10000, return SiZ(n), doesn't worth segmenting
    if (n < 10000)
        return SiZ(n);

    // * 1. Initialization:
    // Initialize primes array with enough capacity to avoid reallocs
    UI64_ARRAY *primes = ui64_init(est_pi_n(n));
    assert(primes && "Memory allocation failed for primes array in SiZm.");

    // Compute vx wheel size that fits in L2 cache
    int vx = compute_l2_vx(n);

    // Initialize and construct base bitmaps for iZm/vx
    BITMAP *base_x5 = bitmap_init(vx + 8, 1);
    BITMAP *base_x7 = bitmap_init(vx + 8, 1);
    if (!base_x5 || !base_x7)
    {
        ui64_free(&primes);
        return NULL;
    }
    iZm_construct_vx_base(vx, base_x5, base_x7);

    // Add the pre-sieved k primes to primes array
    const uint64_t base_primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    int k = 0;
    while ((6 * vx) % base_primes[k] == 0)
        ui64_push(primes, base_primes[k++]);

    // * 2. Process first segment (y = 0) to collect root primes:
    uint64_t x_n = n / 6 + 1; // max x value up to n
    // uint64_t root_limit = sqrt(6 * x_limit) + 1;
    // Initialize active sieve bitmaps from base
    BITMAP *x5 = bitmap_clone(base_x5);
    BITMAP *x7 = bitmap_clone(base_x7);
    if (!x5 || !x7)
    {
        ui64_free(&primes);
        bitmap_free(&base_x5);
        bitmap_free(&base_x7);
        return NULL;
    }
    process_iZ_bitmaps(primes, x5, x7, vx + 1);

    // * 3. Process remaining segments (y >= 1) to collect primes:
    int y_limit = x_n / vx; // number of segments to process
    uint64_t yvx = vx;      // current base value (y * vx)
    for (int y = 1; y <= y_limit; y++)
    {
        // * a. Reset active bitmaps to base state
        memcpy(x5->data, base_x5->data, x5->byte_size);
        memcpy(x7->data, base_x7->data, x7->byte_size);

        int x_limit = (y < y_limit) ? vx : x_n % vx;        // local x limit adjusted for last segment
        uint64_t root_limit = sqrt(6 * (yvx + x_limit)) + 1; // local root limit for current segment
```

```
        // * b. Mark composites of root primes in current segment
        for (int i = k; i < primes->count; i++)
        {
            uint64_t p = primes->array[i];
            if (p > root_limit)
                break;

            // mark composites of p in current segment
            bitmap_clear_steps_simd(x5, p, iZm_solve_for_xp(-1, p, vx, y), x_limit);
            bitmap_clear_steps_simd(x7, p, iZm_solve_for_xp(1, p, vx, y), x_limit);
        }

        // * c. Collect unmarked indices as primes in current segment
        for (int x = 2; x <= x_limit; x++)
        {
            if (bitmap_get_bit(x5, x)) // i.e. iZ- prime
                ui64_push(primes, iZ(yvx + x, -1));

            if (bitmap_get_bit(x7, x)) // i.e. iZ+ prime
                ui64_push(primes, iZ(yvx + x, 1));
        }

        yvx += vx; // advance yvx for next segment
    }

    // * 4. Clean up and finalize
    bitmap_free(&x5);
    bitmap_free(&x7);
    bitmap_free(&base_x5);
    bitmap_free(&base_x7);

    // Handle edge case: if last prime > n, remove it
    if (primes->array[primes->count - 1] > n)
        ui64_pop(primes);

    ui64_resize_to_fit(primes); // Trim excess memory in primes array
    return primes;
}

UI64_ARRAY *SiZm_vy(uint64_t n)
{
    // Bounds check:
    assert(n > 10 && n <= N_LIMIT && "Input must be in the range (10, 10000000000).");

    // if n is less than 10000, return SiZ(n)
    if (n < 10000)
        return SiZ(n);

    // * 1. Initialization:
    // Initialize primes array with enough capacity to avoid reallocs
    UI64_ARRAY *primes = ui64_init(n / log(n) * 1.4);
    assert(primes && "Memory allocation failed for primes array in SiZm.");

    uint64_t x_n = n / 6 + 1; // iZ limit for n
    uint64_t root_limit = sqrt(n) + 1;

    get_root_primes(primes, root_limit);
    int root_count = primes->count;

    int k = 4; // pointing at 11 in root_primes
    int vx = 35;
    if (n >= pow(10, 9))
    {
        vx *= 11;
        k++;
    }

    int vy = x_n / vx;
    BITMAP *vy_bitmap = bitmap_init(vy + 8, 1);

    // * 2. Sieve logic: Process segments in vy dimension
    for (int x = 2; x <= vx; x++)
    {
        // handle iZ- case
        if (gcd(iZ(x, -1), vx) == 1)
        {
            // * a. reset vy_bitmap
            bitmap_set_all(vy_bitmap); // set all bits

            // * b. mark composites of root primes in vy_bitmap
            for (int i = k; i < root_count; i++)
            {
                uint64_t p = primes->array[i];
                int64_t y_0 = iZm_solve_for_yp(-1, p, vx, x);
                bitmap_clear_steps_simd(vy_bitmap, p, y_0, vy);
            }
```

```
            // * c. collect primes from vy_bitmap
            for (int y = 0; y < vy; y++)
            {
                if (bitmap_get_bit(vy_bitmap, y))
                    ui64_push(primes, iZ(y * vx + x, -1));
            }
            // handle partial last row
            if (bitmap_get_bit(vy_bitmap, vy))
            {
                uint64_t p = iZ(vy * vx + x, -1);
                if (p < n)
                    ui64_push(primes, p);
            }
        }

        // handle iZ+ case
        if (gcd(iZ(x, 1), vx) == 1)
        {
            bitmap_set_all(vy_bitmap); // reset vy_bitmap
            for (int i = k; i < root_count; i++)
            {
                uint64_t p = primes->array[i];
                int64_t y_0 = iZm_solve_for_yp(1, p, vx, x);
                bitmap_clear_steps_simd(vy_bitmap, p, y_0, vy);
            }

            for (int y = 0; y < vy; y++)
            {
                if (bitmap_get_bit(vy_bitmap, y))
                    ui64_push(primes, iZ(y * vx + x, 1));
            }
            if (bitmap_get_bit(vy_bitmap, vy))
            {
                uint64_t p = iZ(vy * vx + x, 1);
                if (p < n)
                    ui64_push(primes, p);
            }
        }
    }

    // * 3. Clean up and finalize:
    bitmap_free(&vy_bitmap);
    ui64_resize_to_fit(primes); // Trim excess memory in primes array
    return primes;
}
```

8.19 src/toolkit/bitmap.c File Reference

Implementation of the bitmap module for efficient bit array operations.

#include <bitmap.h>

Functions

- BITMAP * bitmap_init (size_t size, int set_bits)

  Creates and initializes a new bitmap with the specified number of bits.
- void bitmap_free (BITMAP **bitmap)

  Frees all memory associated with a bitmap and nullifies the pointer.
- void bitmap_set_bit (BITMAP *bitmap, size_t idx)

  Sets a specific bit to 1.
- int bitmap_get_bit (BITMAP *bitmap, size_t idx)

  Gets the value of a specific bit.
- void bitmap_flip_bit (BITMAP *bitmap, size_t idx)

  Flips the value of a specific bit.
- void bitmap_clear_bit (BITMAP *bitmap, size_t idx)

  Clears (sets to 0) a specific bit.

- void bitmap_set_all (BITMAP *bitmap)

  Sets all bits in the array to 1.
- void bitmap_clear_all (BITMAP *bitmap)

  Clears all bits in the bitmap (sets them to 0).
- void bitmap_clear_steps (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)

  Clear every step-th bit from start_idx up to limit.
- void bitmap_clear_steps_simd (BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)

  SIMD-optimized version of bitmap_clear_steps.
- BITMAP * bitmap_clone (BITMAP *src)

  Creates a deep copy (clone) of an existing bitmap.
- void bitmap_compute_hash (BITMAP *bitmap)

  Generates a SHA-256 hash for the bitmap->data.
- int bitmap_validate_hash (BITMAP *bitmap)

  Validates the SHA-256 hash stored in bitmap->sha256.
- int bitmap_fwrite (BITMAP *bitmap, FILE *file)

  Writes the bitmap to a binary file.
- BITMAP * bitmap_fread (FILE *file)

  Reads a bitmap from a binary file.

8.19.1 Detailed Description

Implementation of the bitmap module for efficient bit array operations.

This file provides the complete implementation of bitmap functions including memory management, bit manipulation, data integrity verification via SHA-256 hashing, and binary file I/O with automatic hash validation.

8.19.1.1 Implementation Notes

- All functions include NULL pointer and bounds checking
- File I/O operations verify all read/write operations succeed
- Overflow protection is implemented in loop-based operations
- Memory is managed with proper cleanup on errors
- SHA-256 hashing uses OpenSSL library

8.19.1.2 Performance Characteristics

- bitmap_init(): O(n/8) where n is number of bits
- Individual bit ops: O(1)
- Bulk operations: O(n/8)
- Hash computation: O(n/8)
- File I/O: O(n/8)

Author

iZprime.com

Date

October 2025

Version

1.0

See also

bitmap.h for API documentation

test/test_bitmap.c for test suite

Definition in file bitmap.c.

8.20 bitmap.c

Go to the documentation of this file.

```
00001
00030
00031 #include <bitmap.h>
00032
00033 #ifdef __aarch64__
00034 #include <arm_neon.h>
00035 #elif defined(__x86_64__) || defined(_M_X64)
00036 #include <immintrin.h>
00037 #endif
00038
00057 BITMAP *bitmap_init(size_t size, int set_bits)
00058 {
00059     assert(size > 0 && "Bitmap size must be positive");
00060
00061     BITMAP *bitmap = (BITMAP *)malloc(sizeof(BITMAP));
00062     if (bitmap == NULL)
00063     {
00064         log_error("Memory allocation failed for BITMAP struct");
00065         return NULL;
00066     }
00067
00068     bitmap->size = size;
00069     bitmap->byte_size = (size + 7) / 8;
00070     bitmap->data = (unsigned char *)malloc(bitmap->byte_size);
00071     if (bitmap->data == NULL)
00072     {
00073         free(bitmap);
00074         log_error("bitmap_create: Memory allocation failed for BITMAP");
00075         return NULL;
00076     }
00077
00078     if (set_bits)
00079         memset(bitmap->data, 0xFF, bitmap->byte_size);
00080     else
00081         memset(bitmap->data, 0x00, bitmap->byte_size);
00082
00083     memset(bitmap->sha256, 0, SHA256_DIGEST_LENGTH); // Initialize SHA-256 to zero
00084
00085     return bitmap;
00086 }
00087
00103 void bitmap_free(BITMAP **bitmap)
00104 {
00105     // Safe guard against double free and NULL pointer
00106     if (bitmap && *bitmap)
00107     {
00108         if ((*bitmap)->data)
```

```
        {
            free((*bitmap)->data);
            (*bitmap)->data = NULL;
        }

        free(*bitmap);
        *bitmap = NULL;
    }
}

void bitmap_set_bit(BITMAP *bitmap, size_t idx)
{
    bitmap->data[idx / 8] |= (1 « (idx % 8));
}

int bitmap_get_bit(BITMAP *bitmap, size_t idx)
{
    return (bitmap->data[idx / 8] & (1 « (idx % 8))) != 0;
}

void bitmap_flip_bit(BITMAP *bitmap, size_t idx)
{
    bitmap->data[idx / 8] ^= (1 « (idx % 8));
}

void bitmap_clear_bit(BITMAP *bitmap, size_t idx)
{
    // Clear the bit by ANDing with the negation of the bit mask
    bitmap->data[idx / 8] &= ~(1 « (idx % 8));
}

void bitmap_set_all(BITMAP *bitmap)
{
    memset(bitmap->data, 0xFF, (bitmap->size + 7) / 8);
}

void bitmap_clear_all(BITMAP *bitmap)
{
    memset(bitmap->data, 0x00, (bitmap->size + 7) / 8);
}

void bitmap_clear_steps(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
{
    assert(step > 0 && "Step must be positive in bitmap_clear_steps.");
    limit = MIN(limit, bitmap->size - 1);

    for (uint64_t idx = start_idx; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }
}

void bitmap_clear_steps_simd(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
{
    assert(step > 0 && "Step must be positive in bitmap_clear_steps_simd.");
    limit = MIN(limit, bitmap->size - 1);

#ifdef __aarch64__
    // NEON implementation for ARM64
    uint64_t idx = start_idx;

    // Process 4 steps at a time
    // Condition: idx + 3*step <= limit

    if (limit >= 3 * step && idx <= limit - 3 * step)
    {
        uint64x2_t v_step2 = vdupq_n_u64(2 * step);
        uint64x2_t v_step4 = vdupq_n_u64(4 * step);

        // Initialize indices: [idx, idx+step] and [idx+2step, idx+3step]
        uint64x2_t v_idx_01 = vdupq_n_u64(idx);
        v_idx_01 = vsetq_lane_u64(idx + step, v_idx_01, 1);

        uint64x2_t v_idx_23 = vaddq_u64(v_idx_01, v_step2);

        while (idx <= limit - 3 * step)
        {
            // Extract indices and perform operations
            // Note: NEON doesn't have efficient scatter for bytes, so we extract and do scalar stores

            uint64_t i0 = vgetq_lane_u64(v_idx_01, 0);
            uint64_t i1 = vgetq_lane_u64(v_idx_01, 1);
            uint64_t i2 = vgetq_lane_u64(v_idx_23, 0);
            uint64_t i3 = vgetq_lane_u64(v_idx_23, 1);

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));
```

```
            bitmap->data[i2 / 8] &= ~(1 « (i2 % 8));
            bitmap->data[i3 / 8] &= ~(1 « (i3 % 8));

            // Advance indices
            v_idx_01 = vaddq_u64(v_idx_01, v_step4);
            v_idx_23 = vaddq_u64(v_idx_23, v_step4);

            idx += 4 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }

#elif defined(__AVX2__)
    // AVX2 implementation for x86_64
    uint64_t idx = start_idx;

    // Process 4 steps at a time
    if (limit >= 3 * step && idx <= limit - 3 * step)
    {
        __m256i v_step4 = _mm256_set1_epi64x(4 * step);

        // Initialize indices: [idx, idx+step, idx+2step, idx+3step]
        // Note: _mm256_set_epi64x takes arguments in reverse order (e3, e2, e1, e0)
        __m256i v_idx = _mm256_set_epi64x(idx + 3 * step, idx + 2 * step, idx + step, idx);

        while (idx <= limit - 3 * step)
        {
            // Extract indices
            uint64_t i0 = _mm256_extract_epi64(v_idx, 0);
            uint64_t i1 = _mm256_extract_epi64(v_idx, 1);
            uint64_t i2 = _mm256_extract_epi64(v_idx, 2);
            uint64_t i3 = _mm256_extract_epi64(v_idx, 3);

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));
            bitmap->data[i2 / 8] &= ~(1 « (i2 % 8));
            bitmap->data[i3 / 8] &= ~(1 « (i3 % 8));

            // Advance indices
            v_idx = _mm256_add_epi64(v_idx, v_step4);
            idx += 4 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }

#elif defined(__SSE2__)
    // SSE2 implementation for x86_64
    uint64_t idx = start_idx;

    // Process 2 steps at a time
    if (limit >= step && idx <= limit - step)
    {
        __m128i v_step2 = _mm_set1_epi64x(2 * step);
        // Note: _mm_set_epi64x takes arguments in reverse order (e1, e0)
        __m128i v_idx = _mm_set_epi64x(idx + step, idx);

        while (idx <= limit - step)
        {
            // Extract indices
            // _mm_cvtsi128_si64 extracts the lower 64 bits (element 0)
            uint64_t i0 = _mm_cvtsi128_si64(v_idx);
            // Unpack high to low to extract element 1
            uint64_t i1 = _mm_cvtsi128_si64(_mm_unpackhi_epi64(v_idx, v_idx));

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));

            // Advance indices
            v_idx = _mm_add_epi64(v_idx, v_step2);
            idx += 2 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
```

```
    }

#else
    // Fallback to scalar implementation
    for (uint64_t idx = start_idx; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }
#endif
}

BITMAP *bitmap_clone(BITMAP *src)
{
    assert(src && src->data && "Invalid source bitmap passed to bitmap_clone.");

    BITMAP *dest = bitmap_init(src->size, 0);
    memcpy(dest->data, src->data, src->byte_size);
    memcpy(dest->sha256, src->sha256, SHA256_DIGEST_LENGTH);

    return dest;
}

void bitmap_compute_hash(BITMAP *bitmap)
{
    assert(bitmap && bitmap->data && "Invalid bitmap passed to bitmap_compute_hash.");

    // Generate SHA-256 hash and store it in the struct
    SHA256((unsigned char *)bitmap->data, bitmap->byte_size, bitmap->sha256);
}

int bitmap_validate_hash(BITMAP *bitmap)
{
    assert(bitmap && bitmap->data && "Invalid bitmap");

    unsigned char correct_hash[SHA256_DIGEST_LENGTH]; // Buffer to hold the computed hash

    // Generate SHA-256 hash and store it in correct_hash
    SHA256((unsigned char *)bitmap->data, bitmap->byte_size, correct_hash);

    // Compare actual_hash with the stored hash in bitmap->sha256
    if (memcmp(correct_hash, bitmap->sha256, SHA256_DIGEST_LENGTH) == 0)
    {
        return 1; // SHA-256 match
    }
    else
    {
        log_error("SHA-256 checksum validation failed.");
        return 0; // SHA-256 mismatch
    }
}

int bitmap_fwrite(BITMAP *bitmap, FILE *file)
{
    assert(bitmap && bitmap->data && "Invalid bitmap passed to bitmap_fwrite.");
    assert(file && "File pointer is NULL in bitmap_fwrite.");

    // Write size
    if (fwrite(&bitmap->size, sizeof(size_t), 1, file) != 1)
    {
        log_error("Failed to write bitmap size to file.");
        return 0;
    }

    // Write data
    if (fwrite(bitmap->data, sizeof(unsigned char), bitmap->byte_size, file) != bitmap->byte_size)
    {
        log_error("Failed to write bitmap data to file.");
        return 0;
    }

    // Compute SHA-256 hash if not already computed
    // Check if hash is all zeros by examining all bytes
    int hash_is_zero = 1;
    for (size_t i = 0; i < SHA256_DIGEST_LENGTH; i++)
    {
        if (bitmap->sha256[i] != 0)
        {
            hash_is_zero = 0;
            break;
        }
    }

    if (hash_is_zero)
    {
        bitmap_compute_hash(bitmap);
    }

```

```
    // Write SHA-256 hash
    if (fwrite(bitmap->sha256, 1, SHA256_DIGEST_LENGTH, file) != SHA256_DIGEST_LENGTH)
    {
        log_error("Failed to write SHA-256 hash to file.");
        return 0;
    }

    return 1; // Success
}

BITMAP *bitmap_fread(FILE *file)
{
    assert(file && "File pointer is NULL in bitmap_fread.");

    // Read size
    size_t size;
    if (fread(&size, sizeof(size_t), 1, file) != 1)
    {
        log_error("Failed to read bitmap size from file.");
        return NULL;
    }

    BITMAP *bitmap = bitmap_init(size, 0);
    if (bitmap == NULL)
    {
        log_error("Failed to initialize bitmap.");
        return NULL;
    }

    // Read data directly into the already-allocated buffer
    if (fread(bitmap->data, sizeof(unsigned char), bitmap->byte_size, file) != bitmap->byte_size)
    {
        log_error("Failed to read complete bitmap data from file.");
        bitmap_free(&bitmap);
        return NULL;
    }

    // Read SHA-256 hash
    if (fread(bitmap->sha256, 1, SHA256_DIGEST_LENGTH, file) != SHA256_DIGEST_LENGTH)
    {
        log_error("Failed to read SHA-256 hash from file.");
        bitmap_free(&bitmap);
        return NULL;
    }

    if (!bitmap_validate_hash(bitmap))
    {
        log_error("SHA-256 hash validation failed.");
        bitmap_free(&bitmap);
        return NULL;
    }

    return bitmap;
}
```

8.21 src/toolkit/int_arrays.c File Reference

Implementation of dynamic integer array module for 16-bit, 32-bit, and 64-bit unsigned integers.

#include <int_arrays.h>
#include "templates/int_array_impl.inc"

8.21.1 Detailed Description

Implementation of dynamic integer array module for 16-bit, 32-bit, and 64-bit unsigned integers.

This file provides the complete implementation of UI16_ARRAY, UI32_ARRAY, and UI64_ARRAY functions including memory management, automatic capacity growth, data integrity verification via SHA-256 hashing, and binary file I/O with automatic hash validation.

8.21.1.1 Implementation Notes

- All functions include NULL pointer checking
- Automatic capacity doubling (2x growth) ensures amortized O(1) append operations
- File I/O operations automatically compute and validate SHA-256 hashes
- Memory is managed with proper cleanup on errors
- SHA-256 hashing uses OpenSSL library

8.21.1.2 Performance Characteristics

- init(): O(1)
- push() (amortized): O(1)
- push() (worst case): O(n) during resize
- Hash operations: O(n)
- File I/O: O(n)

Author

iZprime.com

Date

October 2025

Version

1.0

See also

int_arrays.h for API documentation

Definition in file int_arrays.c.

8.22 int_arrays.c

Go to the documentation of this file.

```
00029
00030 #include <int_arrays.h>
00031
00032 // ==========================================================================
00033 // UI16_ARRAY IMPLEMENTATION
00034 // ==========================================================================
00036 #define TEMPLATE_TYPE uint16_t
00037 #define TEMPLATE_STRUCT UI16_ARRAY
00038 #define TEMPLATE_FUNC(name) ui16_##name
00039 #define TEMPLATE_NAME_STR "UI16_ARRAY"
00041 #include "templates/int_array_impl.inc"
00043 #undef TEMPLATE_TYPE
00044 #undef TEMPLATE_STRUCT
00045 #undef TEMPLATE_FUNC
00046 #undef TEMPLATE_NAME_STR
00048
00049 // ==========================================================================
00050 // UI32_ARRAY IMPLEMENTATION
00051 // ==========================================================================
00053 #define TEMPLATE_TYPE uint32_t
00054 #define TEMPLATE_STRUCT UI32_ARRAY
00055 #define TEMPLATE_FUNC(name) ui32_##name
00056 #define TEMPLATE_NAME_STR "UI32_ARRAY"
00058 #include "templates/int_array_impl.inc"
00060 #undef TEMPLATE_TYPE
00061 #undef TEMPLATE_STRUCT
00062 #undef TEMPLATE_FUNC
00063 #undef TEMPLATE_NAME_STR
00065
00066 // ==========================================================================
00067 // UI64_ARRAY IMPLEMENTATION
00068 // ==========================================================================
00070 #define TEMPLATE_TYPE uint64_t
00071 #define TEMPLATE_STRUCT UI64_ARRAY
00072 #define TEMPLATE_FUNC(name) ui64_##name
00073 #define TEMPLATE_NAME_STR "UI64_ARRAY"
00075 #include "templates/int_array_impl.inc"
00077 #undef TEMPLATE_TYPE
00078 #undef TEMPLATE_STRUCT
00079 #undef TEMPLATE_FUNC
00080 #undef TEMPLATE_NAME_STR
```

8.23 src/toolkit/iZ_toolkit.c File Reference

Implementation of iZ index space helpers and iZm/VX segment machinery.

#include <iZ_api.h>

Functions

- uint64_t iZ (uint64_t x, int i)

  Computes 6x + i for a given x and i.
- void iZ_mpz (mpz_t z, mpz_t x, int i)

  Computes 6x + i for arbitrary precision values using GMP.
- static int compute_k_vx (IZM *iZm)

  Count how many small primes (>3) divide vx.
- void iZm_construct_vx_base (uint64_t vx, BITMAP *base_x5, BITMAP *base_x7)

Build pre-sieved base bitmaps for a VX segment.

- int vx_set_base_values (VX_SEG *vx_obj, char *y_str)

  Initialize mpz-dependent base fields for a VX segment object.
- void vx_det_sieve (IZM *iZm, VX_SEG *vx_obj)

  Deterministic phase: mark composites using root primes.
- void vx_prob_sieve (VX_SEG *vx_obj)

  Perform probabilistic sieve cleanup for large numeric ranges.

iZ Mapping Helpers

- void process_iZ_bitmaps (UI64_ARRAY *primes, BITMAP *x5, BITMAP *x7, uint64_t x↩ _limit)

  Traverse iZ bitmaps, emit surviving primes, and mark composites.
- void get_root_primes (UI64_ARRAY *primes, uint64_t limit)

  Generate primes up to limit for deterministic sieving.

IZM Lifecycle

- IZM * iZm_init (size_t vx)

  Allocate and initialize an IZM object for a given VX.
- IZM * iZm_clone (IZM *src)

  Deep-copy an IZM object for per-worker ownership.
- void iZm_free (IZM **iZm)

  Release an IZM object and set the caller pointer to NULL.

VX Selection Helpers

- uint64_t compute_vx_k (uint64_t n, int max_k)

  Compute VX by multiplying up to max_k small primes.
- uint64_t compute_l2_vx (uint64_t n)

  Choose VX using an L2-cache-aware heuristic.
- void compute_max_vx (mpz_t vx, int bit_size)

  Compute largest primorial-like VX below 2^bit_size.

Modular Hit Solvers

- uint64_t iZm_solve_for_xp (int m_id, uint64_t p, uint64_t vx, uint64_t y)

  Solve first x-hit of prime p for line m_id in segment y.
- uint64_t iZm_solve_for_xp_mpz (int m_id, uint64_t p, uint64_t vx, mpz_t y)

  GMP variant of iZm_solve_for_xp() for very large y.
- int64_t iZm_solve_for_yp (int m_id, uint64_t p, uint64_t vx, uint64_t x)

  Solve first y-hit for fixed x in vertical (vy) scanning.

VX Segment Lifecycle and Execution

- VX_SEG * vx_init (IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds)

  Initialize and deterministically sieve one VX segment.
- void vx_free (VX_SEG **vx_obj)

  Free a VX segment and all owned resources.
- void vx_collect_p_gaps (VX_SEG *vx_obj)

  Extract prime-gap encoding from a fully sieved segment.
- void vx_full_sieve (VX_SEG *vx_obj, int collect_p_gaps)

  Complete segment processing (probabilistic stage and optional gaps).
- void vx_stream_file (VX_SEG *vx_obj, FILE *output)

  Stream segment primes to a file.

Random Prime Search Routines

- int vx_search_prime (mpz_t p, int m_id, int vx, int bit_size)

  Horizontal iZm/VX random-prime search.
- int vy_search_prime (mpz_t p, int m_id, mpz_t vx)

  Vertical iZm/VY random-prime search.

Variables

- static const int s_primes []
- static int s_primes_count = sizeof(s_primes) / sizeof(int)

8.23.1 Detailed Description

Implementation of iZ index space helpers and iZm/VX segment machinery.

Definition in file iZ_toolkit.c.

8.23.2 Function Documentation

8.23.2.1 compute_k_vx()

```
int compute_k_vx (
            IZM * iZm)    [static]
```

Count how many small primes (>3) divide vx.

Parameters

iZm	Initialized toolkit context.

Returns

Number of pre-sieved small primes encoded in vx.

Definition at line 52 of file iZ_toolkit.c.

References s_primes, and IZM::vx.

Referenced by iZm_init().

8.23.2.2 vx_det_sieve()

```
void vx_det_sieve (
            IZM * iZm,
            VX_SEG * vx_obj)
```

Deterministic phase: mark composites using root primes.

Parameters

iZm	Toolkit context containing root-prime table and base bitmaps.
vx_obj	Segment object to update.

Definition at line 550 of file iZ_toolkit.c.

References UI64_ARRAY::array, VX_SEG::bit_ops, bitmap_clear_steps_simd(), bitmap_get_bit(), UI64_ARRAY::count, VX_SEG::end_x, VX_SEG::is_large_limit, iZm_solve_for_xp(), iZm_solve_for_xp_mpz(), IZM::k_vx, VX_SEG::p_count, VX_SEG::root_limit, IZM::root_primes, VX_SEG::start_x, VX_SEG::vx, VX_SEG::x5, VX_SEG::x7, and VX_SEG::y.

Referenced by vx_init().

8.23.2.3 vx_prob_sieve()

```
void vx_prob_sieve (
            VX_SEG * vx_obj)
```

Perform probabilistic sieve cleanup for large numeric ranges.

Parameters

vx_obj	Segment object containing deterministic survivors.

Definition at line 617 of file iZ_toolkit.c.

References bitmap_clear_bit(), bitmap_get_bit(), VX_SEG::end_x, VX_SEG::is_large_limit, iZ_mpz(), log_info(), VX_SEG::mr_rounds, VX_SEG::p_count, VX_SEG::p_test_ops, VX_SEG::start_x, VX_SEG::x5, VX_SEG::x7, and VX_SEG::yvx.

Referenced by vx_full_sieve().

8.23.2.4 vx_set_base_values()

```
int vx_set_base_values (
            VX_SEG * vx_obj,
            char * y_str)
```

Initialize mpz-dependent base fields for a VX segment object.

Parameters

vx_obj	Segment object to populate.
y_str	Decimal y-coordinate for the segment.

Returns

1 on success, 0 on parse/initialization failure.

Definition at line 513 of file iZ_toolkit.c.

References VX_SEG::is_large_limit, iZ_mpz(), log_error(), VX_SEG::root_limit, VX_SEG::vx, VX_SEG::y, and VX_SEG::yvx.

Referenced by vx_init().

8.23.3 Variable Documentation

8.23.3.1 s_primes

```
const int s_primes[] [static]
```

Initial value:

```
= {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
   43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}
```

Small odd primes used to estimate pre-sieved factors in vx.

Definition at line 10 of file iZ_toolkit.c.

Referenced by compute_k_vx(), compute_l2_vx(), compute_vx_k(), and iZm_construct_vx_base().

8.23.3.2 s_primes_count

int s_primes_count = sizeof(s_primes) / sizeof(int) [static]

Number of entries available in s_primes.

Definition at line 13 of file iZ_toolkit.c.

Referenced by iZm_construct_vx_base().

8.24 iZ_toolkit.c

Go to the documentation of this file.

```
00001
00006
00007 #include <iZ_api.h>
00008
00010 static const int s_primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
00011                               43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97};
00013 static int s_primes_count = sizeof(s_primes) / sizeof(int);
00014
00024 inline uint64_t iZ(uint64_t x, int i)
00025 {
00026     return 6 * x + i;
00027 }
00028
00037 void iZ_mpz(mpz_t z, mpz_t x, int i)
00038 {
00039     mpz_mul_ui(z, x, 6); // z = 6 * x
00040
00041     if (i > 0)
00042         mpz_add_ui(z, z, i); // z = z + i
00043     else
00044         mpz_sub_ui(z, z, -i); // z = z - i
00045 }
00046
00052 static int compute_k_vx(IZM *iZm)
00053 {
00054     int k = 0;
00055     while (iZm->vx % s_primes[k + 2] == 0)
00056     {
00057         k++;
00058     }
00059     return k;
00060 }
00061
00070 void process_iZ_bitmaps(UI64_ARRAY *primes, BITMAP *x5, BITMAP *x7, uint64_t x_limit)
00071 {
00072     uint64_t root_limit = sqrt(6 * x_limit) + 1;
00073
00074     // Iterate through x values in range 0 < x < x_n
00075     for (uint64_t x = 1; x < x_limit; x++)
00076     {
00077         // if x5[x], implying it's iZ- prime
00078         if (bitmap_get_bit(x5, x))
00079         {
00080             uint64_t p = iZ(x, -1); // compute p = iZ(x, -1)
00081             ui64_push(primes, p);   // add p to primes
00082
00083             // if p is root prime, mark its multiples in x5, x7
00084             if (p < root_limit)
00085             {
00086                 bitmap_clear_steps_simd(x5, p, p * x + x, x_limit);
00087                 bitmap_clear_steps_simd(x7, p, p * x - x, x_limit);
00088             }
00089         }
00090
00091         // Do the same if x7[x], inverting the xp signs
00092         if (bitmap_get_bit(x7, x))
00093         {
00094             uint64_t p = iZ(x, 1);
00095             ui64_push(primes, p);
00096
00097             if (p < root_limit)
00098             {
00099                 bitmap_clear_steps_simd(x5, p, p * x - x, x_limit);
```

```
                bitmap_clear_steps_simd(x7, p, p * x + x, x_limit);
            }
        }
    }
}

void get_root_primes(UI64_ARRAY *primes, uint64_t limit)
{
    // Add 2, 3 to primes
    ui64_push(primes, 2);
    ui64_push(primes, 3);

    // Calculate x_n, max x value in iZ space for given n
    uint64_t x_n = limit / 6 + 1;

    // Create bitmap X-Arrays x5, x7, each of size x_n + 1 bits
    BITMAP *x5 = bitmap_init(x_n + 1, 1);
    BITMAP *x7 = bitmap_init(x_n + 1, 1);

    // Memory allocation failed, check logs
    if (!x5 || !x7)
    {
        ui64_free(&primes);
        return;
    }

    // Sieve logic: mark composites and collect primes up to limit
    process_iZ_bitmaps(primes, x5, x7, x_n);

    // Cleanup: free memory of x5, x7
    bitmap_free(&x5);
    bitmap_free(&x7);

    return;
}

// ==========================================================
// * iZm structure:
// ==========================================================

IZM *iZm_init(size_t vx)
{
    // validate vx
    assert(vx % 2 != 0 && vx % 3 != 0 && vx >= 35 && "vx must be at least 35.");

    IZM *iZm = malloc(sizeof(IZM));
    if (!iZm)
    {
        log_error("Memory allocation failed for iZm.");
        return NULL;
    }

    iZm->vx = vx;
    // get root primes for deterministic sieving
    iZm->root_primes = SiZ(vx);
    // iZm->root_primes = root_limit < pow(10, 7) ? SiZ(root_limit) : SiZm(root_limit);
    if (!iZm->root_primes)
    {
        log_error("Root primes generation failed for iZm.");
        free(iZm);
        return NULL;
    }

    iZm->k_vx = compute_k_vx(iZm);

    // initialize base bitmaps
    iZm->base_x5 = bitmap_init(vx + 10, 1);
    iZm->base_x7 = bitmap_init(vx + 10, 1);
    if (!iZm->base_x5 || !iZm->base_x7)
    {
        log_error("Bitmap initialization failed for iZm base segment.");
        iZm_free(&iZm);
        ui64_free(&iZm->root_primes);
        return NULL;
    }

    // construct pre-sieved base_x5, base_x7 bitmaps
    iZm_construct_vx_base(vx, iZm->base_x5, iZm->base_x7);

    return iZm;
}

IZM *iZm_clone(IZM *src)
{
    assert(src && "Invalid source IZM structure for cloning.");

    IZM *clone = malloc(sizeof(IZM));
```

```
    assert(clone && "Memory allocation failed for iZm clone.");

    clone->vx = src->vx;
    clone->k_vx = src->k_vx;
    clone->root_primes = ui64_init(src->root_primes->capacity);
    if (clone->root_primes == NULL)
    {
        log_error("Memory allocation failed for root primes in iZm clone.");
        iZm_free(&clone);
        return NULL;
    }
    // copy root primes
    memcpy(clone->root_primes->array, src->root_primes->array, src->root_primes->count * sizeof(uint64_t));
    clone->root_primes->count = src->root_primes->count;

    // clone base bitmaps
    clone->base_x5 = bitmap_clone(src->base_x5);
    clone->base_x7 = bitmap_clone(src->base_x7);

    return clone;
}

void iZm_free(IZM **iZm)
{
    if (iZm == NULL || *iZm == NULL)
        return;

    // Free root primes object allocated in iZm_init
    ui64_free(&(*iZm)->root_primes);
    bitmap_free(&(*iZm)->base_x5);
    bitmap_free(&(*iZm)->base_x7);
    free(*iZm);
    *iZm = NULL;
}

uint64_t compute_vx_k(uint64_t n, int max_k)
{
    uint64_t x_n = n / 6;
    uint64_t vx = 35; // minimum vx size
    int k = 4;        // pointing to prime 11 in s_primes

    // while vx * k-th prime doesn't exceed x_n
    while (vx * s_primes[k] < x_n && k < max_k + 2)
    {
        vx *= s_primes[k];
        k++;
    }

    return vx;
}

uint64_t compute_l2_vx(uint64_t n)
{
    uint64_t l2 = get_cpu_L2_cache_size_bits();
    uint64_t x_n = n / 6;
    uint64_t vx = 35; // minimum useful vx size
    int k = 4;        // pointing to prime 11 in s_primes

    // while vx * k-th prime doesn't exceed x_n and l2 cache size
    while (vx * s_primes[k] < MIN(l2, x_n))
    {
        vx *= s_primes[k];
        k++;
    }

    return vx;
}

void compute_max_vx(mpz_t vx, int bit_size)
{
    // get some primes to compute the primorial vx
    UI64_ARRAY *primes = SiZ(10000);
    assert(primes && "Failed to generate primes");

    int i = 2;                      // to skip 2, 3
    mpz_set_ui(vx, primes->array[i]); // set vx = 5

    // while vx * primes->array[i] < 2^bit_size
    while (mpz_sizeinbase(vx, 2) < (size_t)bit_size)
    {
        i++;
        mpz_mul_ui(vx, vx, primes->array[i]);
    }

    mpz_div_ui(vx, vx, primes->array[i]); // divide by the last prime
}
```

```
00347 void iZm_construct_vx_base(uint64_t vx, BITMAP *base_x5, BITMAP *base_x7)
00348 {
00349     assert(base_x5 && base_x7 && "Invalid bitmaps passed to iZm_construct_vx_base.");
00350
00351     bitmap_set_all(base_x5);
00352     bitmap_set_all(base_x7);
00353     bitmap_clear_bit(base_x5, 0); // clear the 0th bit
00354     bitmap_clear_bit(base_x7, 0); // clear the 0th bit
00355
00356     // iterate through small primes that divide vx
00357     for (int i = 2; i < s_primes_count; i++)
00358     {
00359         int p = s_primes[i];
00360         if (vx % p != 0)
00361             continue;
00362
00363         int ip = (p % 6 == 1) ? 1 : -1;
00364         int xp = (p + 1) / 6;
00365         // clear composites in iZ- and iZ+
00366         if (ip == -1)
00367         {
00368             bitmap_clear_bit(base_x5, xp);                    // clear the prime itself
00369             bitmap_clear_steps_simd(base_x5, p, p * xp + xp, vx); // mark composites in iZ-
00370             bitmap_clear_steps_simd(base_x7, p, p * xp - xp, vx); // mark composites in iZ+
00371         }
00372         else
00373         {
00374             bitmap_clear_bit(base_x7, xp);                    // clear the prime itself
00375             bitmap_clear_steps_simd(base_x5, p, p * xp - xp, vx); // mark composites in iZ-
00376             bitmap_clear_steps_simd(base_x7, p, p * xp + xp, vx); // mark composites in iZ+
00377         }
00378     }
00379 }
00380
00399 uint64_t iZm_solve_for_xp(int m_id, uint64_t p, uint64_t vx, uint64_t y)
00400 {
00401     uint64_t xp = (p + 1) / 6;
00402     int ip = (p % 6 == 1) ? 1 : -1;
00403
00404     // Immediate solution check for y = 0
00405     if (y == 0)
00406     {
00407         // compute p * xp + m_id * ip * xp
00408         return p * xp + m_id * ip * xp;
00409     }
00410
00411     // Normalize x_p to x_p if p_id = m_id, else to p - x_p
00412     xp = (m_id == ip) ? xp : p - xp;
00413     uint64_t yvx = vx * y;
00414
00415     // Compute the first composite hit of p in the yth vx segment in iZm index space
00416     uint64_t x = p < vx ? p - (yvx - xp) % p : (yvx - xp) % p;
00417     return x;
00418 }
00419
00433 uint64_t iZm_solve_for_xp_mpz(int m_id, uint64_t p, uint64_t vx, mpz_t y)
00434 {
00435     mpz_t tmp;
00436     mpz_init(tmp);
00437
00438     // 1. Normalize x_p to x_p if p_id = m_id, else to p - x_p
00439     uint64_t xp = (p + 1) / 6;
00440     int ip = (p % 6 == 1) ? 1 : -1;
00441     xp = (m_id == ip) ? xp : p - xp;
00442
00443     mpz_mul_ui(tmp, y, vx);
00444     mpz_sub_ui(tmp, tmp, xp);
00445     mpz_mod_ui(tmp, tmp, p);
00446
00447     uint64_t x = p < vx ? p - mpz_get_ui(tmp) : mpz_get_ui(tmp);
00448
00449     mpz_clear(tmp);
00450     return x;
00451 }
00452
00474 int64_t iZm_solve_for_yp(int m_id, uint64_t p, uint64_t vx, uint64_t x)
00475 {
00476     // No solution check, if vx and p are not coprime
00477     if (gcd(vx, p) != 1)
00478     {
00479         printf("There's no solution for the given parameters\n");
00480         return -1; // No solution can be found
00481     }
00482
00483     uint64_t xp = (p + 1) / 6;
00484     int ip = (p % 6 == 1) ? 1 : -1;
00485     xp = (m_id == ip) ? xp : p - xp;
```

```c

    // Immediate solution check
    if (x % p == xp)
        return 0;

    // Compute delta
    uint64_t x_mod_p = x % p;
    uint64_t delta = (xp + p - x_mod_p) % p;

    // Find modular inverse
    uint64_t vx_inv = modular_inverse(vx % p, p);

    // Compute and return y
    uint64_t y = (delta * vx_inv) % p;
    return y;
}

// ==================================================
// * VX_SEG structure:
// ==================================================

int vx_set_base_values(VX_SEG *vx_obj, char *y_str)
{

    // assert y_str is numeric
    mpz_t y_tmp;
    mpz_init(y_tmp);
    if (mpz_set_str(y_tmp, y_str, 10) != 0)
    {
        log_error("Invalid numeric string for y in vx_set_base_values\n");
        mpz_clear(y_tmp);
        return 0;
    }

    mpz_init_set(vx_obj->y, y_tmp);
    mpz_clear(y_tmp);

    // Compute yvx = y * vx
    mpz_init(vx_obj->yvx);
    mpz_mul_ui(vx_obj->yvx, vx_obj->y, vx_obj->vx);

    // Compute root_limit = sqrt(iZ(vx * (y+1), 1))
    mpz_init(vx_obj->root_limit);
    mpz_add_ui(vx_obj->root_limit, vx_obj->yvx, vx_obj->vx);
    iZ_mpz(vx_obj->root_limit, vx_obj->root_limit, 1);
    mpz_sqrt(vx_obj->root_limit, vx_obj->root_limit);

    // Set is_large_limit to determine if probabilistic primality test is needed
    // if root_limit > vx
    vx_obj->is_large_limit = mpz_cmp_ui(vx_obj->root_limit, vx_obj->vx) > 0;
    return 1;
}

void vx_det_sieve(IZM *iZm, VX_SEG *vx_obj)
{
    assert(iZm && "iZm is NULL in vx_det_sieve");
    assert(vx_obj && "vx_obj is NULL in vx_det_sieve");

    int vx = vx_obj->vx; // segment size

    // * Deterministic Sieve: Mark composites of primes < vx in x5, x7
    int start_x = vx_obj->start_x;
    int end_x = vx_obj->end_x;
    int k = 2 + iZm->k_vx; // skip 2, 3 and pre-sieved k_vx primes
    UI64_ARRAY *root_primes = iZm->root_primes;

    // if y < 2^64, use iZm_solve_for_xp version for efficiency
    if (mpz_sizeinbase(vx_obj->y, 2) <= 64)
    {
        uint64_t y = mpz_get_ui(vx_obj->y);
        uint64_t root_limit = mpz_get_ui(vx_obj->root_limit);

        // Iterate through root primes, skipping the first k pre-sieved primes
        for (int i = k; i < root_primes->count; i++)
        {
            uint64_t p = root_primes->array[i];

            if (p > root_limit)
                break;

            // Mark composites of p in x5 and x7
            bitmap_clear_steps_simd(vx_obj->x5, p, iZm_solve_for_xp(-1, p, vx, y), end_x);
            bitmap_clear_steps_simd(vx_obj->x7, p, iZm_solve_for_xp(1, p, vx, y), end_x);

            vx_obj->bit_ops += (2 * end_x) / p; // approximate number of bit operations
        }
    }
```

```
    else
    {
        // the same as above but using iZm_solve_for_xp_mpz version
        for (int i = k; i < root_primes->count; i++)
        {
            int p = root_primes->array[i];

            bitmap_clear_steps_simd(vx_obj->x5, p, iZm_solve_for_xp_mpz(-1, p, vx, vx_obj->y), end_x);
            bitmap_clear_steps_simd(vx_obj->x7, p, iZm_solve_for_xp_mpz(1, p, vx, vx_obj->y), end_x);

            vx_obj->bit_ops += (2 * end_x) / p;
        }
    }

    // if no further probabilistic testing is needed,
    // count unmarked bits in x5 and x7 as p_count
    if (!vx_obj->is_large_limit)
    {
        vx_obj->p_count = 0;
        for (int x = start_x; x <= end_x; x++)
        {
            if (bitmap_get_bit(vx_obj->x5, x))
                vx_obj->p_count++;
            if (bitmap_get_bit(vx_obj->x7, x))
                vx_obj->p_count++;
        }
    }
}

void vx_prob_sieve(VX_SEG *vx_obj)
{
    // If is_large_limit is false, no need for probabilistic testing
    if (!vx_obj->is_large_limit)
    {
        log_info("vx_obj->is_large_limit is false, no need for probabilistic testing");
        return;
    }

    // Initialize GMP reusable variables p, x_p
    mpz_t p, x_p;
    mpz_init(p);
    mpz_init(x_p);

    int r = vx_obj->mr_rounds;
    int s = vx_obj->start_x <= 1 ? 1 : vx_obj->start_x;

    // Iterate through x values in the range start_x <= x <= end_x
    for (int x = s; x <= vx_obj->end_x; x++)
    {
        // Check if iZ(vx * y + x, -1) is prime, if not, clear x in x5
        if (bitmap_get_bit(vx_obj->x5, x))
        {
            // Compute x_p = yvx + x
            mpz_add_ui(x_p, vx_obj->yvx, x);
            iZ_mpz(p, x_p, -1); // Compute p = iZ(x_p, -1)
            int is_prime = mpz_probab_prime_p(p, r);
            vx_obj->p_test_ops++;

            // if is_prime, increment count, else clear composite from x5
            if (is_prime)
            {
                vx_obj->p_count++;
            }
            else
            {
                bitmap_clear_bit(vx_obj->x5, x); // Clear composite from x5
            }
        }

        // same for the 6x+1 candidate
        if (bitmap_get_bit(vx_obj->x7, x))
        {
            mpz_add_ui(x_p, vx_obj->yvx, x);
            iZ_mpz(p, x_p, 1); // Compute p = iZ(x_p, 1)
            int is_prime = mpz_probab_prime_p(p, r);
            vx_obj->p_test_ops++;

            if (is_prime)
            {
                vx_obj->p_count++;
            }
            else
            {
                bitmap_clear_bit(vx_obj->x7, x); // Clear composite from x7
            }
        }
    }
```

```c

    vx_obj->is_large_limit = 0; // all composites cleared

    // Cleanup
    mpz_clears(p, x_p, NULL);
}

VX_SEG *vx_init(IZM *iZm, int start_x, int end_x, char *y_str, int mr_rounds)
{
    // assert vx > 5 and not a multiple of 2 or 3
    assert(iZm && "iZm is NULL in vx_init");
    assert(y_str && "y_str is NULL in vx_init");

    VX_SEG *vx_obj = malloc(sizeof(VX_SEG));
    if (vx_obj == NULL)
    {
        log_error("Memory allocation failed in vx_init\n");
        return NULL;
    }

    // Initialize struct members
    // vx_obj->iZm = iZm;
    vx_obj->vx = iZm->vx;

    // Set base values
    if (!vx_set_base_values(vx_obj, y_str))
    {
        // check logs
        free(vx_obj);
        return NULL;
    }

    vx_obj->mr_rounds = (mr_rounds == 0) ? MR_ROUNDS : mr_rounds; // default 25 rounds

    int vx = vx_obj->vx;
    vx_obj->start_x = MAX(start_x, 1);
    vx_obj->end_x = MIN(end_x, vx);
    vx_obj->x5 = bitmap_clone(iZm->base_x5);
    vx_obj->x7 = bitmap_clone(iZm->base_x7);
    vx_obj->p_count = 0;
    vx_obj->p_gaps = NULL;
    vx_obj->bit_ops = 0;
    vx_obj->p_test_ops = 0;

    // perform deterministic sieving to prepare for probabilistic sieving or streaming
    vx_det_sieve(iZm, vx_obj);

    return vx_obj;
}

void vx_free(VX_SEG **vx_obj)
{
    if (vx_obj == NULL || *vx_obj == NULL)
        return;

    // clear bitmaps
    bitmap_free(&(*vx_obj)->x5);
    bitmap_free(&(*vx_obj)->x7);

    // clear p_gaps array
    ui16_free(&(*vx_obj)->p_gaps);

    // clear mpz_t variables
    mpz_clears((*vx_obj)->y, (*vx_obj)->yvx, (*vx_obj)->root_limit, NULL);

    free(*vx_obj);
    *vx_obj = NULL;
}

void vx_collect_p_gaps(VX_SEG *vx_obj)
{
    assert(vx_obj && vx_obj->p_count > 0 && "Invalid vx_obj in vx_collect_p_gaps");
    assert(mpz_cmp_ui(vx_obj->y, 0) > 0 && "First segment requires special handling in vx_collect_p_gaps");
    if (vx_obj->is_large_limit)
    {
        vx_full_sieve(vx_obj, 0);
    }

    vx_obj->p_gaps = ui16_init(vx_obj->p_count + 2);
    assert(vx_obj->p_gaps && "Memory allocation failed for vx_obj->p_gaps in vx_collect_p_gaps");

    // Initialize gap counter
    int gap = 0;

    // Iterate through x values in the range start_x <= x <= end_x
    for (int x = vx_obj->start_x; x <= vx_obj->end_x; x++)
    {
```

```
        // Increment gap by 4 since: iZ(x, -1) - iZ(x-1, 1) = 4
        gap += 4;

        // Check if iZ(vx * y + x, -1) is prime
        if (bitmap_get_bit(vx_obj->x5, x))
        {
            // Append gap to p_gaps
            ui16_push(vx_obj->p_gaps, gap);
            gap = 0; // Reset gap
        }

        // Increment gap by 2
        gap += 2;

        // Check if iZ(vx * y + x, 1) is prime
        if (bitmap_get_bit(vx_obj->x7, x))
        {
            // Append gap to p_gaps
            ui16_push(vx_obj->p_gaps, gap);
            gap = 0; // Reset gap
        }
    }

    // append final gap for backward calculations
    ui16_push(vx_obj->p_gaps, gap);
}

void vx_full_sieve(VX_SEG *vx_obj, int collect_p_gaps)
{
    assert(vx_obj && "vx_obj is NULL in vx_full_sieve");

    // If is_large_limit is true, perform probabilistic primality tests
    if (vx_obj->is_large_limit)
        vx_prob_sieve(vx_obj);

    // If collect_p_gaps is true, populate the p_gaps array
    if (collect_p_gaps)
        vx_collect_p_gaps(vx_obj);
}

void vx_stream_file(VX_SEG *vx_obj, FILE *output)
{
    assert(vx_obj && "vx_obj is NULL in vx_stream_file");
    assert(output && "output file is NULL in vx_stream_file");

    // Initialize GMP reusable variables p, x_p
    mpz_t p, x_p;
    mpz_init(p);
    mpz_init(x_p);

    int r = vx_obj->mr_rounds;

    // Iterate through x values in the range start_x <= x <= end_x
    for (int x = vx_obj->start_x; x <= vx_obj->end_x; x++)
    {
        // Check if iZ(vx * y + x, -1) is prime, if not, clear x in x5
        if (bitmap_get_bit(vx_obj->x5, x))
        {
            // Compute x_p = yvx + x
            mpz_add_ui(x_p, vx_obj->yvx, x);
            iZ_mpz(p, x_p, -1); // Compute p = iZ(x_p, -1)
            int is_prime = 1;

            if (vx_obj->is_large_limit)
            {
                vx_obj->p_test_ops++;
                is_prime = mpz_probab_prime_p(p, r);
            }

            if (is_prime)
            {
                if (vx_obj->is_large_limit)
                {
                    vx_obj->p_count++; // otherwise already counted in det_sieve
                }
                gmp_fprintf(output, "%Zd ", p);
            }
            else
            {
                bitmap_clear_bit(vx_obj->x5, x); // Clear composite from x5
            }
        }

        // test iZ(vx * y + x, 1) for primality
        if (bitmap_get_bit(vx_obj->x7, x))
        {
            mpz_add_ui(x_p, vx_obj->yvx, x);
```

```
            iZ_mpz(p, x_p, 1); // Compute p = iZ(x_p, 1)
            int is_prime = 1;

            if (vx_obj->is_large_limit)
            {
                vx_obj->p_test_ops++;
                is_prime = mpz_probab_prime_p(p, r);
            }

            if (is_prime)
            {
                if (vx_obj->is_large_limit)
                {
                    vx_obj->p_count++;
                }
                gmp_fprintf(output, "%Zd ", p);
            }
            else
            {
                bitmap_clear_bit(vx_obj->x7, x); // Clear composite from x7
            }
        }
    }

    mpz_clears(p, x_p, NULL);
}

// ==================================================
// * iZm Search primes routines:
// ==================================================

int vx_search_prime(mpz_t p, int m_id, int vx, int bit_size)
{
    // assert p is not NULL
    assert(p && "p cannot be NULL in vx_search_prime.");

    bit_size = MAX(bit_size, 10);

    // set m_id randomly if not provided
    if (m_id != -1 && m_id != 1)
        m_id = (rand() % 2) ? 1 : -1;

    gmp_randstate_t state;
    gmp_randinit_default(state);
    gmp_seed_randstate(state);

    UI64_ARRAY *root_primes = SiZm(vx);
    if (!root_primes)
    {
        log_error("Failed to initialize root primes in vx_search_prime");
        gmp_randclear(state);
        return 0;
    }

    mpz_t z, x_z, y, yvx;
    mpz_init(z);
    mpz_init(x_z);
    mpz_init(y);
    mpz_init(yvx);

    mpz_urandomb(y, state, bit_size);
    mpz_fdiv_q_ui(y, y, 6 * vx);
    mpz_mul_ui(yvx, y, vx); // compute yvx = vx * y

    int found = 0;
    while (!found)
    {
        BITMAP *bitmap = bitmap_init(vx + 10, 1);

        // sieve the bitmap with root primes skipping 2 and 3
        for (int i = 2; i < root_primes->count; i++)
        {
            uint64_t q = root_primes->array[i];
            // mark composites of q in the bitmap
            bitmap_clear_steps(bitmap, q, iZm_solve_for_xp_mpz(m_id, q, vx, y), vx);
        }

        int random_x = rand() % (vx / 2); // random x < vx/2
        for (int x = random_x; x < vx; x++)
        {
            // check if z is prime candidate in bitmap
            if (bitmap_get_bit(bitmap, x))
            {
                // compute x_z = vx * y + x
                mpz_add_ui(x_z, yvx, x);
                iZ_mpz(z, x_z, m_id);
                // check if z is prime
```

```
            found = mpz_probab_prime_p(z, MR_ROUNDS);

            // if z is prime, set p = z
            if (found)
            {
                mpz_set(p, z);
                break;
            }
        }
    }

    // cleanup
    bitmap_free(&bitmap);

    // increment y by 1 for next vx segment
    mpz_add_ui(y, y, 1);
    mpz_add_ui(yvx, yvx, vx);
}

ui64_free(&root_primes);
mpz_clears(z, x_z, y, yvx, NULL);
gmp_randclear(state);

return found;
}

int vy_search_prime(mpz_t p, int m_id, mpz_t vx)
{
    assert(p && vx && "Input parameters cannot be NULL");

    int found = 0; // flag to indicate if a prime was found

    // set m_id randomly if not provided
    if (m_id != -1 && m_id != 1)
        m_id = (rand() % 2) ? 1 : -1;

    gmp_randstate_t state;
    gmp_randinit_default(state);
    gmp_seed_randstate(state); // seed random state

    mpz_t z, g;
    mpz_init(z);
    mpz_init(g);

    // set random x value in the range of vx
    mpz_urandomm(z, state, vx);
    // compute z = 6 * z + i
    iZ_mpz(z, z, m_id);

    // search for x value such that gcd(iZ(x, m_id), z) = 1
    for (;;)
    {
        // increment z by 6 to advance x by 1
        mpz_add_ui(z, z, 6);

        // Compute g = gcd(vx, z)
        mpz_gcd(g, vx, z);

        // break if g = 1,
        // implying current x value can yield primes of the form iZ(x + vx * y, p_id)
        if (mpz_cmp_ui(g, 1) == 0)
            break;
    }

    // set g = 6 * vx to use as a step
    mpz_mul_ui(g, vx, 6);
    // add z by rand * g
    int rand_steps = rand() % 100;
    mpz_addmul_ui(z, g, rand_steps);

    while (!found)
    {
        // increment z by 6 * vx to advance y by 1
        mpz_add(z, z, g);

        // check if z is prime
        found = mpz_probab_prime_p(z, MR_ROUNDS);

        // if z is prime, set p = z
        if (found)
            mpz_set(p, z);
    }

    // cleanup
    gmp_randclear(state);
    mpz_clears(z, g, NULL);

```

```
    return found;
}
```

8.25 src/toolkit/logger.c File Reference

Logging module.

8.25.1 Detailed Description

Logging module.

This module provides functions for logging messages at different levels.

Definition in file logger.c.

8.26 logger.c

Go to the documentation of this file.

```

#ifdef ENABLE_LOGGING

#include "logger.h"
#include <stdio.h>
#include <time.h>
#include <stdarg.h>
#include <string.h>
#include <sys/stat.h>
#include <pthread.h>
#include <stdlib.h>
#include <errno.h>

// Initialize the mutex for thread-safe logging
static pthread_mutex_t log_mutex = PTHREAD_MUTEX_INITIALIZER;
static LogLevel current_log_level = LOG_DEBUG; // Default log level

const char *log_level_to_string(LogLevel level)
{
    switch (level)
    {
    case LOG_DEBUG:
        return "DEBUG";
    case LOG_INFO:
        return "INFO";
    case LOG_WARNING:
        return "WARNING";
    case LOG_ERROR:
        return "ERROR";
    case LOG_FATAL:
        return "FATAL";
    default:
        return "UNKNOWN";
    }
}

void get_current_timestamp(char *buffer, size_t size)
{
    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    strftime(buffer, size, "%Y-%m-%d %H:%M:%S", t);
}

void log_rotate(const char *log_file, size_t max_size)
{
    struct stat st;
    if (stat(log_file, &st) == 0 && st.st_size >= (off_t)max_size) // Cast max_size to off_t to match types
    {
        char old_log_file[256];
        for (int i = 5; i > 0; --i)
        {
```

```
            snprintf(old_log_file, sizeof(old_log_file), "%s.%d", log_file, i);
            if (i == 5)
            {
                remove(old_log_file); // Remove the oldest log file
            }
            else
            {
                char prev_log_file[256];
                snprintf(prev_log_file, sizeof(prev_log_file), "%s.%d", log_file, i + 1);
                rename(old_log_file, prev_log_file);
            }
        }

        char new_log_file[256];
        snprintf(new_log_file, sizeof(new_log_file), "%s.1", log_file);
        rename(log_file, new_log_file);
    }
}

void log_init(const char *log_file)
{
    struct stat st = {0};
    if (stat(LOG_DIR, &st) == -1)
    {
        mkdir(LOG_DIR, 0700); // Create log directory if it doesn't exist
    }

    log_rotate(log_file, LOG_MAX_SIZE); // Rotate logs if needed
}

void log_shutdown(void)
{
    pthread_mutex_destroy(&log_mutex);
}

void log_set_log_level(LogLevel level)
{
    current_log_level = level;
}

static void log_message_va(LogLevel level, const char *format, va_list args)
{
    if (level < current_log_level)
        return; // Don't log messages below the current log level

    char message[1024];
    // Format string comes from a variadic function parameter - suppress warning
#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wformat-nonliteral"
    vsnprintf(message, sizeof(message), format, args);
#pragma clang diagnostic pop

    char timestamp[20];
    get_current_timestamp(timestamp, sizeof(timestamp));

    pthread_mutex_lock(&log_mutex);

    FILE *file = fopen(LOG_FILE, "a");
    if (!file)
    {
        perror("Failed to open log file");
        pthread_mutex_unlock(&log_mutex); // Ensure mutex is unlocked
        return;
    }

    fprintf(file, "[%s] [%s] %s\n", timestamp, log_level_to_string(level), message);
    fclose(file);

    pthread_mutex_unlock(&log_mutex);
}

void log_message(LogLevel level, const char *format, ...)
{
    va_list args;
    va_start(args, format);
    log_message_va(level, format, args);
    va_end(args);
}

void log_message_extended(LogLevel level, const char *file_name, int line_number, const char *format, ...)
{
    if (level < current_log_level)
        return; // Don't log messages below the current log level

    va_list args;
    va_start(args, format);
    char message[1024];
```

```c
    // Format string comes from a variadic function parameter - suppress warning
#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wformat-nonliteral"
    vsnprintf(message, sizeof(message), format, args);
#pragma clang diagnostic pop
    va_end(args);

    char timestamp[20];
    get_current_timestamp(timestamp, sizeof(timestamp));

    pthread_mutex_lock(&log_mutex);

    FILE *file = fopen(LOG_FILE, "a");
    if (!file)
    {
        perror("Failed to open log file");
        pthread_mutex_unlock(&log_mutex);
        return;
    }

    fprintf(file, "[%s] [%s] %s (File: %s, Line: %d)\n",
            timestamp, log_level_to_string(level), message, file_name, line_number);
    fclose(file);

    pthread_mutex_unlock(&log_mutex);
}

void log_console(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    char message[1024];
    // Format string comes from a variadic function parameter - suppress warning
#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wformat-nonliteral"
    vsnprintf(message, sizeof(message), format, args);
#pragma clang diagnostic pop
    va_end(args);

    char timestamp[20];
    get_current_timestamp(timestamp, sizeof(timestamp));

    // Print the message with a timestamp to the console
    printf("[%s] %s\n", timestamp, message);
}

void log_test(int result, int test_id, const char *unit_name, const char *format, ...)
{
    va_list args;
    va_start(args, format);
    char message[1024];
    // Format string comes from a variadic function parameter - suppress warning
#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wformat-nonliteral"
    vsnprintf(message, sizeof(message), format, args);
#pragma clang diagnostic pop
    va_end(args);

    if (result)
    {
        printf("[%02d] %-30s [PASS] %s\n", test_id, unit_name, message);
    }
    else
    {
        printf("[%02d] %-30s [FAIL] %s\n", test_id, unit_name, message);
    }
}

static void print_line(int length, char fill_char)
{
    // if empty fill_char, use '-'
    if (fill_char == '\0')
        fill_char = '-';

    printf("\n");
    for (int i = 0; i < length; i++)
        printf("%c", fill_char);
    printf("\n");
}

void log_test_summary(char *module_name, int passed, int failed, int verbose)
{
    print_line(60, '*');
    printf("Results Summary for %s", module_name);
    print_line(60, '-');
    printf("%-32s: %d\n", "Total Tests", passed + failed);
    printf("%-32s: %d\n", "Passed", passed);
```

```
    printf("%-32s: %d", "Failed", failed);
    print_line(60, '-');
    if (failed == 0)
    {
        printf("[SUCCESS] ALL %s TESTS PASSED!", module_name);
    }
    else
    {
        printf("[FAILURE] SOME %s TESTS FAILED :\\", module_name);
        if (!verbose)
            printf("Run with verbose=1 for detailed output.");
    }
    print_line(60, '*');
}

void log_debug(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    log_message_va(LOG_DEBUG, format, args);
    va_end(args);
}

void log_info(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    log_message_va(LOG_INFO, format, args);
    va_end(args);
}

void log_warn(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    log_message_va(LOG_WARNING, format, args);
    va_end(args);
}

void log_error(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    log_message_va(LOG_ERROR, format, args);
    va_end(args);
}

void log_fatal(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    log_message_va(LOG_FATAL, format, args);
    va_end(args);
}

#endif // ENABLE_LOGGING
```

8.27 src/toolkit/utils.c File Reference

Implementations for the shared utility layer.

#include <utils.h>

Functions

- int is_numeric_str (const char *str)

  Check if a string is numeric.
- void print_line (int length, char fill_char)

  Print a single line composed of repeated characters.
- void print_centered_text (const char *text, int line_length, char fill_char)

  Print text centered inside a filled line.

- void print_sha256_hash (const unsigned char *hash)

  Print the SHA-256 hash in hexadecimal format.
- int create_dir (const char *dir)

  Create a directory if it does not exist.
- uint64_t gcd (uint64_t a, uint64_t b)

  Compute the greatest common divisor (GCD) of two integers.
- uint64_t modular_inverse (uint64_t a, uint64_t m)

  Compute the modular inverse of a modulo m.
- void gmp_seed_randstate (gmp_randstate_t state)

  Seed the GMP random state.
- int get_cpu_cores_count (void)

  Compute the number of CPU cores available.
- int get_cpu_L2_cache_size_bits (void)

  Get the CPU L2 cache size in kilobytes.
- void print_module_header (char *module_name)

  Print module-level banner for test output.
- void print_test_header (void)

  Print standard test-table column headers.

8.27.1 Detailed Description

Implementations for the shared utility layer.

Definition in file utils.c.

8.28 utils.c

Go to the documentation of this file.

```
00001
00006
00007 #include <utils.h>
00008
00015 int is_numeric_str(const char *str)
00016 {
00017     if (str == NULL || *str == '\0')
00018         return 0;
00019
00020     while (*str)
00021     {
00022         if (*str < '0' || *str > '9')
00023             return 0;
00024         ++str;
00025     }
00026     return 1;
00027 }
00028
00034 void print_line(int length, char fill_char)
00035 {
00036     // if empty fill_char, use '-'
00037     if (fill_char == '\0')
00038         fill_char = '-';
00039
00040     for (int i = 0; i < length; i++)
00041         printf("%c", fill_char);
00042     printf("\n");
00043 }
00044
00051 void print_centered_text(const char *text, int line_length, char fill_char)
00052 {
00053     int text_length = strlen(text);
00054     if (text_length >= line_length)
00055     {
00056         printf("%s\n", text);
```

```
        return;
    }

    int padding = (line_length - text_length + 1) / 2;

    for (int i = 0; i < padding; i++)
        printf("%c", fill_char);
    printf("%s", text);
    for (int i = 0; i < padding; i++)
        printf("%c", fill_char);
    printf("\n");
}

void print_sha256_hash(const unsigned char *hash)
{
    for (int i = 0; i < SHA256_DIGEST_LENGTH; i++)
        printf("%02x", hash[i]);

    printf("\n");
}

int create_dir(const char *dir)
{
    struct stat st = {0};
    if (stat(dir, &st) == -1)
    {
        if (mkdir(dir, 0700) != 0)
        {
            // log error if mkdir fails
            log_error("mkdir failed to create %s :\\", dir);
            return -1;
        }
    }

    return 0; // Directory already exists or was successfully created
}

uint64_t gcd(uint64_t a, uint64_t b)
{
    while (b != 0)
    {
        uint64_t temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

uint64_t modular_inverse(uint64_t a, uint64_t m)
{
    if (m == 1)
        return 0;

    uint64_t m0 = m;
    int64_t x0 = 0, x1 = 1;

    while (a > 1)
    {
        uint64_t q = a / m;
        uint64_t t = m;

        m = a % m;
        a = t;
        t = x0;

        x0 = x1 - (int64_t)q * x0;
        x1 = t;
    }

    if (x1 < 0)
        x1 += (int64_t)m0;
    return (uint64_t)x1;
}

void gmp_seed_randstate(gmp_randstate_t state)
{
    unsigned long seed;
    int random_fd = open("/dev/urandom", O_RDONLY); // Open /dev/urandom for reading
    if (random_fd == -1)
    {
        // Fallback if /dev/urandom cannot be opened
        seed = (unsigned long)time(NULL);
    }
    else
    {
        read(random_fd, &seed, sizeof(seed)); // Read random bytes from /dev/urandom
        close(random_fd);
```

```
    }

    gmp_randseed_ui(state, seed); // Seed the state with the random value
}

int get_cpu_cores_count(void)
{
    int cores = sysconf(_SC_NPROCESSORS_ONLN);
    if (cores < 1)
        cores = 1; // Fallback to 1 if unable to determine
    return cores;
}

int get_cpu_L2_cache_size_bits(void)
{
    int size_kb = 0;

#ifdef ___linux___
    // Linux: Try reading from sysfs
    FILE *fp = fopen("/sys/devices/system/cpu/cpu0/cache/index2/size", "r");
    if (fp != NULL)
    {
        char buffer[32];
        if (fgets(buffer, sizeof(buffer), fp) != NULL)
        {
            // Parse size with K suffix (e.g., "256K")
            if (sscanf(buffer, "%dK", &size_kb) == 1)
            {
                fclose(fp);
                return size_kb * 1024 * 8; // Convert KB to bits
            }
        }
        fclose(fp);
    }
#endif

#if defined(___APPLE___) || defined(___FreeBSD___) || defined(___OpenBSD___) || defined(___NetBSD___)
    // macOS and BSD: Use sysctl
    size_t size_bytes = 0;
    size_t len = sizeof(size_bytes);

    // Try hw.l2cachesize (macOS/some BSDs)
    if (sysctlbyname("hw.l2cachesize", &size_bytes, &len, NULL, 0) == 0)
    {
        if (size_bytes > 0)
        {
            return (int)(size_bytes * 8); // Convert bytes to bits
        }
    }

    // Fallback: Try machdep.cpu.cache.L2_cache_size (older macOS)
    if (sysctlbyname("machdep.cpu.cache.L2_cache_size", &size_bytes, &len, NULL, 0) == 0)
    {
        if (size_bytes > 0)
        {
            return (int)(size_bytes * 8); // Convert bytes to bits
        }
    }
#endif

    // Conservative fallback: 256 KB is common for L2 cache per core
    return 256 * 1024 * 8; // Convert 256 KB to bits
}

void print_module_header(char *module_name)
{
    print_line(60, '*');
    printf("* %s MODULE TEST SUITE\n", module_name);
    print_line(60, '*');
}

void print_test_header(void)
{
    print_line(92, '-');
    printf("[%s] %-30s %s %-66s\n", "ID", "Unit Name", "Result", "Details");
    print_line(92, '-');
}
```

Chapter 9

Examples

9.1 /Users/hazemmounir/Workspace/iZproject/iZprime/src/toolkit/↩ bitmap.c

Clears bits at regular intervals, typically used in sieve algorithms.

Clears bits at regular intervals, typically used in sieve algorithms.Starting from start_idx, this function clears every step-th bit up to the specified limit (inclusive). This is commonly used in prime sieve algorithms where you need to mark all multiples of a number as composite.

The function includes:

- Automatic capping of limit to bitmap size
- Integer overflow protection in the loop
- Early termination to prevent infinite loops

Parameters

bitmap	Pointer to the bitmap to modify
step	Interval between bits to clear (must be > 0, typically a prime)
start_idx	Starting bit index (inclusive, typically first multiple)
limit	Upper bound for clearing (inclusive, auto-capped to size-1)

Note

If limit > bitmap->size, it is automatically reduced to bitmap->size - 1

No operation if bitmap is NULL, step is 0, or start_idx > limit

Includes overflow prevention: stops if next index would overflow

// For Sieve of Eratosthenes: mark multiples of 3 starting from 9 bitmap_clear_steps(bitmap, 3, 9, 1000); // Clears 9, 12, 15, 18, ...

```
#include <bitmap.h>

#ifdef __aarch64__
#include <arm_neon.h>
#elif defined(__x86_64__) || defined(_M_X64)
#include <immintrin.h>
#endif

BITMAP *bitmap_init(size_t size, int set_bits)
{
    assert(size > 0 && "Bitmap size must be positive");

    BITMAP *bitmap = (BITMAP *)malloc(sizeof(BITMAP));
    if (bitmap == NULL)
    {
        log_error("Memory allocation failed for BITMAP struct");
        return NULL;
    }

    bitmap->size = size;
    bitmap->byte_size = (size + 7) / 8;
    bitmap->data = (unsigned char *)malloc(bitmap->byte_size);
    if (bitmap->data == NULL)
    {
        free(bitmap);
        log_error("bitmap_create: Memory allocation failed for BITMAP");
        return NULL;
    }

    if (set_bits)
        memset(bitmap->data, 0xFF, bitmap->byte_size);
    else
        memset(bitmap->data, 0x00, bitmap->byte_size);

    memset(bitmap->sha256, 0, SHA256_DIGEST_LENGTH); // Initialize SHA-256 to zero

    return bitmap;
}

void bitmap_free(BITMAP **bitmap)
{
    // Safe guard against double free and NULL pointer
    if (bitmap && *bitmap)
    {
        if ((*bitmap)->data)
        {
            free((*bitmap)->data);
            (*bitmap)->data = NULL;
        }

        free(*bitmap);
        *bitmap = NULL;
    }
}

void bitmap_set_bit(BITMAP *bitmap, size_t idx)
{
    bitmap->data[idx / 8] |= (1 « (idx % 8));
}

int bitmap_get_bit(BITMAP *bitmap, size_t idx)
{
    return (bitmap->data[idx / 8] & (1 « (idx % 8))) != 0;
}

void bitmap_flip_bit(BITMAP *bitmap, size_t idx)
{
    bitmap->data[idx / 8] ^= (1 « (idx % 8));
}

void bitmap_clear_bit(BITMAP *bitmap, size_t idx)
{
    // Clear the bit by ANDing with the negation of the bit mask
    bitmap->data[idx / 8] &= ~(1 « (idx % 8));
}

void bitmap_set_all(BITMAP *bitmap)
{
    memset(bitmap->data, 0xFF, (bitmap->size + 7) / 8);
}

void bitmap_clear_all(BITMAP *bitmap)
```

```
{
    memset(bitmap->data, 0x00, (bitmap->size + 7) / 8);
}

void bitmap_clear_steps(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
{
    assert(step > 0 && "Step must be positive in bitmap_clear_steps.");
    limit = MIN(limit, bitmap->size - 1);

    for (uint64_t idx = start_idx; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }
}

void bitmap_clear_steps_simd(BITMAP *bitmap, uint64_t step, uint64_t start_idx, uint64_t limit)
{
    assert(step > 0 && "Step must be positive in bitmap_clear_steps_simd.");
    limit = MIN(limit, bitmap->size - 1);

#ifdef __aarch64__
    // NEON implementation for ARM64
    uint64_t idx = start_idx;

    // Process 4 steps at a time
    // Condition: idx + 3*step <= limit

    if (limit >= 3 * step && idx <= limit - 3 * step)
    {
        uint64x2_t v_step2 = vdupq_n_u64(2 * step);
        uint64x2_t v_step4 = vdupq_n_u64(4 * step);

        // Initialize indices: [idx, idx+step] and [idx+2step, idx+3step]
        uint64x2_t v_idx_01 = vdupq_n_u64(idx);
        v_idx_01 = vsetq_lane_u64(idx + step, v_idx_01, 1);

        uint64x2_t v_idx_23 = vaddq_u64(v_idx_01, v_step2);

        while (idx <= limit - 3 * step)
        {
            // Extract indices and perform operations
            // Note: NEON doesn't have efficient scatter for bytes, so we extract and do scalar stores

            uint64_t i0 = vgetq_lane_u64(v_idx_01, 0);
            uint64_t i1 = vgetq_lane_u64(v_idx_01, 1);
            uint64_t i2 = vgetq_lane_u64(v_idx_23, 0);
            uint64_t i3 = vgetq_lane_u64(v_idx_23, 1);

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));
            bitmap->data[i2 / 8] &= ~(1 « (i2 % 8));
            bitmap->data[i3 / 8] &= ~(1 « (i3 % 8));

            // Advance indices
            v_idx_01 = vaddq_u64(v_idx_01, v_step4);
            v_idx_23 = vaddq_u64(v_idx_23, v_step4);

            idx += 4 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }

#elif defined(__AVX2__)
    // AVX2 implementation for x86_64
    uint64_t idx = start_idx;

    // Process 4 steps at a time
    if (limit >= 3 * step && idx <= limit - 3 * step)
    {
        __m256i v_step4 = _mm256_set1_epi64x(4 * step);

        // Initialize indices: [idx, idx+step, idx+2step, idx+3step]
        // Note: _mm256_set_epi64x takes arguments in reverse order (e3, e2, e1, e0)
        __m256i v_idx = _mm256_set_epi64x(idx + 3 * step, idx + 2 * step, idx + step, idx);

        while (idx <= limit - 3 * step)
        {
            // Extract indices
            uint64_t i0 = _mm256_extract_epi64(v_idx, 0);
            uint64_t i1 = _mm256_extract_epi64(v_idx, 1);
            uint64_t i2 = _mm256_extract_epi64(v_idx, 2);
```

```
            uint64_t i3 = __mm256_extract_epi64(v_idx, 3);

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));
            bitmap->data[i2 / 8] &= ~(1 « (i2 % 8));
            bitmap->data[i3 / 8] &= ~(1 « (i3 % 8));

            // Advance indices
            v_idx = __mm256_add_epi64(v_idx, v_step4);
            idx += 4 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }

#elif defined(__SSE2__)
    // SSE2 implementation for x86_64
    uint64_t idx = start_idx;

    // Process 2 steps at a time
    if (limit >= step && idx <= limit - step)
    {
        __m128i v_step2 = __mm_set1_epi64x(2 * step);
        // Note: _mm_set_epi64x takes arguments in reverse order (e1, e0)
        __m128i v_idx = __mm_set_epi64x(idx + step, idx);

        while (idx <= limit - step)
        {
            // Extract indices
            // _mm_cvtsi128_si64 extracts the lower 64 bits (element 0)
            uint64_t i0 = __mm_cvtsi128_si64(v_idx);
            // Unpack high to low to extract element 1
            uint64_t i1 = __mm_cvtsi128_si64(_mm_unpackhi_epi64(v_idx, v_idx));

            bitmap->data[i0 / 8] &= ~(1 « (i0 % 8));
            bitmap->data[i1 / 8] &= ~(1 « (i1 % 8));

            // Advance indices
            v_idx = __mm_add_epi64(v_idx, v_step2);
            idx += 2 * step;
        }
    }

    // Handle remaining steps
    for (; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }

#else
    // Fallback to scalar implementation
    for (uint64_t idx = start_idx; idx <= limit; idx += step)
    {
        bitmap->data[idx / 8] &= ~(1 « (idx % 8));
    }
#endif
}

BITMAP *bitmap_clone(BITMAP *src)
{
    assert(src && src->data && "Invalid source bitmap passed to bitmap_clone.");

    BITMAP *dest = bitmap_init(src->size, 0);
    memcpy(dest->data, src->data, src->byte_size);
    memcpy(dest->sha256, src->sha256, SHA256_DIGEST_LENGTH);

    return dest;
}

void bitmap_compute_hash(BITMAP *bitmap)
{
    assert(bitmap && bitmap->data && "Invalid bitmap passed to bitmap_compute_hash.");

    // Generate SHA-256 hash and store it in the struct
    SHA256((unsigned char *)bitmap->data, bitmap->byte_size, bitmap->sha256);
}

int bitmap_validate_hash(BITMAP *bitmap)
{
```

```
    assert(bitmap && bitmap->data && "Invalid bitmap");

    unsigned char correct_hash[SHA256_DIGEST_LENGTH]; // Buffer to hold the computed hash

    // Generate SHA-256 hash and store it in correct_hash
    SHA256((unsigned char *)bitmap->data, bitmap->byte_size, correct_hash);

    // Compare actual_hash with the stored hash in bitmap->sha256
    if (memcmp(correct_hash, bitmap->sha256, SHA256_DIGEST_LENGTH) == 0)
    {
        return 1; // SHA-256 match
    }
    else
    {
        log_error("SHA-256 checksum validation failed.");
        return 0; // SHA-256 mismatch
    }
}

int bitmap_fwrite(BITMAP *bitmap, FILE *file)
{
    assert(bitmap && bitmap->data && "Invalid bitmap passed to bitmap_fwrite.");
    assert(file && "File pointer is NULL in bitmap_fwrite.");

    // Write size
    if (fwrite(&bitmap->size, sizeof(size_t), 1, file) != 1)
    {
        log_error("Failed to write bitmap size to file.");
        return 0;
    }

    // Write data
    if (fwrite(bitmap->data, sizeof(unsigned char), bitmap->byte_size, file) != bitmap->byte_size)
    {
        log_error("Failed to write bitmap data to file.");
        return 0;
    }

    // Compute SHA-256 hash if not already computed
    // Check if hash is all zeros by examining all bytes
    int hash_is_zero = 1;
    for (size_t i = 0; i < SHA256_DIGEST_LENGTH; i++)
    {
        if (bitmap->sha256[i] != 0)
        {
            hash_is_zero = 0;
            break;
        }
    }

    if (hash_is_zero)
    {
        bitmap_compute_hash(bitmap);
    }

    // Write SHA-256 hash
    if (fwrite(bitmap->sha256, 1, SHA256_DIGEST_LENGTH, file) != SHA256_DIGEST_LENGTH)
    {
        log_error("Failed to write SHA-256 hash to file.");
        return 0;
    }

    return 1; // Success
}

BITMAP *bitmap_fread(FILE *file)
{
    assert(file && "File pointer is NULL in bitmap_fread.");

    // Read size
    size_t size;
    if (fread(&size, sizeof(size_t), 1, file) != 1)
    {
        log_error("Failed to read bitmap size from file.");
        return NULL;
    }

    BITMAP *bitmap = bitmap_init(size, 0);
    if (bitmap == NULL)
    {
        log_error("Failed to initialize bitmap.");
        return NULL;
    }

    // Read data directly into the already-allocated buffer
    if (fread(bitmap->data, sizeof(unsigned char), bitmap->byte_size, file) != bitmap->byte_size)
    {
```

```
        log_error("Failed to read complete bitmap data from file.");
        bitmap_free(&bitmap);
        return NULL;
    }

    // Read SHA-256 hash
    if (fread(bitmap->sha256, 1, SHA256_DIGEST_LENGTH, file) != SHA256_DIGEST_LENGTH)
    {
        log_error("Failed to read SHA-256 hash from file.");
        bitmap_free(&bitmap);
        return NULL;
    }

    if (!bitmap_validate_hash(bitmap))
    {
        log_error("SHA-256 hash validation failed.");
        bitmap_free(&bitmap);
        return NULL;
    }

    return bitmap;
}
```

Index

array
    UI16_ARRAY, 90
    UI32_ARRAY, 91
    UI64_ARRAY, 92

base
    SIEVE_LIMIT, 88
base_x5
    IZM, 87
base_x7
    IZM, 87
bit_ops
    VX_SEG, 93
BITMAP, 83
    byte_size, 83
    data, 83
    sha256, 84
    size, 84
Bitmap Module, 19
    bitmap_clear_all, 21
    bitmap_clear_bit, 21
    bitmap_clear_steps, 21
    bitmap_clear_steps_simd, 22
    bitmap_clone, 23
    bitmap_compute_hash, 24
    bitmap_flip_bit, 24
    bitmap_fread, 25
    bitmap_free, 25
    bitmap_fwrite, 26
    bitmap_get_bit, 27
    bitmap_init, 28
    bitmap_set_all, 29
    bitmap_set_bit, 29
    bitmap_validate_hash, 30
    TEST_BITMAP, 30
bitmap_clear_all
    Bitmap Module, 21
bitmap_clear_bit
    Bitmap Module, 21
bitmap_clear_steps
    Bitmap Module, 21
bitmap_clear_steps_simd
    Bitmap Module, 22
bitmap_clone
    Bitmap Module, 23
bitmap_compute_hash
    Bitmap Module, 24
bitmap_flip_bit
    Bitmap Module, 24
bitmap_fread
    Bitmap Module, 25
bitmap_free
    Bitmap Module, 25
bitmap_fwrite
    Bitmap Module, 26
bitmap_get_bit
    Bitmap Module, 27
bitmap_init
    Bitmap Module, 28
bitmap_set_all
    Bitmap Module, 29
bitmap_set_bit
    Bitmap Module, 29
bitmap_validate_hash
    Bitmap Module, 30
byte_size
    BITMAP, 83

capacity
    UI16_ARRAY, 90
    UI32_ARRAY, 91
    UI64_ARRAY, 92
compute_k_vx
    iZ_toolkit.c, 143
compute_l2_vx
    Toolkit (iZ/iZm), 51
compute_max_vx
    Toolkit (iZ/iZm), 51
compute_vx_k
    Toolkit (iZ/iZm), 52
count
    UI16_ARRAY, 90
    UI32_ARRAY, 91
    UI64_ARRAY, 92
create_dir
    Utilities, 76

data
    BITMAP, 83
DIR_output
    Utilities, 75

end_x
    VX_SEG, 93
est_pi_n
    prime_sieve.c, 126
exp
    SIEVE_LIMIT, 88

filepath

INPUT_SIEVE_RANGE, 85
function
SIEVE_MODEL, 89

gcd
Utilities, 76
get_cpu_cores_count
Utilities, 76
get_cpu_L2_cache_size_bits
Utilities, 77
get_root_primes
Toolkit (iZ/iZm), 52
gmp_seed_randstate
Utilities, 77

include/bitmap.h, 97, 98
include/int_arrays.h, 99, 101
include/iZ_api.h, 102, 104
include/iZ_toolkit.h, 104, 106
include/logger.h, 107, 109
include/test_api.h, 110, 111
include/utils.h, 112, 113
INPUT_SIEVE_RANGE, 85
filepath, 85
mr_rounds, 85
range, 85
start, 86
Integer Arrays, 31
TEST_GENERIC_INT_ARRAYS, 33
is_large_limit
VX_SEG, 94
is_numeric_str
Utilities, 78
iZ
Toolkit (iZ/iZm), 53
iZ Public API, 34
iZ_next_prime, 36
N_LIMIT, 35
SiZ, 37
SiZ_count, 38
SiZ_stream, 39
SiZm, 39
SiZm_vy, 40
SoA, 41
SoE, 42
SoEu, 43
SoS, 44
SSoE, 44
vx_random_prime, 45
vy_random_prime, 46
iZ_mpz
Toolkit (iZ/iZm), 53
iZ_next_prime
iZ Public API, 36
iZ_toolkit.c
compute_k_vx, 143
s_primes, 144
s_primes_count, 144
vx_det_sieve, 143
vx_prob_sieve, 143
vx_set_base_values, 144
IZM, 86
base_x5, 87
base_x7, 87
k_vx, 87
root_primes, 87
vx, 87
iZm_clone
Toolkit (iZ/iZm), 54
iZm_construct_vx_base
Toolkit (iZ/iZm), 54
iZm_free
Toolkit (iZ/iZm), 55
iZm_init
Toolkit (iZ/iZm), 55
iZm_solve_for_xp
Toolkit (iZ/iZm), 56
iZm_solve_for_xp_mpz
Toolkit (iZ/iZm), 57
iZm_solve_for_yp
Toolkit (iZ/iZm), 59
iZprime, 1
iZprime User Manual, 11

k_vx
IZM, 87

log_console
Logging, 68
LOG_DEBUG
Logging, 67
log_debug
Logging, 68
LOG_DIR
Logging, 66
LOG_ERROR
Logging, 67
log_error
Logging, 68
LOG_FATAL
Logging, 67
log_fatal
Logging, 68
LOG_FILE
Logging, 66
LOG_INFO
Logging, 67
log_info
Logging, 69
log_init
Logging, 69
log_level_to_string
Logging, 69
LOG_MAX_SIZE
Logging, 67
log_message
Logging, 70
log_message_extended

Logging, 70
log_set_log_level
Logging, 71
log_test
Logging, 71
log_test_summary
Logging, 71
log_warn
Logging, 73
LOG_WARNING
Logging, 67
LOGGER_FORMAT_PRINTF
Logging, 67
Logging, 65
log_console, 68
LOG_DEBUG, 67
log_debug, 68
LOG_DIR, 66
LOG_ERROR, 67
log_error, 68
LOG_FATAL, 67
log_fatal, 68
LOG_FILE, 66
LOG_INFO, 67
log_info, 69
log_init, 69
log_level_to_string, 69
LOG_MAX_SIZE, 67
log_message, 70
log_message_extended, 70
log_set_log_level, 71
log_test, 71
log_test_summary, 71
log_warn, 73
LOG_WARNING, 67
LOGGER_FORMAT_PRINTF, 67
LogLevel, 67
LogLevel
Logging, 67

MAX_CORES
Utilities, 75
modular_inverse
Utilities, 78
MR_ROUNDS
Toolkit (iZ/iZm), 49
mr_rounds
INPUT_SIEVE_RANGE, 85
VX_SEG, 94

N_LIMIT
iZ Public API, 35
name
SIEVE_MODEL, 89

p_count
VX_SEG, 94
p_gaps
VX_SEG, 94
p_test_ops
VX_SEG, 94
prime_sieve.c
est_pi_n, 126
process_eratosthenes_bitmap, 126
print_centered_text
Utilities, 79
print_line
Utilities, 79
print_module_header
Utilities, 81
print_sha256_hash
Utilities, 81
print_test_header
Utilities, 81
process_eratosthenes_bitmap
prime_sieve.c, 126
process_iZ_bitmaps
Toolkit (iZ/iZm), 60

range
INPUT_SIEVE_RANGE, 85
root_limit
VX_SEG, 95
root_primes
IZM, 87

s_primes
iZ_toolkit.c, 144
s_primes_count
iZ_toolkit.c, 144
sha256
BITMAP, 84
UI16_ARRAY, 90
UI32_ARRAY, 91
UI64_ARRAY, 92
SIEVE_FN
Tests and Benchmarks, 74
SIEVE_LIMIT, 88
base, 88
exp, 88
SIEVE_MODEL, 88
function, 89
name, 89
SiZ
iZ Public API, 37
SiZ_count
iZ Public API, 38
SiZ_stream
iZ Public API, 39
size
BITMAP, 84
SiZm
iZ Public API, 39
SiZm_vy
iZ Public API, 40
SoA
iZ Public API, 41
SoE

iZ Public API, 42
SoEu
iZ Public API, 43
SoS
iZ Public API, 44
src/iZ_apps.c, 114, 115
src/prime_sieve.c, 125, 127
src/toolkit/bitmap.c, 133, 135
src/toolkit/int_arrays.c, 139, 141
src/toolkit/iZ_toolkit.c, 141, 145
src/toolkit/logger.c, 155
src/toolkit/utils.c, 158, 159
SSoE
iZ Public API, 44
start
INPUT_SIEVE_RANGE, 86
start_x
VX_SEG, 95

TEST_BITMAP
Bitmap Module, 30
TEST_GENERIC_INT_ARRAYS
Integer Arrays, 33
Tests and Benchmarks, 73
SIEVE_FN, 74
Toolkit (iZ/iZm), 47
compute_l2_vx, 51
compute_max_vx, 51
compute_vx_k, 52
get_root_primes, 52
iZ, 53
iZ_mpz, 53
iZm_clone, 54
iZm_construct_vx_base, 54
iZm_free, 55
iZm_init, 55
iZm_solve_for_xp, 56
iZm_solve_for_xp_mpz, 57
iZm_solve_for_yp, 59
MR_ROUNDS, 49
process_iZ_bitmaps, 60
VX2, 49
VX3, 49
VX4, 49
VX5, 50
VX6, 50
VX7, 50
VX8, 50
vx_collect_p_gaps, 60
vx_free, 61
vx_full_sieve, 61
vx_init, 62
vx_search_prime, 63
vx_stream_file, 64
vy_search_prime, 64

UI16_ARRAY, 89
array, 90
capacity, 90
count, 90
sha256, 90
UI32_ARRAY, 90
array, 91
capacity, 91
count, 91
sha256, 91
UI64_ARRAY, 91
array, 92
capacity, 92
count, 92
sha256, 92
Utilities, 74
create_dir, 76
DIR_output, 75
gcd, 76
get_cpu_cores_count, 76
get_cpu_L2_cache_size_bits, 77
gmp_seed_randstate, 77
is_numeric_str, 78
MAX_CORES, 75
modular_inverse, 78
print_centered_text, 79
print_line, 79
print_module_header, 81
print_sha256_hash, 81
print_test_header, 81

vx
IZM, 87
VX_SEG, 95
VX2
Toolkit (iZ/iZm), 49
VX3
Toolkit (iZ/iZm), 49
VX4
Toolkit (iZ/iZm), 49
VX5
Toolkit (iZ/iZm), 50
VX6
Toolkit (iZ/iZm), 50
VX7
Toolkit (iZ/iZm), 50
VX8
Toolkit (iZ/iZm), 50
vx_collect_p_gaps
Toolkit (iZ/iZm), 60
vx_det_sieve
iZ_toolkit.c, 143
vx_free
Toolkit (iZ/iZm), 61
vx_full_sieve
Toolkit (iZ/iZm), 61
vx_init
Toolkit (iZ/iZm), 62
vx_prob_sieve
iZ_toolkit.c, 143
vx_random_prime
iZ Public API, 45

vx_search_prime
Toolkit (iZ/iZm), 63
VX_SEG, 93
bit_ops, 93
end_x, 93
is_large_limit, 94
mr_rounds, 94
p_count, 94
p_gaps, 94
p_test_ops, 94
root_limit, 95
start_x, 95
vx, 95
x5, 95
x7, 95
y, 96
yvx, 96
vx_set_base_values
iZ_toolkit.c, 144
vx_stream_file
Toolkit (iZ/iZm), 64
vy_random_prime
iZ Public API, 46
vy_search_prime
Toolkit (iZ/iZm), 64

x5
VX_SEG, 95
x7
VX_SEG, 95

y
VX_SEG, 96
yvx
VX_SEG, 96